

预测驱动的微网购电分层滚动优化与日内合同调整策略

摘 要

针对负荷、光伏与电价不确定、储能跨时段耦合及合同日内不可修改等问题，本文在非前视信息约束下，构建分层滚动优化框架，结合历史点预测、残差经验分位风险修正、线性规划（LP）和模型预测控制（MPC）；结算时使用真实负荷与光伏（问题四另使用真实电价）评估各方案。

问题一基于典型日全信息建立确定性 LP 模型，考虑功率平衡、储能 SOC 递推、充放电功率上限和日始末电量闭环。求得典型日购电量为**59482.70 kWh**、购电费用为**35126.95 元**，较无储能情形的**61789.94 kWh**下降 3.73%。问题二拓展至全年 334 天，比较 F1/F2 历史预测器，并以净负荷残差经验分位构造安全轨迹；由报童模型得到理论分位 0.8，主方案取 $W = 35, \alpha = 0.805, H = 49$ ，年度现金费用为**14786101.35 元**，应急购电量为**245483.29 kWh**。

问题三构建“日前合同优化—日内滚动 LP 执行”的两层框架，在 0:00、6:00、12:00 和 18:00 依据新光伏预报与实际储能状态修正未执行合同。主方案取 $W = 35, \alpha = 0.7725, H = 6$ ，年度费用为**14664345.40 元**，较不使用日内新预报的方案节省**566030.72 元**，单次合同更新的盈亏平衡成本约为**564.90 元**。问题四进一步考虑只能由历史数据预测的分时波动电价，对比固定合同与可调合同：可调合同费用为**15465962.60 元**，较固定合同的**17378687.69 元**下降 11.01%，应急购电量下降 63.14%；相对已知真实电价的理想下界 15383049.81 元，仅增加 0.54%。

结果表明，日内合同调整能够同时降低年度现金费用和应急购电风险；各方案均满足功率平衡、储能边界及跨日连续性约束，相关残差处于 10^{-13} 量级。敏感性分析显示，储能功率上限属于高敏感参数，预测误差倍率的影响具有非单调性。在本文数据范围内，该框架可作为含储能园区微网的调度基线，迁移到其他年份或市场规则时需重新标定参数。

关键词：微网购电；线性规划；风险修正；滚动优化；合同调整；波动电价

一 问题重述与分析

1.1 问题背景与任务拆解

微网由光伏、负荷、储能和外购电通道组成，需在预测误差、合同计费和储能约束下平衡常规购电与高价应急购电。相关研究通常将各设备纳入统一的能量管理优化框架[1, 2]。全年数据结构与季节性特征如图1所示。

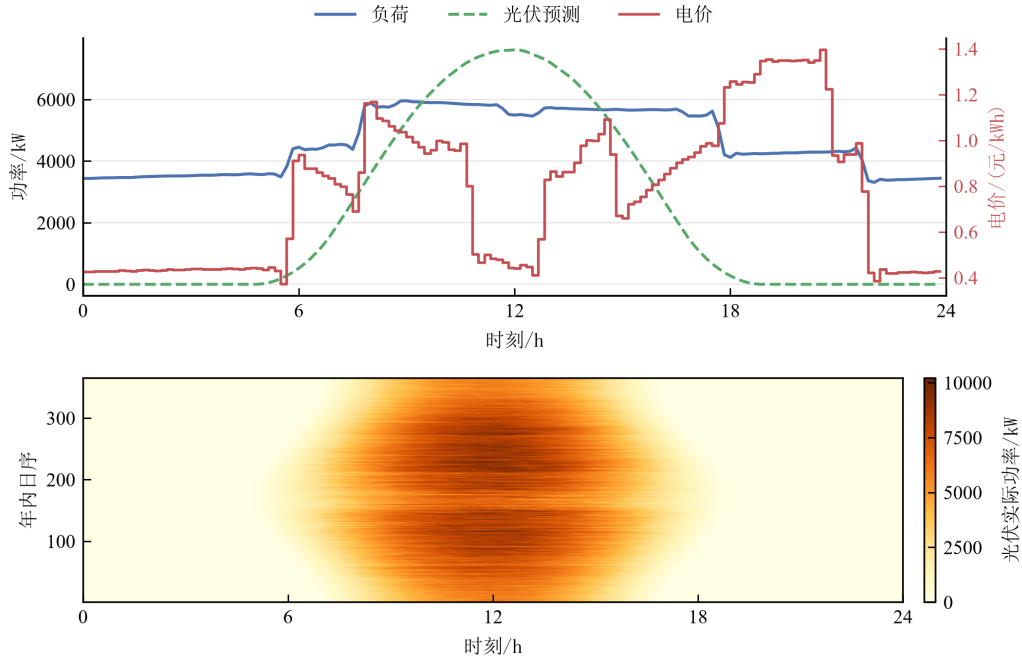


图 1: 全年负荷、光伏与电价数据概览及光伏季节性。

1.2 数据画像与预处理

官方全年数据以 10 分钟为间隔，未发现缺失值或越界记录，故不作插值和删改。功率按 $E = P\Delta t$ 转换为时段电量，其中 $\Delta t = 1/6$ h；第 t 行对应区间 $[(t-1) \times 10, t \times 10)$ min。所有预测只使用决策前已经获得的信息，1 月用于预热，2–12 月用于正式统计。

基于这些数据，四个问题依次处理“确定性基准—日前计划—日内合同修正—波动电价扩展”四个层次。建模与求解流程如图2所示。

具体递进关系如下：

1. 问题一建立典型日确定性调度基准；
2. 问题二扩展到仅在每日 0:00 签约的全年日前调度；
3. 问题三利用 0:00、6:00、12:00 和 18:00 预报调整未执行合同；
4. 问题四加入波动电价，比较固定合同与可调合同。

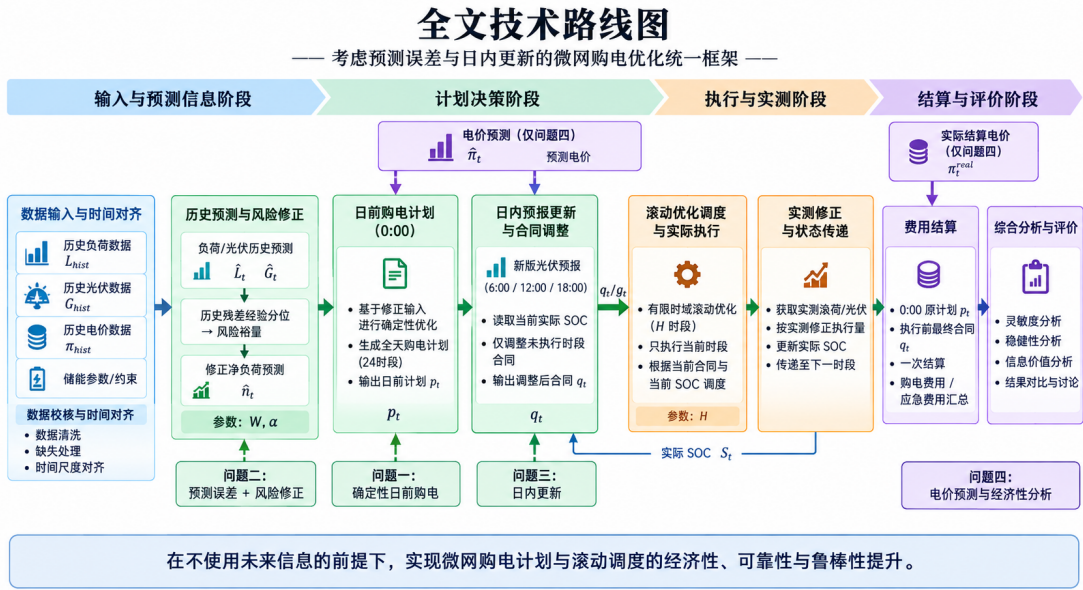


图 2: 全文建模与求解技术流程

各全年方案均从 1 月 1 日 6000 kWh 的初始储电量启动。1 月用于积累历史样本并使储能状态进入连续运行过程，2 月 1 日至 12 月 31 日的 334 天用于正式统计；预热期仍执行完整预测、优化和结算流程。预测数据不足时按“只取已发生观测”降级： $d \geq 7$ 时，F1 负荷取 $L_{d-7,t}$ 、光伏取前 7 日同一时段均值； $1 \leq d \leq 6$ 时取此前已有日的同一时段均值； $d = 0$ 时采用附件 1 典型日数据。早期残差不足时风险修正项取零，因而全过程均满足非前视要求。

总购电费用是主要评价指标，同时报告应急购电量、闲置合同电量、弃光量和储能终态；预测误差用 MAE、RMSE 等指标衡量。模型的关键在于：只能使用决策时刻可得的信息，同时处理预测误差与储能跨时段约束。

二 模型假设、数据口径与基础物理模型

2.1 模型基本假设

模型遵循题目计费规则，并作如下假设以便复现：

- 交易与费用：**微网只能购电，不能售电；多余光伏可弃用，未消纳的合同电量仍按原价计费。模型只统计合同购电费和应急购电费。
- 储能参数：**容量为 12000 kWh，运行范围为 [1200, 10800] kWh；充、放电功率上限均为 5000 kW，效率均为 0.9。
- 充放电互斥：**同一时段不同时充、放电。规划层按命题 1 处理，执行层按命题 3 处理，并加入极小的充放电总量惩罚项。

4. **信息边界**：预测、购电计划和合同调整只能使用决策时刻之前已经获得的信息，不能使用未来真实负荷、光伏或电价。

2.2 统一时间口径与量纲转换

单时段为 10 分钟，即 $\Delta t = 1/6$ h。功率按 $E = P\Delta t$ 换算为电量，负荷、光伏、合同及储能动作均以 kWh 计；第 t 行数据对应区间 $[(t-1) \times 10, t \times 10)$ min。仿真和 SOC 更新均按这一时间口径执行。

2.3 基础功率平衡物理模型

记 $L_t, V_t, g_t, w_t, v_t, x_t, y_t, u_t, S_t$ 分别为负荷、可用光伏、合同购电、闲置合同、光伏消纳、充电、放电、应急购电和时段初储电量。

时段能量平衡方程：

$$g_t - w_t + v_t + y_t + u_t = L_t + x_t$$

储能荷电状态递推关系：

$$S_{t+1} = S_t + \eta_c x_t - \frac{y_t}{\eta_d}, \quad \eta_c = \eta_d = 0.9$$

系统约束条件：

$$\begin{cases} 0 \leq v_t \leq V_t, & 0 \leq w_t \leq g_t, & u_t \geq 0 \\ 0 \leq x_t, y_t \leq B_{\max} \\ 1200 \leq S_t \leq 10800 \end{cases}$$

2.4 模型符号说明

表 1: 模型符号说明

符号	单位	含义说明
决策变量		
g_t	kWh	时段 t 实际生效的合同购电量。问题一中为确定性购电量。
p_t	kWh	每日 0:00 签订的日前合同购电量。
q_t	kWh	问题三中经过日内调整后最终生效的合同购电量。
δ_t^+, δ_t^-	kWh	合同上调量和下调量，分别表示对尚未执行合同的增加量与减少量。
v_t	kWh	实际用于供给负荷或储能充电的光伏电量，满足 $0 \leq v_t \leq V_t$ 。
x_t, y_t	kWh	储能充电量和放电量。
u_t	kWh	真实执行阶段为弥补供电缺口而购买的应急电量。
w_t	kWh	已签合同中未被实际消纳的闲置电量，按合同价格计费。
参数与状态量		
L_t	kWh	时段 t 的实际负荷电量。
V_t	kWh	时段 t 可利用的光伏电量上限。

符号	单位	含义说明
S_t	kWh	时段 t 开始时的储能电量。
$S_{\min}, S_{\max}$	kWh	储能允许的最低、最高电量，本文分别为 1200 和 10800。
$B_{\max}$	kWh	单时段充、放电电量上限， $B_{\max} = P_{\max} \Delta t$ 。
c_t	元/kWh	时段 t 的计划购电单价，应急购电单价为 $5c_t$ 。
η_c, η_d	—	储能充电效率和放电效率，本文均取 0.9。
$\hat{L}_{d,t}, \hat{V}_{d,t}$	kWh	负荷和光伏的中心点预测值。
$\tilde{L}_{d,t}, \tilde{V}_{d,t}$	kWh	加入风险裕度后用于合同规划的保守预测值。
$\hat{N}_{d,t}, \tilde{N}_{d,t}$	kWh	中心净负荷预测和安全净负荷预测，其中 $N = L - V$ 。
$e_{d,t}^L, e_{d,t}^V, e_{d,t}^N$	kWh	负荷、光伏和净负荷的预测残差。
Q_α	kWh	历史残差的 α 分位数，用于构造风险裕度。
$J(p)$	元	单时段合同购电与应急购电组成的期望费用函数。
A_t	kWh	预测盈余，定义为合同购电加光伏预测减去负荷预测。
W	日	计算经验分位数时使用的历史残差窗口长度。
α	—	风险分位水平。
H	时段	滚动 LP 每次向前看的时域长度。
λ	元/kWh	终端储能价值系数，仅用于引导优化，不计入实际费用。
ε	元/kWh	充放电总量的极小惩罚系数，用于选取数值上不同时充、放电的解。
集合与信息		
d, t, j, h	—	日、10 分钟时段、历史日和预测提前期索引。
$\mathcal{H}_t$	—	从当前时段 t 开始的滚动优化时域。
$I_{d,t}$	—	第 d 日第 t 时段决策时已经获得的信息集。
$\mathcal{F}_{d,h}$	—	第 d 日提前期为 h 时可使用的历史预测信息集。

信息集定义为

$$I_{d,t} = \{\text{时隙 } t \text{ 前已完成的负荷、光伏与电价观测, } S_{d,t}, \\ \text{截至 } t \text{ 已发布的预测、合同与模型参数}\}.$$

因此， $I_{d,t}$ 不包含第 d 日尚未发生时隙的真实负荷、光伏或电价。所有计划与调度变量必须是 $I_{d,t}$ 的函数；真实值仅在该时隙决策完成后进入执行修正与结算。

三 问题一：确定性典型日购电优化模型

3.1 问题分析与模型构建：确定性典型日基准调度场景

问题一给出完整典型日的负荷、光伏出力和分时电价，调度开始时已知全部数据；储能还需满足日内电量闭环。因此，本题可作为确定性线性规划基准算例，用来检查微网基础模型。模型求解调用 SciPy 提供的线性规划工具 [8]。

在通用微网物理模型基础上，本题关闭应急购电 $u_t = 0$ 和闲置弃购量 $w_t = 0$ ，并施加储能日闭环约束 $S_1 = S_{145} = 6000$ kWh (S_t 为 t 时段末储能荷电量)。功率平衡与

储能状态递推约束如下：

$$g_t + v_t + y_t = L_t + x_t, \quad (1)$$

$$S_{t+1} = S_t + 0.9x_t - \frac{y_t}{0.9}, \quad (2)$$

$$0 \leq v_t \leq V_t, \quad 0 \leq x_t, y_t \leq B_{\max}, \quad (3)$$

$$1200 \leq S_t \leq 10800. \quad (4)$$

式中：\$g_t\$ 为 \$t\$ 时段电网购电量；\$v_t\$ 为本时段光伏出力；\$y_t\$ 为储能放电电量；\$L_t\$ 为负荷电量；\$x_t\$ 为储能充电电量；\$V_t\$ 为光伏最大可用出力。

模型主目标为最小化全天购电费用：

$$\min \sum_{t=1}^{144} c_t g_t. \quad (5)$$

其中 \$c_t\$ 代表 \$t\$ 时段购电单价。

仅以费用作为目标时，最优解集合内可能出现无物理意义的微小同时充放电数值解。为此采用两阶段优化策略：第一步求解得到最优理论费用 \$f^*\$；第二步在总购电费用不超过 \$f^* + \varepsilon\$ (\$\varepsilon = 10^{-4}\$ 元) 的约束下，以最小化储能充放电总量作为次级目标：

$$\min \sum_{t=1}^{144} (x_t + y_t) \quad \text{s.t.} \quad \sum_{t=1}^{144} c_t g_t \leq f^* + \varepsilon. \quad (6)$$

两级问题均为线性规划，求解器可分别给出全局最优解。

命题 1 (连续 LP 松弛的可行性) 在电价非负、微网不可售电、无充放电补贴且充放电效率不超过 1 的条件下，费用最优解集合中必存在一个不同时充、放电的解。因此，无须在基础模型中引入充放电 0-1 互斥变量。

证明 若某费用最优解在时段 \$t\$ 满足 \$x_t > 0, y_t > 0\$，取充分小的 \$\delta > 0\$，令 \$x'_t = x_t - \delta/\eta_c\$、\$y'_t = y_t - \delta\eta_d\$。则储能变化量不变；因 \$\eta_c\eta_d \leq 1\$，功率平衡所需的外购电量不增加，费用也不增加。重复该操作，直至 \$x_t = 0\$ 或 \$y_t = 0\$，即可得到同样费用且不同时充、放电的可行解。逐时段处理即得结论；第二阶段的充放电总量最小化仅用于选取这一类数值解。证毕。

命题 1 的证明不依赖问题一的具体场景，只要求外购电量以正单价线性计入目标函数且为自由变量。因此，它也适用于问题二至四的日前规划层（将 \$g_t\$ 替换为 \$p_t\$ 或 \$q_t\$）；执行层的合同费用已确定、\$u_t\$ 显式定价，则由命题 3 单独处理。

3.2 仿真求解与结果验证：基准模型校验

求解得到典型日全天计划购电量 **59482.70 kWh**，总购电费用 **35126.95 元**。按题目表 1、表 2 格式整理的指定时段购电量与分段储能结果见附录 A；完整全天调度时序曲线见图 3。

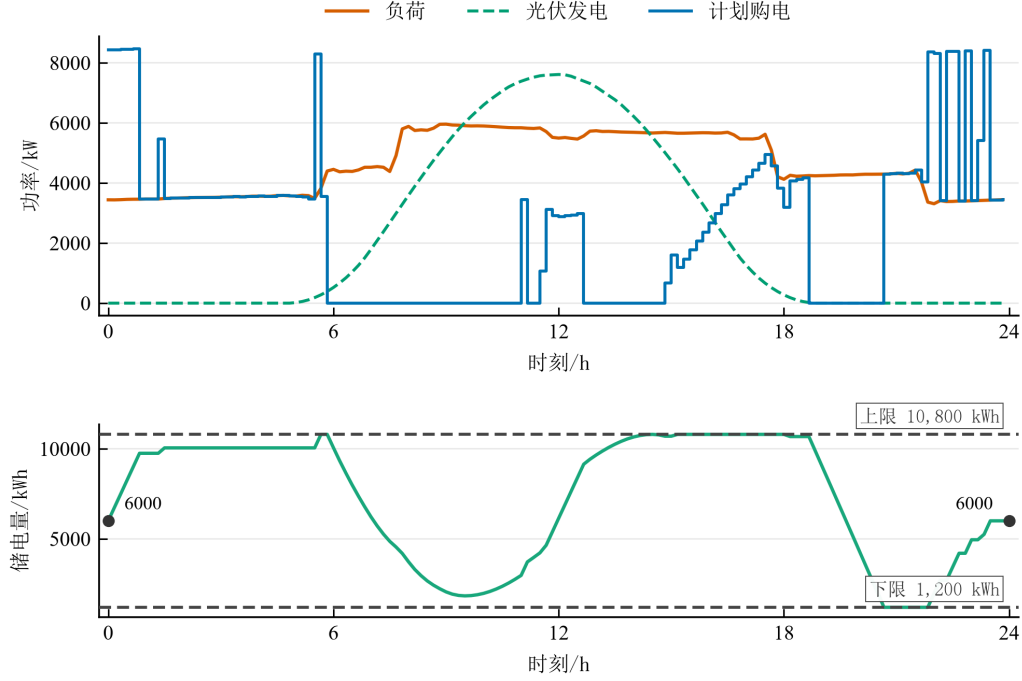


图 3: 典型日微网购电、储能充放电与荷电状态时序曲线

调度结果体现储能低电价充电、高电价放电的套利特性：凌晨低电价时段购电并给储能充电；光伏大发时段依靠光伏出力供给负荷，部分时段购电量降至 0；晚高峰电价抬升时，储能集中放电，降低高价时段购电需求。全天总充电量 20740.66 kWh，总放电量 16799.94 kWh，光伏全部就地消纳，无弃光。

逐时段功率平衡与 SOC 递推残差处于 10^{-13} 量级，属于求解器浮点计算误差范围；储能荷电量全程保持在安全区间内，并满足日初-日末 SOC 闭环约束。这说明基础微网模型满足预期约束，可作为问题二至四不确定性调度的基准。

四 问题二：基于日初预测的年度日前购电优化

4.1 问题分析与模型构建：负荷与光伏预测不确定性下的日前购电调度

问题一基于典型日完全信息得到确定性调度基准；问题二将模型拓展至全年连续仿真，只允许每日 0:00 利用当前信息签订全天日前购电合同，合同签订后日内不可调整。若出现功率缺口，缺口部分按 5 倍电价应急购电。本问题的核心权衡是：合同电量偏低会触发高额应急购电成本，合同过量则产生无法退回的闲置合同费用。

本文对问题二的处理分为五个环节：先比较两组只使用历史数据的点预测方案并确定预测器；再对同一预测版本下的净负荷残差作逐时段经验分位修正，得到安全轨迹；随后以安全轨迹、真实日初储电量与分时电价求解 0:00 日前合同线性规划；日内以预测驱动的滚动时域线性规划（MPC 执行器）执行储能动作；最后代入真实负荷与光伏完成裁剪、平衡与结算。技术流程如图4所示。

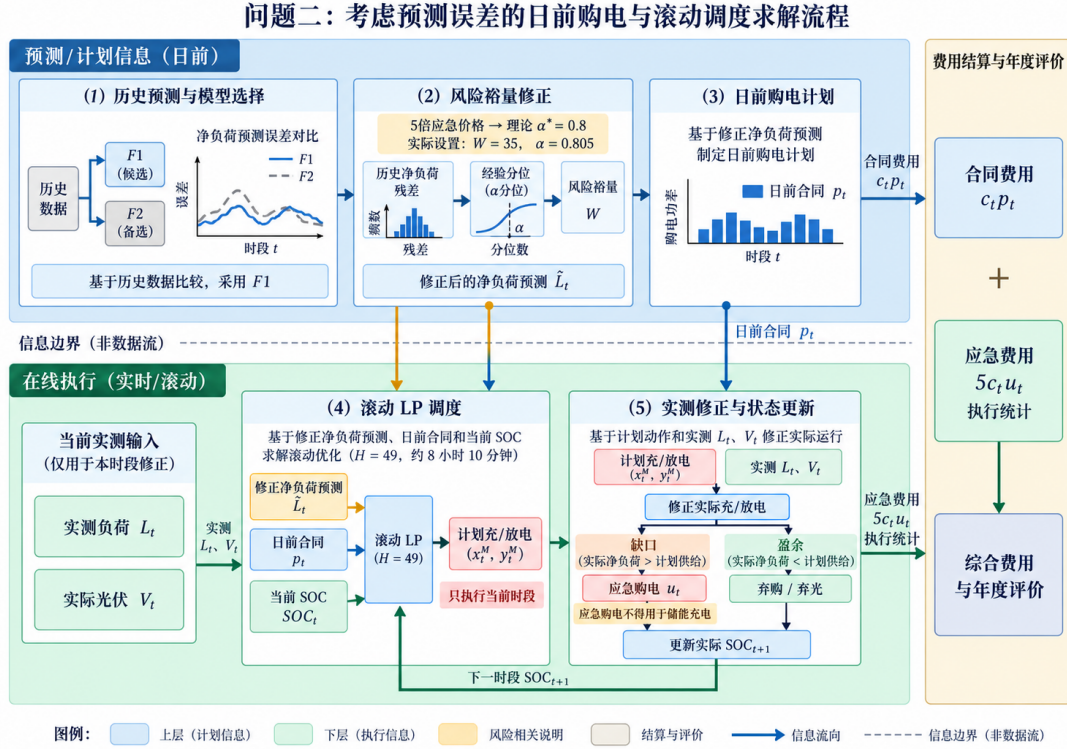


图 4: 问题二预测驱动日前购电与 MPC 执行流程

4.2 历史预测方案对比与选型依据

本文比较两种只使用目标日前历史观测的负荷与光伏预测方法, 记目标日为 d 、时段为 t 。负荷和光伏预测的常用方法及误差来源可参考相关综述 [3, 4]。F1 为主预测器: 负荷采用一周前同一时段的观测值, 光伏采用前七日同一时段的均值, 分别为

$$\hat{L}_{d,t}^{F1} = L_{d-7,t}, \quad (7)$$

$$\hat{V}_{d,t}^{F1} = \frac{1}{7} \sum_{k=1}^7 V_{d-k,t}. \quad (8)$$

F2 仅将负荷改为前三个滞后周同一时段观测值的中位数, 以减弱单周异常影响; 光伏预测与 F1 相同:

$$\hat{L}_{d,t}^{F2} = \text{median}(L_{d-7,t}, L_{d-14,t}, L_{d-21,t}). \quad (9)$$

历史样本不足时按实际可用数据构造预测; 1 月初无法取得完整滞后样本时的降级处理, 统一按第 1.2 节的规则执行。

点预测精度用平均绝对误差与均方根误差衡量:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |z_i - \hat{z}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (z_i - \hat{z}_i)^2}. \quad (10)$$

两种预测方案在 2025 年 2 月 1 日至 12 月 31 日上的误差如表 2 所示。

表 2: 历史预测方案的误差对比

预测器	负荷 MAE	光伏 MAE	净负荷 MAE	净负荷 RMSE
F1	176.80	153.40	273.35	389.97
F2	187.90	153.40	284.35	403.25

单位: kW。

由表2可得, F1 的净负荷 MAE 与 RMSE 均低于 F2; 二者光伏预测完全一致, 因此差异可归因于负荷预测方法。基于上述误差及第4.5节统一执行、结算口径下的闭环费用, 本文选取 F1 为主预测器。

4.3 净负荷残差的经验分位风险修正

记中心净负荷预测为 $\hat{N}_{d,t} = \hat{L}_{d,t} - \hat{V}_{d,t}$, 同一预测版本下已完成日 $j < d$ 的逐时段残差定义为

$$e_{j,t}^N = N_{j,t} - \hat{N}_{j,t}, \quad j < d. \quad (11)$$

使用目标日之前最近 W 个已完成日的同一时段残差经验分位构造安全净负荷:

$$\tilde{N}_{d,t} = \hat{N}_{d,t} + Q_\alpha \{e_{j,t}^N : d - W \leq j < d\}. \quad (12)$$

式(12)直接对残差的经验分布取分位, 不要求残差服从正态分布, 也不需要额外的场景生成步骤; W 控制所使用历史误差的时间范围, α 控制风险裕度的位置。加入分位裕度后净负荷可能出现负值, 为保持重构后的负荷与光伏非负, 按

$$\tilde{V}_{d,t} = \max(\hat{V}_{d,t}, -\tilde{N}_{d,t}, 0), \quad \tilde{L}_{d,t} = \tilde{V}_{d,t} + \tilde{N}_{d,t} \quad (13)$$

将其重新表示为非负的负荷预测与光伏预测。该处理只改变净负荷的分解方式, 不改变 $\tilde{N}_{d,t}$ 本身, 因而不影响供需缺口的计算。

分位水平 α 由计划电价与应急购电价格的比值确定。考虑单个时段, 计划购电价格为 $c > 0$ 、计划购电量为 p , 该时段净负荷 N 为随机变量, 剩余缺口按 $5c$ 应急购电, 则该时段的期望费用为

$$J(p) = cp + 5c \mathbb{E}[(N - p)^+], \quad (N - p)^+ = \max(N - p, 0). \quad (14)$$

命题 2 (5 倍应急价格对应的报童临界分位) 在合同单位成本为 $c > 0$ 、缺电应急购电单价为 $5c$ 、富余合同不退款的条件下, 使式(14)最小的计划购电量为 N 的 0.8 分位, 即 $p^* = Q_{0.8}(N)$ 。

证明 记 N 的分布函数为 $F(\cdot)$ 。对式(14)关于 p 求导,

$$J'(p) = c - 5c \mathbb{P}(N > p) = c[1 - 5(1 - F(p))].$$

令 $J'(p) = 0$ 得 $F(p) = 1 - c/(5c) = 0.8$, 即 $p^* = Q_{0.8}(N)$ 。再对 $J'(p)$ 求导得 $J''(p) = 5cf(p) \geq 0$, 故 $J(p)$ 为凸函数, 该驻点即全局最小值点; N 取离散分布时按次梯度理解, 结论不变。证毕。

命题 2 的边际含义是: 多购 1 kWh 计划电需支付 c , 少购 1 kWh 计划电则以 $1 - \alpha$ 的概率支付 $5c$, 两种边际代价相等时分位水平为 $1 - 1/5 = 0.8$ 。实际计算中净负荷的分布未知, 本文用最近 $W = 35$ 个已完成日的同一时段历史残差经验分位数实现该分位水平; 受有限样本影响, 分位取值与理论值存在小幅差别, 主配置取 $\alpha = 0.805$ 。该分位仅表示单时段风险基准: 命题 2 针对单个时段的边际分布, 0.8 并不表示整日供电可靠度为 80%, 也不保证计入储能跨时段耦合后合同量随分位单调变化, 因此 W 与 α 的最终取值由第 4.5 节的参数敏感性确定。以夏至日为例, 分位风险修正前后的净负荷预测与实际净负荷对比如图 5 所示。

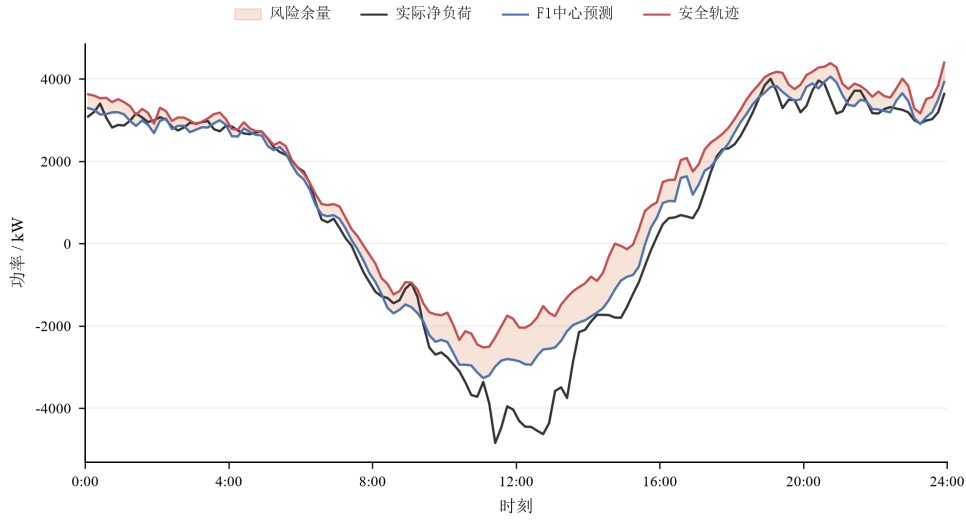


图 5: 分位风险修正预测、风险裕量与实际净负荷 (以 2025-06-21 为例)

4.4 日前合同优化与 MPC 滚动执行规则

每天 0:00 以真实日初储电量为初值、以安全轨迹为输入求解日前合同。规划层令生效合同量 $g_t = p_t$, 并沿用前文的能量平衡、储能递推和设备边界约束:

$$\min \sum_{t=1}^{144} c_t p_t - \lambda (S_{145}^P - S_{\min}), \quad (15)$$

式(15)的第一项表示日前合同费用; 第二项为有限时域末储能状态设置的机会价值, 用于减轻优化时域截断造成的过度放空, λ 仅用于引导求解, 不计入实际现金费用。规划层不设置可自由替代的应急购电变量, 风险只通过安全轨迹进入合同决策, 5 倍应急电价在真实执行和结算阶段体现。

问题二按日结算的实际现金费用为

$$C_2 = \sum_{t=1}^{144} (c_t p_t + 5c_t u_t). \quad (16)$$

计划购电量无论是否全部消纳均按合同计费，弃购成本已包含在 $c_t p_t$ 中； $5c_t u_t$ 仅在真实执行出现供电缺口时产生。全年 334 天闭环仿真中，将各日的 C_2 累加得到现金总费用 **14786101.35 元**，其中计划购电费用为 **13665855.74 元**，应急购电费用为 **1120245.61 元**。由于题目未设并网功率上限，若在日前规划中加入可自由替代的应急变量，模型会倾向于以应急购电替代合同购电，从而失去风险合同规划的含义。因此，本文将合同规划与真实闭环结算分开处理。

合同签订后日内不可调整，储能动作由预测驱动的滚动时域线性规划（MPC 执行器，下文简称滚动 LP）给出。合同费用在签订时已经确定，因此滚动 LP 只优化预测的应急购电费用、储能终端价值和极小充放电总量惩罚。在时段 t ，滚动 LP 以 $\mathcal{H}_t = \{t, \dots, \min(t + H - 1, 144)\}$ 为范围，取 $H = 49$ （约 8 小时 10 分钟），在 $I_{d,t}$ 给定的信息下安排充放电、光伏利用、弃购、应急购电与储电量：

$$\min \sum_{\tau \in \mathcal{H}_t} 5\hat{c}_\tau u_\tau + \varepsilon \sum_{\tau \in \mathcal{H}_t} (x_\tau + y_\tau) - \lambda S_{\text{end}}, \quad (17)$$

其中约束为通用能量平衡、储能递推与设备运行边界， $\hat{c}_\tau$ 为当前可用的电价预测。每次求解只执行当前时段动作，下一时段再用更新后的真实储电量重新计算； ε 为极小罚项，用于在多重最优解中抑制同时充放电的数值伪解。

由于执行层的合同费用已经确定，命题 1 关于规划层自由购电变量的结论不能直接套用。对执行层， $\varepsilon(x_\tau + y_\tau)$ 用于选取无同时充放电的数值解，并有如下条件性说明。

命题 3（滚动执行层的储能互斥性，附条件） 考虑窗口 $\mathcal{H}_t$ 内任一时段 τ 的滚动 LP（式(17)）及其功率平衡、SOC 递推约束。若某最优解在 τ 处满足 $x_\tau > 0, y_\tau > 0$ ，且下列两个条件至少有一个成立，则与最优性矛盾：

- (A) 当期存在吸收余量： $w_\tau < g_\tau$ ，或 $v_\tau > 0$ （此时 $V_\tau > 0$ ），或 $u_\tau > 0$ ；
- (B) 电池有下游余量：存在 $\delta > 0$ ，使从 τ 到窗口末尾的所有时段，将 SOC 抬高 $\delta(1/\eta_d - \eta_c)$ 后仍不超过 S_{max} 。

因此，在上述任一条件下，任一最优解均满足 $\min(x_\tau, y_\tau) = 0$ 。

证明 先取充分小的 $\delta > 0$ ，令 $x'_\tau = x_\tau - \delta/\eta_c$ 、 $y'_\tau = y_\tau - \delta\eta_d$ 。由

$$\eta_c x'_\tau - y'_\tau / \eta_d = \eta_c x_\tau - y_\tau / \eta_d$$

知 τ 及其后 SOC 轨迹不变， $-\lambda S_{\text{end}}$ 项不受影响。功率平衡左右两侧之差为 $\delta(1/\eta_c - \eta_d) > 0$ ，需由 w_τ, v_τ, u_τ 之一吸收；若条件 (A) 成立，取相应变量的吸收该差额后仍可行。若

由 w_τ 或 v_τ 吸收, 应急购电项不变; 若由 u_τ 吸收, 该项进一步减少。无论何种情形, $\varepsilon(x_\tau + y_\tau)$ 项均严格减少 $\varepsilon\delta(1/\eta_c + \eta_d)$, 目标函数严格改进, 与最优性矛盾。

若条件 (A) 不成立, 即 $w_\tau = g_\tau, v_\tau = 0, u_\tau = 0$ 同时发生, 则功率平衡退化为 $y_\tau = L_\tau + x_\tau$ 。取 $\delta \leq \min(x_\tau, y_\tau)$, 令 $x'_\tau = x_\tau - \delta, y'_\tau = y_\tau - \delta$, 功率平衡两侧同减 δ 而保持不变, SOC 增量变为 $\delta(1/\eta_d - \eta_c) > 0$; 若条件 (B) 成立则仍可行。此时 $\varepsilon(x_\tau + y_\tau)$ 项减少 $2\varepsilon\delta$, 且 S_{end} 增大使 $-\lambda S_{\text{end}}$ 项不增, 目标函数同样严格改进, 仍与最优性矛盾。证毕。

说明 当两个条件均不满足时, 可能出现外部吸收渠道和电池余量同时受限的特殊边界情形。对此, 本文不再引入额外的整数变量或复杂的互斥约束, 而是在目标函数中加入极小的充放电总量惩罚项, 以较低的建模和求解代价选取无同时充放电的解, 同时保持滚动 LP 的线性结构。第七章的全量求解结果均满足 $\min(x_t, y_t) = 0$, 表明这一简化处理在全年闭环仿真中未造成实际偏差。

设滚动 LP 给出的当前时隙动作记为 (x_t^M, y_t^M) 。本时隙真实负荷与光伏到达后, 只允许在已决定动作的基础上缩减:

$$x_t^E = \min \{x_t^M, (g_t + V_t - L_t)^+\}, \quad (18)$$

$$y_t^E = \min \{y_t^M, (L_t - g_t - V_t + x_t^E)^+\}, \quad (19)$$

$$b_t = L_t + x_t^E - y_t^E - g_t - V_t. \quad (20)$$

若 $b_t \geq 0$, 则 $u_t = b_t, w_t = 0$; 若 $b_t < 0$, 则 $u_t = 0$, 并将 $-b_t$ 依次解释为未利用合同电量与弃光电量。式(18)至式(20)保证应急电只补实际缺口、不用于主动充电, 同时禁止执行层依据本时隙实测缺口临时增加放电, 使两种执行器在同一信息集下可比。时隙末真实储电量由储能递推关系传入下一时隙。

作为同信息集对照, 本文另设预测驱动贪心执行器。令预测盈余

$$A_t = g_t + \tilde{V}_{d,t} - \tilde{L}_{d,t},$$

若 $A_t \geq 0$, 则盈余用于充电:

$$x_t = \min \left\{ A_t, B_{\max}, \frac{S_{\max} - S_t^E}{\eta_c} \right\}, \quad y_t = u_t = 0;$$

若 $A_t < 0$, 则先放电、缺口再应急:

$$y_t = \min \{ -A_t, B_{\max}, (S_t^E - S_{\min})\eta_d \}, \quad u_t = -A_t - y_t, \quad x_t = w_t = 0.$$

贪心只依据当前一个时隙的安全轨迹决定动作, 同样在动作确定之后才读取实测值并调用式(18)至式(20)的统一裁剪器。两种执行器共享同一合同、预测方法、风险裕度、储能约束与结算规则, 唯一差别是滚动 LP 额外利用了未来 H 个时隙的价格与安全轨迹信息。贪心执行器作为同信息集下的可行基线, 不代表跨时段最优; 年度正式结果以滚动 LP 为主执行器。

4.5 年度仿真结果与对比分析

334 天仿真结束后，年度关键指标如表3所示。

表 3: 问题二年度运行汇总结果（2-12 月）

指标	数值	指标	数值
计划购电量	22382650.65 kWh	计划购电费用	13665855.74 元
应急购电量	245483.29 kWh	应急购电费用	1120245.61 元
应急购电发生时段数	6822	闲置合同电量	1596110.94 kWh
弃光电量	1741589.34 kWh	储能充电量	7112133.95 kWh
储能放电量	5760889.78 kWh	期末储电量	10731.91 kWh
年度现金总费用			14786101.35 元

由表3可得，全年计划购电量为 **22382650.65 kWh**，现金总费用为 **14786101.35 元**，其中计划购电费用为 **13665855.74 元**、应急购电费用为 **1120245.61 元**。应急购电量为 **245483.29 kWh**，仅占合同电量的 1.10%，但对总费用的贡献达到 7.58%，说明风险修正将部分预测误差损失转化为可控的合同冗余。全年弃购 **1596110.94 kWh**、弃光 **1741589.34 kWh**；储能全年充电 **7112133.95 kWh**、放电 **5760889.78 kWh**，期末储电量为 **10731.91 kWh**，全程满足安全边界。按月统计，12 月费用最高，6 月应急购电量最高，4 月费用最低，反映了负荷和光伏的季节性影响。

为检验主配置的稳健性，对预测器、残差窗口、分位水平与滚动时域逐一重跑全年闭环仿真，结果如表4所示。

表 4: 问题二预测器与参数敏感性汇总（2-12 月）

类别	设置	年度费用/元	应急购电/kWh	费用变化/元
主配置	F1, $W = 35, \alpha = 0.805, H = 49$	14786101.35	245483.29	0.00
预测器	F2	14788842.58	249032.38	+2741.23
风险分位	$\alpha = 0.800/0.810$	14820043.34/14786818.42	289390.70/245477.85	+33941.99/+717.07
残差窗口	$W = 30/45$	14837364.77/14815783.49	247334.59/260994.56	+51263.42/+29682.14
滚动时域	$H = 48/50$	14786354.19/14786319.50	245523.92/245518.43	+252.84/+218.15

表4将预测器、风险分位、残差窗口和滚动时域的试验统一列示。各备选配置相对主方案的费用变化不超过 0.35%；其中 H 的小幅变化影响最小，F2 替换 F1 仅增加 2741.23 元，而较低分位和较短残差窗口会明显增加费用。因此采用 F1、 $W = 35$ 、 $\alpha = 0.805$ 和 $H = 49$ 作为主配置。

为评价风险修正策略的相对水平，另设两组参照方案开展全周期对照：理想信息参照假设调度日 0:00 完全已知真实负荷与光伏，仅作为事后不可实施的理论下界；无储能基线为理想净负荷信息下的无储能参照。结果如表5所示。

理想信息参照与主配置的差额 2495076.58 元，度量了全年负荷与光伏预测不确定性带来的总经济代价。无储能参照仅用于刻画“即使不给储能、但已知当期净负荷”时

表 5: 不同参照方案年度运行指标对比 (2-12 月)

方案	年度现金费用/元	应急购电费用/元	应急购电量/kWh
主配置 F1-W35- α 0.805-H49	14786101.35	1120245.61	245483.29
理想信息参照 (事后下界, 不可实施)	12291024.77	0.00	0.00
无储能基线	16407319.63	0.00	0.00

的参考费用，不用于严格分离储能边际贡献。该参照方案为规避缺口必须按更保守的口径采购合同电量，全年不产生应急购电，但现金费用高出主配置 1621218.28 元 (约 11.0%)。该对比说明储能的削峰填谷收益只有在合理预测与风险裕度配合下才能实现：储能本身并不能消除预测误差，它把误差从高价应急购电转移到合同冗余与充放电总量两侧，而这一转移是否盈利取决于分位裕度设置是否合理。

四个代表日的运行结果如表6与图6所示。

表 6: 问题二四个代表日 (春分、夏至、秋分、冬至) 运行结果

日期	日购电费用/元	计划购电量/kWh	应急购电量/kWh	日初-日末储电量/kWh
2025-03-20	45158.05	70894.49	542.74	10800.00-10795.24
2025-06-21	24282.59	41689.27	9.36	10800.00-10800.00
2025-09-23	47616.38	72083.06	600.29	10760.83-10793.77
2025-12-21	66632.03	101642.57	234.75	10756.53-10800.00

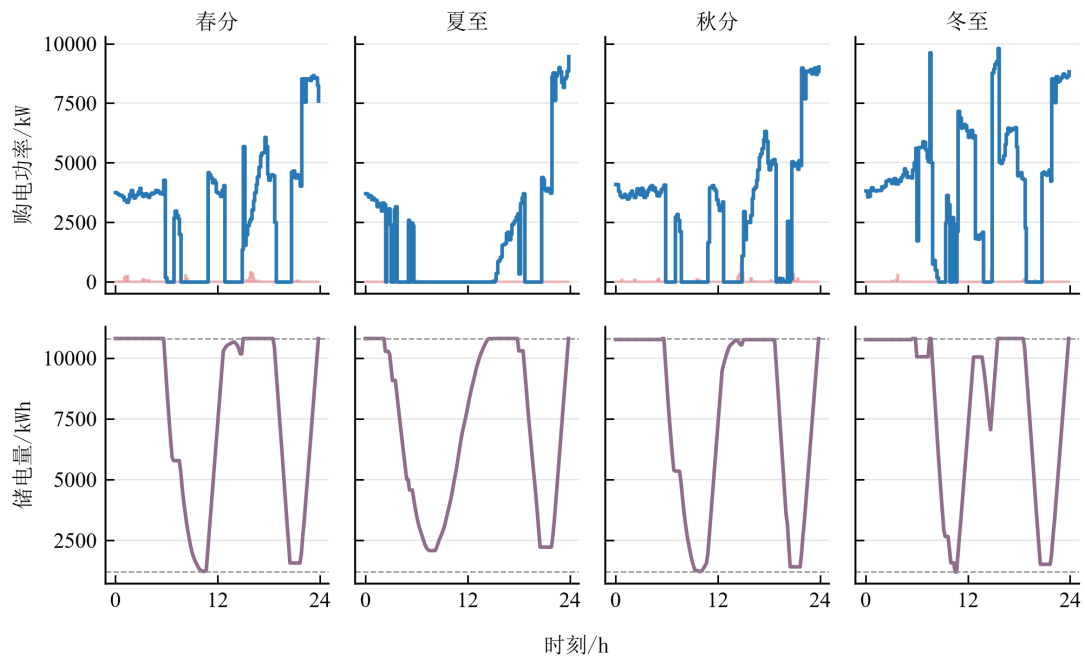


图 6: 春分、夏至、秋分、冬至四个代表日的购电功率与储电量

代表日结果与光伏季节性一致：12 月 21 日负荷高、光伏弱，日费用最高 (66632.03 元)；6 月 21 日光伏充足，日费用最低 (24282.59 元)；3 月 20 日与 9 月 23 日的应急购

电分别集中在凌晨与清晨的连续缺口时段，说明应急购电风险与日内负荷、光伏变化密切相关，而非由全天平均工况决定。上述代表日按题目要求整理的指定时段购电量、4小时充放电与储电量、应急购电区间见附录。

最后需要说明风险裕度的工程折中关系：裕度设置过高会抬高合同采购成本并增加弃购，裕度设置过低则应急购电风险上升。仿真会观测到同一日内同时存在闲置合同电量与应急购电的现象，根源在于日前合同按时隙固定，跨时段合同电量无法调剂；这属于赛题交易规则与逐时段负荷、光伏变化共同造成的客观结果，并非模型求解缺陷。主配置在计入该折中关系后，于表4的参数网格内取得最低现金费用，年度结果稳定可行。

五 问题三：考虑日内购电合同修正的微网滚动优化策略

5.1 问题分析：日内合同修正的经济价值

问题二的结果表明，在仅有日前固定合同、不允许日内调整的约束下，MPC 的跨时段优化增益有限，其收益大部分被负荷与光伏预测不确定性抵消。在此基础上，问题三引入日内购电合同修正机制。微网在日前市场依据光伏功率预测签订购电合同，得到计划购电量 p_t ；受光伏发电随机性与预测误差影响，日内光伏实际出力与日前预测存在显著偏差。若不允许对日前购电合同做任何修改，功率偏差只能依赖储能充放电平抑，储能触及 SOC 上下限后剩余偏差只能通过高价应急购电、闲置合同惩罚与弃光三类兜底手段承受，显著推高运行成本。

问题三要求在日内运行阶段引入合同修正机制：利用更新后的光伏预报，在合理约束内对尚未执行时段的购电合同适度修正，得到实际执行的最终购电合同 q_t ，以此削弱预测误差的不利影响。题目在每日 0:00、6:00、12:00、18:00 四个时点发布光伏预报，每个时段的最终购电量只相对 0:00 原计划结算一次，中间更新仅改变尚未执行时段的购电量。

为定量评估日内调账机制的经济价值，设置两组对照方案：主策略开启全部三个日内预报时点的合同调整；基线方案 $S = \emptyset$ 不使用任何日内新预报，仍保留相同的滚动调度方法与结算口径。两组方案采用完全一致的负荷、光伏时序、储能参数、价格体系与预测输入，仿真区间为 2025-02-01 至 2025-12-31，共计 334 天。

5.2 决策时序与信息边界

一天内共有四个预报版本，新预测只能在规定发布时间获得。0:00 版本用于形成初始计划购电量 p_t ；6:00、12:00 和 18:00 时，新版本只影响尚未执行的时段，已执行时段不再调整。每次预报更新先读取上一时段结束后的实际储能 SOC，再调整剩余时段的购电量，最后进入下一时段的滚动调度。

5.3 官方预报校正与分层风险

对目标日 d 、发布时刻 r 和提前期 h ，使用历史同版本误差对官方预报校正：

$$\hat{V}_{d,r,h}^{\text{cal}} = \max\left(0, \hat{V}_{d,r,h}^{\text{raw}} + \text{mean}_{j < d} e_{j,r,h}^V\right), \quad (21)$$

式中 $e_{j,r,h}^V$ 为历史第 j 日同发布时刻、同提前期的光伏预报残差。校正后插值到 10 分钟网格。不同提前期的预报误差存在差异，预测时域越长通常不确定性越大。若对每个提前期单独估计分位，会因样本不足引入较大统计噪声。本文按提前期将误差分为六层：0-1、1-2、2-3、3-4、4-5、5-6 小时，提前期超过 6 小时的远期时段统一归入第六层；0:00 版本覆盖的 6-24 小时部分及后续版本中仍超过 6 小时的远期时段均按第六层处理。令 s_h 为提前期 h 对应残差的经验均方根尺度，标准化残差 $z_{j,r,h} = e_{j,r,h}^V / s_h$ ，则风险裕量写为

$$R_{d,r,h} = s_h Q_\alpha \{z_{j,r,h} : d - W \leq j < d\}, \quad W = 35, \alpha = 0.77, \quad (22)$$

式中 Q_α 为经验分位函数。该方法保留“越远期误差通常越大”的结构，同时避免小样本下逐层估计的不稳定。校正后 1 小时、6 小时的误差 MAE 分别约为 19.38 kW 和 92.71 kW，其增长趋势为分层风险设置提供了直接数据依据。

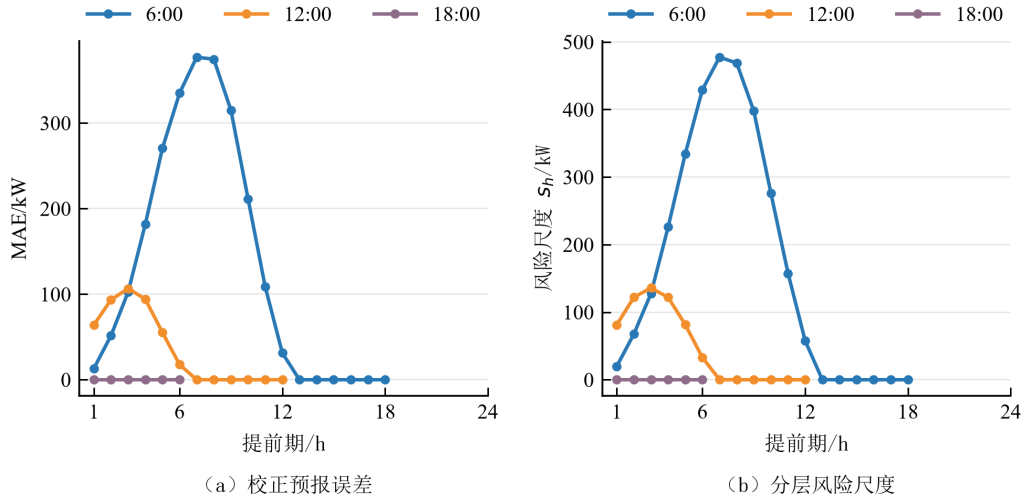


图 7: 问题三不同预报提前期的校正质量。

5.4 优化模型构建：MPC 滚动求解与合同调整规则

5.4.1 目标函数与结算规则

在 0:00 形成初始计划购电量 p_t ，6:00、12:00、18:00 保持已执行时段不变，以当时实际 SOC 重新计算剩余时段购电量。令

$$q_t = p_t + \delta_t^+ - \delta_t^-, \quad \delta_t^+, \delta_t^- \geq 0, \quad (23)$$

式中 δ_t^+, δ_t^- 分别为上调、下调量。剩余时域优化目标为

$$\min \sum_{t \in \tau_r} (c_t p_t + 1.5c_t \delta_t^+ - 0.5c_t \delta_t^- + 5c_t u_t) - \lambda_r (S_{145} - S_{\min}), \quad (24)$$

式中 c_t 为 t 时段购电单价, τ_r 为第 r 次更新时刻尚未执行的时段集合, λ_r 为终端储能价值罚项权重。 p_t 是 0:00 签订的原始合同量, q_t 是最后一次调整后实际生效的合同量。 u_t 为规划层对未来缺口的应急购电代理变量, 用于避免合同调整层无成本保留缺口; 真实结算仍采用实际执行后的应急购电量。最终调整购电量始终相对 0:00 计划购电量结算一次:

与问题二相同, 上述合同调整层中的 x_t, y_t 仅用于估计终端价值 $\lambda_r (S_{145} - S_{\min})$ 的规划代理值, 不是物理执行值。物理执行仍由第4.4节定义的同一滚动 LP 执行器给出: 该执行器在合同调整层每次更新 q_t 后, 以最新合同重新执行, 保证规划层与执行层在合同口径上一致; 执行层保留 $\varepsilon(x_\tau + y_\tau)$ 项以抑制同时充放电, 适用命题 3 的条件性保证。

$$C_{3,t} = c_t \min(p_t, q_t) + 1.5c_t (q_t - p_t)^+ + 0.5c_t (p_t - q_t)^+ + 5c_t u_t, \quad (25)$$

式中 u_t 为真实负荷与光伏到达并完成物理执行后计算的应急购电量, 不参与此前决策时刻的未来真值输入。该公式具有清楚的边际含义: 下调部分按 $0.5c_t$ 计费, 上调部分按 $1.5c_t$ 计费, 电量缺口执行 5 倍应急购电价格。每个时段的最终购电量只相对 0:00 原计划结算一次, 中间多次滚动优化的目标值不直接相加, 因此不会重复计费。

5.4.2 约束条件

1. 功率平衡约束

$$q_t - w_t + v_t + y_t + u_t = L_t + x_t, \quad (26)$$

式中各变量均为单个 10 分钟时段电量 (kWh); v_t 为光伏消纳量, w_t 为闲置合同电量, x_t, y_t 为充、放电量。

2. 储能系统运行约束

$$\begin{cases} S_{t+1} = S_t + 0.9x_t - \frac{y_t}{0.9} \\ S_{\min} \leq S_t \leq S_{\max} \\ 0 \leq x_t, y_t \leq B_{\max} \end{cases} \quad (27)$$

式中 S_t 为储能 t 时刻储能量, x_t, y_t 分别为该 10 分钟时段的充、放电量; 其上限由 $P_{\max} \Delta t$ 给出, $S_{\min}, S_{\max}$ 为 SOC 上下限。

3. 合同调整仅作用于尚未执行时段, 且 $q_t \geq 0$; 已执行时段满足 $q_t = p_t$, 未执行时段满足 $q_t = p_t + \delta_t^+ - \delta_t^-$, 并且 $0 \leq w_t \leq q_t$ 。

4. 非负约束

$$u_t \geq 0, \quad P_{\text{curt},t} \geq 0. \quad (28)$$

5.4.3 滚动求解配置

在相邻预报发布时点之间，滚动 LP 同时考虑未来价格、修正后的净负荷和 SOC 边界，每次只执行当前时段动作，再用最新状态重新优化，构成 MPC 执行过程 [5]。对 $H = 1, 3, 6, 12$ 的全年试验中， $H = 1$ 费用最低； $H = 6$ 仅高 4273.26 元 (0.03%)，但能够联合安排未来 1 小时的储能动作。综合样本费用、多时段协调能力和计算规模，主方案取 $H = 6$ 。

5.5 仿真结果与分析

5.5.1 主策略与基线方案对比

将两组方案按日聚合统计，得到 334 天仿真周期内的年度关键指标，如表7所示。主

表 7: 问题三主策略与基线方案年度指标对比

指标	主策略 (启用全部日内更新)	基线方案 $S = \emptyset$	变化量
年度总费用/元	14664345.40	15230376.12	-566030.72
应急购电量/kWh	210761.48	334601.06	-123839.58

策略年度费用较基线下降 **566030.72 元**，降幅约 3.72%，应急购电量同时下降 37%。此外，0:00 计划购电量为 **21702724.37 kWh**，最终调整购电量为 **21702916.77 kWh**；累计上调量 **513849.94 kWh**、下调量 **513657.53 kWh**，两者基本相抵但发生时段不同，说明合同调整将风险在各时段之间重新分配，整日合同没有单向变化；弃购量为 **1228166.90 kWh**。

5.5.2 预报信息消融实验

为隔离日内预报信息本身的价值，在保持同一滚动调度方法、同一物理约束与同一结算口径的前提下，仅屏蔽不同时刻的官方预报信息，开展 8 组信息消融实验，结果如表8所示。由表8可得，不使用任何日内新预报的方案费用为 15230376.12 元，全部启用三个日内预报后降至 14664345.40 元，节省 56.60 万元 (3.72%)。6:00、12:00、18:00 更新的平均边际收益分别约为 6.15、28.41、16.27 万元，表明预报信息价值存在明显时点差异，其中 12:00 时点临近负荷晚高峰，信息经济价值最高。只采用任一单一时点费用均低于不使用日内预报，说明在题目提供的预报范围内三个日内时点均有实际作用。

表 8: 问题三不同日内预报组合的结果比较

预报组合 $\mathcal{S}$	年度费用/元	应急购电/kWh	节省费用/元
$\emptyset$	15230376.12	334601.06	0
{6}	15038617.00	266410.12	191759.11
{12}	14746909.96	224631.56	483466.16
{18}	14805047.54	228882.13	425328.58
{6, 12}	14679898.82	217968.28	550477.30
{6, 18}	14864586.68	218423.74	365789.44
{12, 18}	14710994.09	212056.29	519382.03
{6, 12, 18}	14664345.40	210761.48	566030.72

5.5.3 盈亏平衡更新成本

将消融实验的费用差折算为单次合同更新的盈亏平衡成本：

$$\kappa^* = \frac{C_{\emptyset} - C_{\{6,12,18\}}}{\Delta n} = \frac{566030.72}{1002} \approx 564.90 \text{ 元/次}, \quad (29)$$

式中 $\Delta n = 1002$ 为全年合同更新次数。其经济含义为：若一次预报获取、传输、计算、合同重签与执行的综合成本低于 564.90 元，则全年启用全部日内预报在当前样本内具有正净收益。若只能选择一个时点，则 12:00 优先级最高。

5.5.4 其他预报时点的近似测试

题目仅在 0:00、6:00、12:00、18:00 提供独立官方预报；对其他时点的检验仅作为额外重优化机会的敏感性分析，不解释为新增独立预报的信息价值。近似测试显示，3:00 和 9:00 的合同重算会增加现金费用；15:00 的收益很小，仅可作为谨慎候选。21:00 虽有数值改善，但此后实际光伏功率恒为零，其收益来自多一次基于当前 SOC 的合同重优化而非光伏预报信息，故不纳入推荐策略。

5.5.5 代表日分析

选取春、夏、秋、冬 4 个典型代表日，绘制日前计划购电 p_t 与最终合同 q_t 的轨迹，并标注 6:00、12:00、18:00 更新事件。不同季节光伏工况下，模型均会依据新预报对尚未执行的合同作向上或向下修正；指定时段的完整数值结果见附录。

5.6 本章小结

本章建立支持日内购电合同修正的微网滚动 MPC 模型，并以不使用日内新预报的方案为基线完成 334 天仿真。模型在 6:00、12:00、18:00 基于当前 SOC 重优化剩余合同，最终合同只相对 0:00 原计划结算一次；分层风险裕量贯穿日前规划、合同调整与滚动执行。主策略相对基线使年度总费用下降 3.72%、应急购电量下降 37%。8 组消融实验表明全部启用三个官方预报节省 566030.72 元，12:00 的边际价值最高，单次合同更

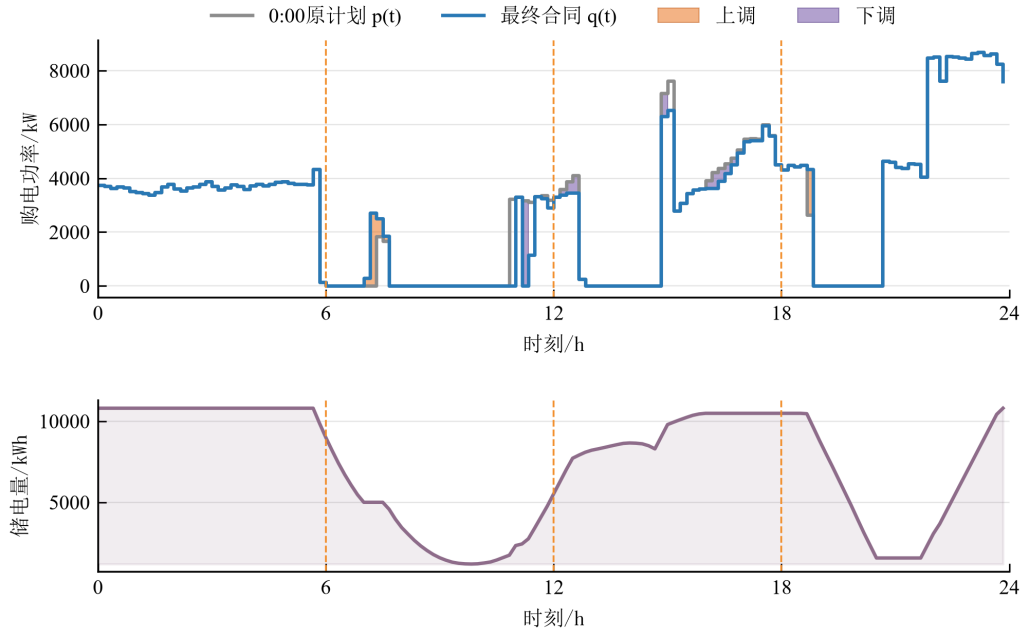


图 8: 问题三事件驱动的合同调整过程

新的盈亏平衡成本约 564.90 元；补充时点测试不改变仅采用官方四个发布时点的策略选择。

六 问题四：考虑电价预测不确定性的微网多时间尺度调度优化

6.1 问题描述：波动电价场景下的购电策略

在问题三日内合同修正调度框架基础上，问题四将固定购电电价替换为分时波动市场电价。0 时刻无法预知全天真实电价，仅能依托历史时序数据构建电价预测；电价预测偏差会造成日前购电合同与储能调度方案偏离最优工况，带来应急购电、闲置合同等额外经济损失。本节对比两种合同模式：Q4-2 采用固定日前合同，0 时刻签约后全天不可修改；Q4-3 支持日内合同调整，在 6:00、12:00、18:00 基于更新信息修正剩余时段购电计划。两种方案均采用同一套预测基线，保证对比口径统一，用于量化日内合同调整机制在电价不确定性场景下的经济效益。

6.2 基于历史信息的电价预测

采用四种历史预测曲线构建 0 时刻电价预测基线：昨日同时段、近 7 日均值、近 30 日均值、近 30 日中位数。对每个目标时段，选取对应时段历史电价进行统计估计。表9汇总四种预测方案的 MAE、RMSE、尖峰 MAE 与最大绝对误差。

本文选取近 30 日中位数作为统一预测基线。该选择为 Q4-2、Q4-3 提供完全一致的价格输入，隔离合同调整机制本身带来的收益，同时在四种基准方法中综合误差相对

表 9: 四种 0:00 基准价格预测方法误差

基准方法	MAE	RMSE	尖峰 MAE	最大绝对误差
昨日同时段	0.08	0.12	0.09	0.53
近 7 日均值	0.08	0.11	0.10	0.50
近 30 日均值	0.09	0.11	0.11	0.54
近 30 日中位数	0.08	0.12	0.10	0.62

单位：元/kWh。

较优。

设 0 时刻的近 30 日中位数价格基线为 $\hat{c}_{d,t}^{(0)}$ ；日内 k 时刻之前已完成时段的价格误差为 $e_{d,j}^c = c_{d,j} - \hat{c}_{d,j}^{(0)}$ 。利用已观测历史误差加权平均，随目标时段与当前时刻距离指数衰减，构造日内修正后的电价预测：

$$\hat{c}_{d,t|k} = \max \left\{ 10^{-4}, \hat{c}_{d,t}^{(0)} + \bar{e}_{d,k}^c \exp[-(t-k)/18] \right\}, \quad t \geq k. \quad (30)$$

式中下限 10^{-4} 用于防止修正后出现非正 (≤ 0) 价格，仅为数值护栏。经核验，附件 4 给出的 2025 年真实电价全部严格为正（最小值约 0.0076 元/kWh，近 30 日中位数预测基线的最小值约 0.3155 元/kWh），该下限在本数据上从不生效，因此不会改变任何决策；费用结算直接采用附件 4 的原始真实电价 $c_{d,t}$ ，未经该下限处理，预测侧的截断不进入结算口径。该公式仅使用 k 时刻前已经观测到的价格数据，未来真实电价不参与决策，仅在结算阶段使用。

四种基准预测方法的误差排序与年度总费用排序并不完全一致。MAE 对所有时段等权重，但运行费用还受负荷、应急购电和储能状态影响。两种合同方案统一采用近 30 日中位数价格预测，并沿用问题二的负荷预测与官方光伏预报方案，以隔离合同调整机制的影响。

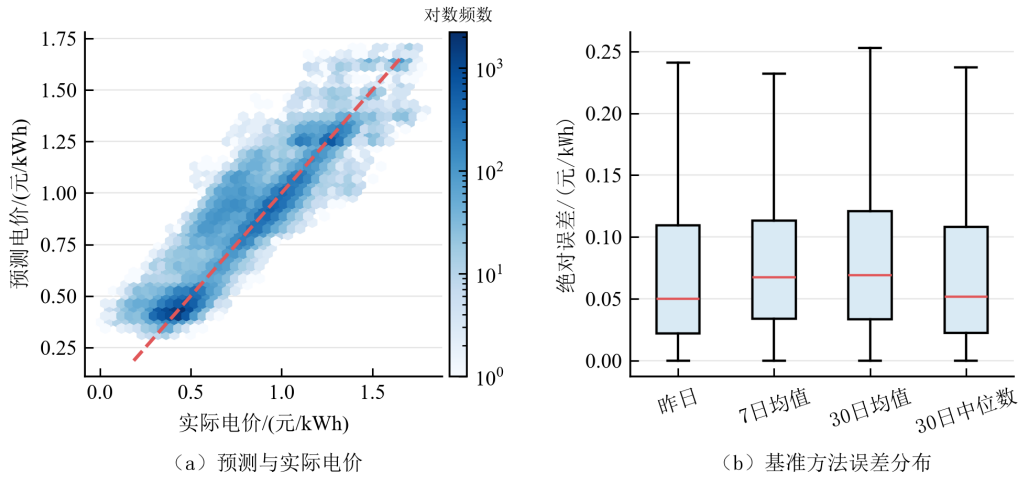


图 9: 波动电价预测的拟合与误差分布

6.3 调度模型与两种合同模式

Q4-2 与 Q4-3 的核心差异在于购电合同是否允许日内调整。

- **Q4-2 (固定合同模式)**: 在 0 时刻一次性生成全天购电合同, 合同一经确定全天保持不变; 日内仅依靠储能充放电与应急购电平抑功率缺口, 不支持合同修改。
- **Q4-3 (可调合同模式)**: 0 时刻生成初始购电合同, 在 6:00、12:00、18:00 三个时点, 利用更新的电价、光伏与负荷信息, 对尚未执行的时段重新优化购电计划, 已执行时段不可回溯修改。

两种模式在滚动 LP 优化中, 均使用预测电价、预测负荷与光伏安排调度; 合同调整沿用问题三的结算规则, 仅相对 0 时刻原始计划进行一次结算, 最终运行费用使用附件 4 的真实电价计算。

Q4-2 与 Q4-3 均沿用问题二、三的规划层-执行层分工: 真实充放电动作由同一滚动 LP 执行器给出, 规划层电价预测的变化只改变合同签订或调整决策, 不改变执行层的目标函数结构。

6.4 仿真结果与分析

6.4.1 固定合同与可调合同对比

两种模式全年仿真汇总结果如表10。

表 10: 问题四两种模式的年度结果 (电量单位: kWh, 费用单位: 元)

模式	总费用/元	应急购电量/kWh	更新次数
Q4-2 固定合同	17378687.69	570732.66	0
Q4-3 可调合同	15465962.60	210394.02	1002

由表可见, Q4-3 相比 Q4-2 年度总费用减少 **1912725.09 元**, 降幅 11.01%; 应急购电量下降 **360338.64 kWh**, 降幅 63.14%。结果表明, 在分时电价波动场景下, 日内合同调整的价值体现在两方面: 一是直接削减功率缺口对应的高价应急购电, 二是通过储能跨时段优化, 合理安排各时段购电量, 降低电价波动带来的成本。

6.4.2 不同价格基准与理想信息下界分析

为验证结论鲁棒性, 在四种历史电价预测基线、以及 0 时刻预知全部真实电价的理想信息场景下, 分别求解 Q4-2 与 Q4-3, 结果如表11。

四种历史基准方法下, Q4-2 费用极差为 23619.64 元, Q4-3 费用极差为 31199.58 元, 说明合同调整机制的费用优势在不同价格预测器下均能观察到。将全知理想电价场景作为下界, 因无法提前获取真实电价带来的额外费用损失: Q4-2 损失 78602.31 元, Q4-3

表 11: 问题四价格基准方法与理想信息结果

价格基准方法	Q4-2 费用/元	Q4-3 费用/元
昨日同时段	17402307.33	15485837.98
近 7 日均值	17390589.20	15454638.40
近 30 日均值	17400942.40	15455190.24
近 30 日中位数	17378687.69	15465962.60
0:00 已知实际电价	17300085.39	15383049.81

损失 82912.80 元，仅占各自总费用的 0.45% 与 0.54%。这说明波动电价带来的成本变化，不能全部归因于电价预测误差，同时包含电价水平、尖峰位置、光伏与负荷缺口的耦合作用。

在四种基准方案下，Q4-3 费用均低于 Q4-2，节约幅度稳定在约 190 万元。对比不同预测方法之间的费用差异，合同更新机制带来的收益，远大于更换简单电价预测器带来的收益。

6.5 本章小结

本章针对分时波动电价场景，构建基于历史时序的电价预测方法，搭建固定合同、日内可调合同两类微网购电调度方案，开展全周期仿真对比。得到结论如下：

1. 在近 30 日中位数电价预测基线下，相比固定合同方案，支持日内调整的 Q4-3 年度总费用降低 11.01%，应急购电量下降 63.14%，日内合同调整可以显著对冲电价波动带来的经济损失；
2. 在多种电价预测基准下，可调合同策略均稳定优于固定合同，合同调整机制的收益不依赖特定的电价预测模型；
3. 与预知全部真实电价的理想场景对比，电价预测误差带来的额外损失占总成本比例很低，电价波动场景的运行成本，是电价时序、负荷与光伏造成的供需缺口、储能调度、合同规则多因素耦合的结果。

本章从电价不确定性角度验证了日内合同调整机制的价值，与问题三光伏预测不确定性的结论形成完整互补，共同论证了多信息更新下滚动合同修正策略在微网经济调度中的有效性。

七 模型检验与灵敏度分析

7.1 数值与物理检验、规划-执行偏差诊断

为验证模型求解结果满足物理约束与数值可靠性，对 Q1-Q4 四个问题的仿真输出逐项校验。检验维度包含：母线功率平衡残差、蓄电池 SOC 时序递推校验、变量上下

界约束、储能充放电互斥约束、储能跨日状态连续传递、全周期逐时段现金流水与汇总核算一致性。同时开展**规划-执行偏差诊断**：对比 MPC 规划输出的储能状态与真实工况执行后的 SOC，计算一步预测误差。该指标仅作为辅助诊断，不能单独证明不存在未来信息泄露；严格的非前视性应由每个决策时刻的输入时间戳审计确认。

对全量求解结果校验可得：全部方案的储能 SOC 时序递推、母线功率平衡残差均处于 10^{-13} 量级，属于求解器浮点计算误差范围；Q2-Q4 的全部滚动执行方案满足储能充放电互斥约束 $\min(x_t, y_t) = 0$ ，验证了命题 3 中证明未覆盖的退化角点在本文仿真中未被触及，即执行层的互斥性保证在本文数据范围内完整成立。仿真全程储能 SOC 始终处于 $1200 \leq S_t \leq 10800$ kWh 的硬件边界之内。各方案规划-执行一步 SOC 误差如表12所示，一步 MAE 为 5-6 kWh 量级，RMSE 为 17-20 kWh 量级，说明预测规划与真实执行之间存在可解释偏差。该结果只能作为流程诊断；非前视性还需由代码记录的输入最大时间戳逐次验证，确保其不晚于决策时刻。全年度逐时段现金流累加结果与年

表 12: 各方案规划-执行一步 SOC 偏差诊断

方案	一步 MAE/kWh	一步 RMSE/kWh	最大绝对误差/kWh
Q2 主策略	5.33	19.67	284.58
Q3 $S = \{6, 12, 18\}$	5.06	17.63	271.98
Q4-2 固定合同	4.80	17.30	271.98
Q4-3 可调合同	5.16	18.07	271.98

度汇总总费用的最大绝对偏差为 0 元，费用分项核算完全一致，不存在口径冲突。综合以上校验，各问题调度输出满足全部物理约束，数值求解误差很小，规划与执行之间存在合理偏差，仿真结果具备可信性。

7.2 尾部风险与跨月稳定性检验

仅依靠年度平均总费用不足以完整评价调度策略的经济性能，进一步对日运行费用分布做尾部风险度量，采用 95% 分位 VaR、CVaR 以及最坏单日费用评估极端工况下的风险水平。风险约束调度在微网可靠性评价中的应用可参考 Vahedipour-Dahraie 等 [6]，结果如表13所示。对比 Q4-2 与 Q4-3 可看出，启用日内合同调整后，CVaR_{0.95} 由

表 13: 各方案尾部风险指标（95% 置信水平）

方案	VaR _{0.95} /元	CVaR _{0.95} /元	最坏单日费用/元
Q2 主策略	66075.35	70048.64	88726.19
Q3 $S = \{6, 12, 18\}$	65177.30	70474.34	77111.50
Q4-2 固定合同	87294.40	101705.29	134220.63
Q4-3 可调合同	76237.79	83469.29	92033.53

101705.29 元降至 83469.29 元，下降 18235.99 元；最坏单日费用由 134220.63 元降至

92033.53 元。合同调整机制不仅降低平均费用，也显著压缩极端高费用日风险，策略具备良好抗风险能力。

进一步开展跨月稳定性检验，考察各月份启用全部日内预报相对不使用新预报（Q3空集）的费用改善幅度。2-12 月各月日均费用差均为负值，即每月均实现成本节约，收益覆盖全年所有月份，并非由个别异常月份单独驱动，策略改善效果具备时间维度的稳健性。

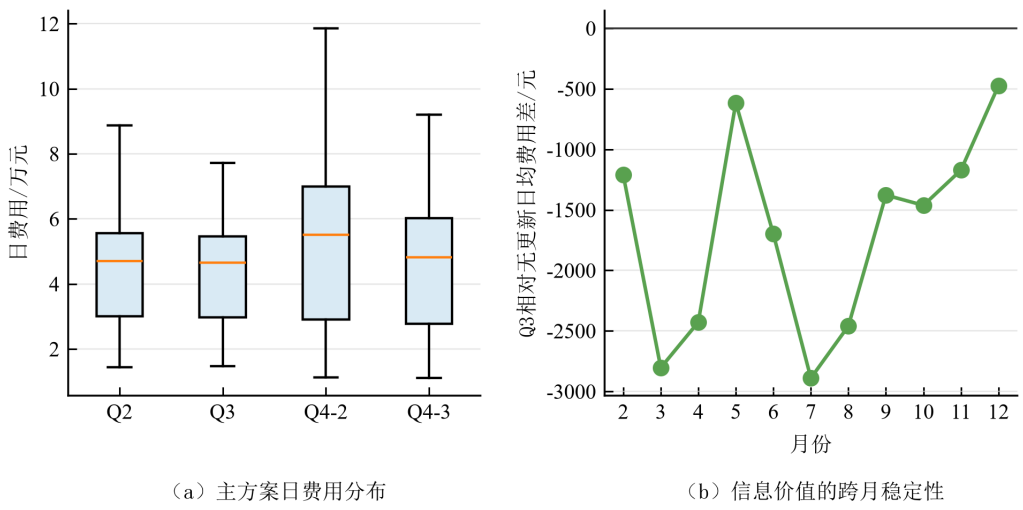


图 10: 固定合同与可调合同的尾部风险及跨月稳定性。

7.3 结构敏感性分析

为评估模型输出结果对关键参数的鲁棒性，开展多维度敏感性压力测试。**重要测试原则：**全部敏感性实验均在相同负荷序列、电价信息集、结算规则下完整重跑全周期仿真，不采用”直接替换参数代入原有调度轨迹”的简化估算，保证各组对比公平。测试变量包含：仿真初始 SOC、储能容量/功率上限、负荷预测误差倍率，结果如图11所示。

7.3.1 初始 SOC 敏感性

分别取储能初始 SOC 为额定容量的 20%、50%、80%。三种初始状态下年度总费用均为 14786101.35 元，应急购电量也完全一致，说明经过 1 月预热周期后，初始 SOC 差异对正式统计窗口的影响被完全消除，全年调度结果对初始荷电状态不敏感。该结果从数值层面确认预热期（1 月 1 日至 1 月 31 日，其预测降级规则与非前视性见第1.2节）足以吸收初始条件扰动，正式统计窗口内的年度结论不依赖仿真起点的取值。

7.3.2 储能容量与功率上限敏感性

将储能容量、充放电功率上限分别取基准值的 80%、100%、120%。容量降低 20% 使总费用升至 15392912.44 元 (+4.10%)，容量增加 20% 降至 14267282.70 元 (-3.51%)。

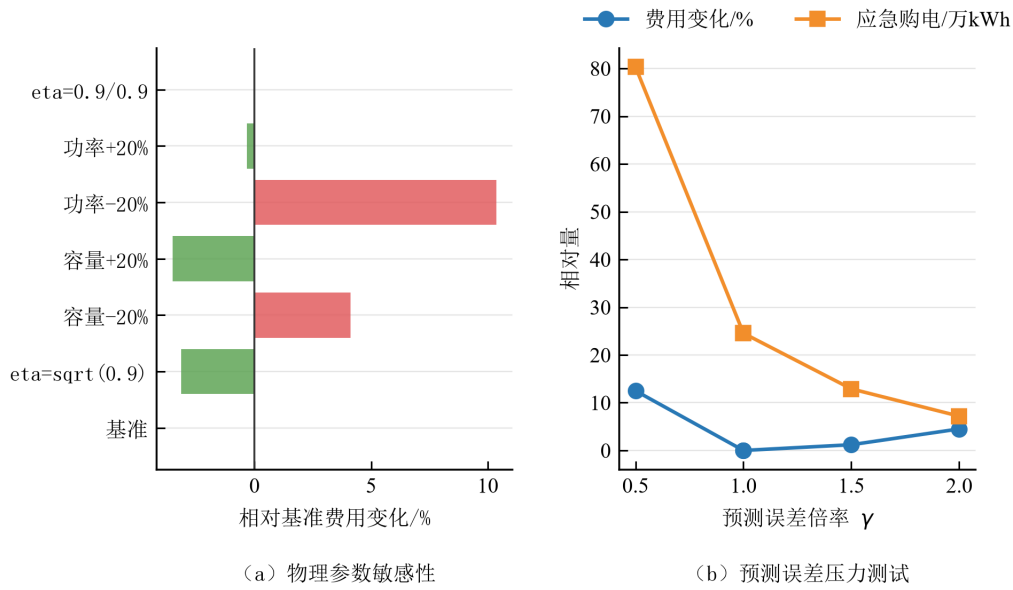


图 11: 多维度参数敏感性测试结果汇总

相比之下，**功率上限降低 20%** 使总费用大幅升至 16318749.90 元 (+10.37%)，应急购电量由 245483.29 增至 563286.32 kWh；功率增加 20% 仅带来 -0.33% 的微幅改善。储能功率上限属于高敏感参数，功率不足会显著限制储能移峰能力，推高应急购电成本；容量参数敏感度相对更低。

7.3.3 负荷预测误差倍率压力测试

令 γ 作用于历史残差的风险修正项，而非直接替换真实负荷，并在每个 γ 下重新生成合同、滚动执行和结算。结果并非单调： $\gamma = 0.5$ 时风险裕度不足，缺口应急购电增加； $\gamma = 1.5$ 时合同冗余在本样本中部分抵消应急费用； $\gamma = 2.0$ 时弃购成本再次上升。因而这里应表述为风险裕度存在样本依赖的折中区间，而不能简单写成误差倍率增大导致费用上升。

综合全部敏感性实验结果：

1. 储能充放电功率上限、负荷预测误差倍率属于高敏感参数；全年仿真结果对初始 SOC 敏感度最低，对储能额定容量敏感度相对较弱。
2. 1 月预热机制能有效消除仿真初始 SOC 带来的短期扰动，长期年度结论不受初始储能状态干扰。
3. 在合理预测扰动范围内，整套滚动风险修正与合同调整框架可稳定发挥能量转移与风险对冲作用，同时上述结果明确了模型策略的适用边界。

八 模型评价与推广

8.1 模型特点

本文以统一物理约束和因果信息集贯通四个问题，仅按赛题改变合同调整权限与电价时序；实测真值仅在当期决策生成后进入执行修正和结算。预测、风险裕量、合同规划、滚动执行与现金结算构成闭环，所有方案均通过逐时段供需平衡、SOC 递推、费用分项和互斥性检验。

问题三、四复用同一滚动调度框架，通过预报组合、边际收益和理想电价基准分别识别日内信息与电价未知的经济影响；同时报告平均费用、应急购电与尾部风险，避免以单一指标作出结论。六层提前期风险裕量保留了“预报越远、误差通常越大”的时序结构；对 21:00 的补充检验也表明，数值收益必须与新增信息的物理意义共同解释。

8.2 局限与推广

题目未给出并网容量上限以及售电相关机制，本文假设外购电无上限，且微网不可向主网反向送电。若工程场景存在并网容量约束，则需增加约束 $g_t + u_t \leq G_{\max}$ 。模型采用经验分位数刻画历史残差尾部特征，未刻画连续多时段预测误差间的相关性，因此逐时段覆盖率不能等价为整日可靠概率。Q3、Q4 的收益还可能同时来自信息更新、合同调整和重新优化，当前对照不能完全识别三者的边际贡献。仿真采用 10 分钟聚合时间粒度，忽略时隙内部测量延迟与功率爬坡过程；实际工程应用时，可缩短仿真步长或增加延迟状态变量予以修正。本文全部结论基于 2025 年历史数据得到，更换其他年份数据集时，需要重新校验模型参数与费用表现。

按预报提前时间分组设置风险裕量时，采用同一种风险裕量的全局方案，在样本上的总费用相比分组方案低 8860.56 元。本文仍然选用分组方案，原因在于该方案保留了“预报提前期越长，预测误差通常越大”的数据固有特征。问题三中 $H = 1$ 对应的样本费用略低于 $H = 6$ ，说明滚动时域长度需要在样本经济性与多时段规划能力之间做折中取舍。

模型可扩展到含风电、可控柔性负荷与多组储能的园区微网场景。新增分布式风电资源，只需在供需平衡方程内增加对应出力供给变量；引入需求响应资源，则增加负荷平移变量，并配套用户用电舒适度约束。若放开售电权限或存在负电价场景，应当采用带有显式充放电互斥约束的混合整数规划模型，规避充放电循环套利问题。微网分层控制的相关研究可参见 [7]。

若能够获取多年实测数据，可将当前 35 日滑动分位替换为季节条件分位或场景树方法，并采用独立年份样本完成参数 W, α, H 的优选；当前参数与 λ 均在同一年度样本上选择并评价，存在样本内调参偏差，正式应用时应采用时间滚动交叉验证或独立年份检验。如果预报供应商按次收取预报费用，可将 κ 纳入预报获取方案优化，把盈亏平衡分析拓展为 0-1 整数选择模型；当园区配备多块储能电池时，可以对每一块电池单独建

立 SOC 状态方程，额外增加共享并网容量约束与电池寿命折损成本。上述所有拓展场景，均可沿用本文的购电计划生成、日内实际调度以及费用结算完整流程。

九 结论

本文在统一储能与供需平衡模型下完成四个子问题的仿真。问题一的典型日购电量为 **59482.70 kWh**，费用为 **35126.95 元**。问题二采用 F1 预测与风险裕量，334 天总费用为 **14786101.35 元**，应急购电量为 **245483.29 kWh**。问题三在 0:00、6:00、12:00 和 18:00 更新预报并调整合同，总费用为 **14664345.40 元**，相比不使用日内新预报的方案节省 **566030.72 元**，应急购电量为 **210761.48 kWh**。问题四中，固定合同和可调合同的总费用分别为 **17378687.69 元**和 **15465962.60 元**，可调合同节省 **1912725.09 元**，并使应急购电量下降 63.14%。

上述结论仅适用于不可售电、无并网功率上限、10 分钟离散以及 2025 年历史样本的设定。数值检验显示供需平衡和 SOC 递推误差处于求解器浮点误差范围；敏感性分析表明，储能功率上限和预测误差倍率是影响费用的主要因素。模型还可扩展到含风电、需求响应和多储能的园区微网，但扩展后需要重新核验参数和费用口径。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模方案比较、参数分析、代码调试、图示设计和语言润色等，详细使用情况见支撑材料。

参考文献

- [1] Thirunavukkarasu M, et al. Role of Optimization Techniques in Microgrid Energy Management Systems—A Review[J]. Energy Strategy Reviews, 2022,43:100899. DOI:10.1016/j.esr.2022.100899.
- [2] Nemati M, et al. Optimization of Unit Commitment and Economic Dispatch in Microgrids Based on Genetic Algorithm and MILP[J]. Applied Energy, 2018,210:944–963. DOI:10.1016/j.apenergy.2017.07.007.
- [3] Hong T, Pinson P, Wang Y, Weron R, Yang D, Zareipour H. Energy Forecasting: A Review and Outlook[J]. IEEE Open Journal of the Power and Energy Society, 2020,7:1–20. DOI:10.1109/OAJPE.2020.3029979.
- [4] Iheanetu K J. Solar Photovoltaic Power Forecasting: A Review[J]. Sustainability, 2022,14(24):17005. DOI:10.3390/su142417005.
- [5] Parisio A, Rikos E, Glielmo L. Stochastic Model Predictive Control for Economic/Environmental Operation Management of Microgrids: An Experimental Case Study[J]. Journal of Process Control, 2016,43:24–37. DOI:10.1016/j.jprocont.2016.04.008.
- [6] Vahedipour-Dahraie M, Anvari-Moghaddam A, Guerrero J M. Evaluation of Reliability in Risk-Constrained Scheduling of Autonomous Microgrids with Demand Response and Renewable Resources[J]. IET Renewable Power Generation, 2018,12(6):657–667. DOI:10.1049/iet-rpg.2017.0720.
- [7] Bouzid A M, Guerrero J M, Chériti A, et al. A Survey on Control of Electric Power Distributed Generation Systems for Microgrid Applications[J]. Renewable and Sustainable Energy Reviews, 2015,44:751–766. DOI:10.1016/j.rser.2015.01.016.
- [8] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python[J]. Nature Methods, 2020,17(3):261–272. DOI:10.1038/s41592-019-0686-2.

A 题目要求表格与指定日期结果

题目要求在论文中呈现的结果表格统一列于本附录；正文仅保留模型、分析和关键结论。

1.1 问题一：表 1 与表 2

表 14: 问题一指定时间段购电量及全天汇总

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量	59482.70			全天购电费/元	35126.95

表 15: 问题一分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	4500.00	0.00	4:00-8:00	833.33	6365.84
8:00-12:00	4787.96	1703.00	12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13	20:00-24:00	5333.33	2859.87
0:00 储电量	6000.00 kWh		24:00 储电量	6000.00 kWh	

1.2 问题 2：表 1、表 2 与表 3

表 16: 问题 2：2025-03-20 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	479.31	14:00-14:10	0.00
16:00-16:10	562.26	18:00-18:10	698.00	20:00-20:10	5.40

表 17: 问题 2：2025-03-20 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	0.00	5499.86
8:00-12:00	5900.91	3122.72	12:00-16:00	5283.31	436.64
16:00-20:00	0.00	5388.59	20:00-24:00	10252.91	2920.55
日初储电量	10800.00		日末储电量	10795.24	

表 18: 问题 2: 2025-06-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	78.57	18:00-18:10	0.00	20:00-20:10	0.00

表 19: 问题 2: 2025-06-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	2482.63	4:00-8:00	0.00	5366.66
8:00-12:00	6224.02	0.00	12:00-16:00	3482.98	13.38
16:00-20:00	0.00	5234.59	20:00-24:00	9531.47	2485.90
日初储电量	10800.00		日末储电量	10800.00	

表 20: 问题 2: 2025-09-23 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	3.39	12:00-12:10	354.19	14:00-14:10	0.00
16:00-16:10	589.43	18:00-18:10	764.33	20:00-20:10	11.74

表 21: 问题 2: 2025-09-23 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	43.52	5804.35
8:00-12:00	5817.31	2823.94	12:00-16:00	5004.29	178.42
16:00-20:00	0.00	5602.91	20:00-24:00	10426.13	2806.64
日初储电量	10760.83		日末储电量	10793.77	

表 22: 问题 2: 2025-12-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	932.83	14:00-14:10	0.00
16:00-16:10	801.38	18:00-18:10	715.41	20:00-20:10	0.00

表 23: 问题 2: 2025-12-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	881.63	1508.33
8:00-12:00	5655.40	7806.67	12:00-16:00	8333.33	2690.88
16:00-20:00	0.00	5512.67	20:00-24:00	10315.06	2842.53
日初储电量	10756.53		日末储电量	10800.00	

表 24: 问题 2: 指定日期紧急购电量

日期	紧急购电时间段	购电量/kWh
2025-03-20	0:20-0:30	1.94
2025-03-20	1:00-1:40	116.54
2025-03-20	3:10-3:30	36.11
2025-06-21	1:20-1:30	1.70
2025-06-21	1:50-2:00	1.24
2025-06-21	4:00-4:10	0.61
2025-09-23	0:40-0:50	13.55
2025-09-23	2:10-2:20	14.23
2025-09-23	4:50-5:10	38.37
2025-12-21	3:00-3:10	7.54
2025-12-21	3:30-3:50	65.05
2025-12-21	10:30-10:50	28.95

1.3 问题 3: 表 1、表 2 与表 3

表 25: 问题 3: 2025-03-20 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	549.71	14:00-14:10	0.00
16:00-16:10	605.82	18:00-18:10	718.21	20:00-20:10	0.00

表 26: 问题 3: 2025-03-20 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	0.00	6154.48
8:00-12:00	3970.58	2485.52	12:00-16:00	6708.75	328.09
16:00-20:00	43.56	5094.62	20:00-24:00	10253.90	2928.49
日初储电量	10800.00		日末储电量	10800.00	

表 27: 问题 3: 2025-06-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	163.73	18:00-18:10	0.00	20:00-20:10	0.00

表 28: 问题 3: 2025-06-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	2944.22	4:00-8:00	191.07	5150.11
8:00-12:00	8517.34	0.00	12:00-16:00	1293.95	7.58
16:00-20:00	0.00	5541.12	20:00-24:00	9909.90	2485.90
日初储电量	10800.00		日末储电量	10800.00	

表 29: 问题 3: 2025-09-23 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	362.95	14:00-14:10	0.00
16:00-16:10	565.35	18:00-18:10	783.44	20:00-20:10	0.00

表 30: 问题 3: 2025-09-23 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	0.00	6591.76
8:00-12:00	5624.09	2048.24	12:00-16:00	5445.49	388.79
16:00-20:00	0.00	5602.91	20:00-24:00	10391.99	2752.18
日初储电量	10800.00		日末储电量	10800.00	

表 31: 问题 3: 2025-12-21 指定时段购电量

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	945.70	14:00-14:10	0.00
16:00-16:10	858.30	18:00-18:10	750.11	20:00-20:10	0.00

表 32: 问题 3: 2025-12-21 分段充放电量及始末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	0.00	0.00	4:00-8:00	833.33	1508.33
8:00-12:00	5218.33	7703.98	12:00-16:00	7534.09	1792.14
16:00-20:00	0.00	5434.51	20:00-24:00	10218.56	2842.53
日初储电量	10800.00		日末储电量	10800.00	

表 33: 问题 3: 指定日期紧急购电量

日期	紧急购电时间段	购电量/kWh
2025-03-20	0:20-0:30	7.34
2025-03-20	1:00-1:40	99.45
2025-03-20	3:10-3:30	36.04
2025-06-21	0:20-0:30	5.36
2025-06-21	1:20-1:30	11.72
2025-06-21	1:50-2:00	12.69
2025-09-23	0:40-0:50	14.46
2025-09-23	2:10-2:20	14.35
2025-09-23	4:50-5:10	21.85
2025-12-21	1:30-1:40	7.73
2025-12-21	3:00-3:10	27.03
2025-12-21	3:30-3:50	104.82

B 支撑材料清单

本论文按竞赛规则提交配套支撑材料压缩包，用于结果复现与使用过程核查。支撑材料按“程序代码、文献资料、AI 使用报告”三类组织，文件清单如表34所示，表中路径均以压缩包根目录为基准。

表 34: 支撑材料文件清单

路径	说明
01_ 程序代码/py/ （结果复现的全部源程序）	
run_lite.py	全流程运行入口，按顺序调用问题一至问题四求解脚本
microgrid_core.py	公共参数、数据读取、储能与合同线性规划、安全轨迹及预测器等底层函数
solution_lib.py	年度连续仿真与滚动调度执行库；预测、合同优化、MPC 执行、结算与校验
问题 1_ 求解.py	问题一：确定性典型日线性规划基准调度
问题 2_ 求解.py	问题二：负荷与光伏预测不确定性下的年度日前购电调度
问题 3_ 求解.py	问题三：含日内购电合同修正的滚动优化策略
问题 4_ 求解.py	问题四：电价预测不确定性下的固定合同与可调合同方案
问题 3_ 其他时刻增量.py	问题三：备选预报更新时刻的增量收益测算
补充敏感性计算.py	结构敏感性分析（初始 SOC、容量与功率等）
终端价值敏感性.py	终端价值罚项系数敏感性分析
收口计算.py	各子问题收口口径复核与关键数值复算
汇总结果.py	汇总各问题输出并生成面向论文的结果表
02_ 文献资料/ （参考文献资料）	
*.pdf（共 8 篇）	论文实际引用的参考文献全文
03_AI 使用报告/ （AI 工具使用说明）	
AI 工具使用详情.pdf	与论文正文“AI 工具使用声明”一致的报告
04_ 结果/ （各问题结果文件）	
result1.xlsx	问题一典型日优化结果
result2.xlsx	问题二年度日前购电优化结果
result3.xlsx	问题三日内合同调整优化结果
result4-2.xlsx	问题四固定合同方案结果
result4-3.xlsx	问题四可调合同方案结果

使用说明如下。程序运行环境为 Python 3.10 及以上，依赖 `numpy`、`pandas` 与 `scipy`（线性规划求解器 HiGHS）；复现时在 `01_ 程序代码/py/`目录下执行 `python run_lite.py`，加`--list` 参数可仅列出各复现步骤而不实际运行。按竞赛规则，支撑材料不重复包含赛题提供的原始数据，运行前需将附件 1-4 放入程序目录的 `data/`附件/下。支撑材料的完整文件名列表另见交付物支撑列表/支撑材料文件列表.md。全部文件与路径均不含学校名称、队员姓名等身份信息。

C 完整程序代码

本附录嵌入支撑材料中用于结果复现的全部 Python 源代码；各文件的用途与运行方式见附录34（支撑材料清单）。

Listing 1: run_lite.py

```
from __future__ import annotations

import argparse
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent
SCRIPT_DIR = Path(__file__).resolve().parent

def commands() -> list[list[str]]:
    py = sys.executable
    return [
        [py, str(SCRIPT_DIR / "问题1_求解.py"), "--no-figures"],
        [py, str(SCRIPT_DIR / "问题2_求解.py"), "--main-only", "--days",
         "365", "--executor", "mpc"],
        [py, str(SCRIPT_DIR / "问题3_求解.py"), "--days", "365", "--workers", "1", "--main-only"],
        [py, str(SCRIPT_DIR / "问题4_求解.py"), "--days", "365", "--workers", "2", "--main-only"],
    ]

def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--list", action="store_true", help="只列出复现步骤，不执行")
    args = ap.parse_args()
    jobs = commands()
    if args.list:
        for job in jobs:
            print(subprocess.list2cmdline(job))
        return 0
    started = time.perf_counter()
    for i, job in enumerate(jobs, 1):
        print(f"[Lite {i}/{len(jobs)}] {subprocess.list2cmdline(job)}",
              flush=True)
        subprocess.run(job, cwd=ROOT, check=True)
    print(f"[Lite] all done, elapsed_s={time.perf_counter() - started:.1f}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

```
ROOT = Path(__file__).resolve().parent
DATA_DIR = ROOT / "data" / "附件"
RESULT_DIR = ROOT / "results"
FIGURE_DIR = ROOT / "figures"

T = 144
DT = 1.0 / 6.0
ETA_C = 0.9
ETA_D = 0.9
E_MAX = 12000.0
S_MIN = 1200.0
S_MAX = 10800.0
P_MAX = 5000.0
B_MAX = P_MAX * DT
SO = 6000.0
EMERGENCY_FACTOR = 5.0
OVER_FACTOR = 1.5
UNDER_FACTOR = 0.5

DAY0 = _dt.date(2025, 1, 1)
START_DATE = _dt.date(2025, 2, 1)
END_DATE = _dt.date(2025, 12, 31)
N_OFFICIAL = (END_DATE - START_DATE).days + 1

EPS_THROUGHPUT = 1e-4

_TIME_COLS = None

def _read_power_matrix(path: Path, sheet) -> np.ndarray:
    df = pd.read_excel(path, sheet_name=sheet, header=0)
    if df.shape[1] != 145:
        raise ValueError(f"{path.name}/{sheet} 列数异常: {df.shape[1]}")
    vals = df.iloc[:, 1:].to_numpy(dtype=float)
    if vals.shape != (365, T):
        raise ValueError(f"{path.name}/{sheet} 形状异常: {vals.shape}")
    if not np.isfinite(vals).all():
        raise ValueError(f"{path.name}/{sheet} 含缺失值")
    return vals

def load_inputs() -> dict:
    global _TIME_COLS
    a1 = pd.read_excel(DATA_DIR / "附件1.xlsx", sheet_name=0, header=0)
    a1.columns = [str(c).strip() for c in a1.columns]
    price1 = a1.iloc[:, 1].to_numpy(dtype=float)
    load1 = a1.iloc[:, 2].to_numpy(dtype=float)
    pv1 = a1.iloc[:, 3].to_numpy(dtype=float)
    if not (len(price1) == len(load1) == len(pv1) == T):
        raise ValueError("附件1 长度异常")

    load_act = _read_power_matrix(DATA_DIR / "附件2.xlsx", "小区负载")
    pv_act = _read_power_matrix(DATA_DIR / "附件2.xlsx", "光伏发电实际功率")
    price_act = _read_power_matrix(DATA_DIR / "附件4.xlsx", 0)

    a3 = pd.read_excel(DATA_DIR / "附件3.xlsx", sheet_name=0, header
```

Listing 2: microgrid_core.py

```
from __future__ import annotations

import datetime as _dt
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import linprog
```

```

        =0)
    if a3.shape != (1460, 26):
        raise ValueError(f"附件3 形状异常: {a3.shape}")
    raw_date = a3.iloc[:, 0].ffill()
    dates3 = pd.to_datetime(raw_date, format="%Y-%m-%d", errors="coerce")
    if dates3.isna().any():

        parsed = []
        cur = None
        for v in a3.iloc[:, 0]:
            if isinstance(v, str) and v.strip():
                cur = pd.Timestamp(_dt.datetime.strptime(v.strip(), "%Y-%m-%d").date())
                parsed.append(cur)
            dates3 = pd.Series(parsed)
        issue = a3.iloc[:, 1].astype(str).str.strip().to_numpy()
        fc_raw = a3.iloc[:, 2:26].to_numpy(dtype=float)

    issue_hours = np.array([int(s.split(":")[0]) for s in issue])

    day_idx = np.array([(d.date() - DAY0).days for d in dates3])
    slot_idx = issue_hours // 6
    fc = np.full((365, 4, 24), np.nan)
    for k in range(len(day_idx)):
        fc[day_idx[k], slot_idx[k]] = fc_raw[k]
    if not np.isfinite(fc).all():
        raise ValueError("附件3 预报张量存在空洞")

    _TIME_COLS = [str(c) for c in pd.read_excel(DATA_DIR / "附件2.xlsx", sheet_name="小区负载", header=0).columns[1:]]

    return {
        "price1": price1,
        "load1": load1,
        "pv1": pv1,
        "load_act": load_act,
        "pv_act": pv_act,
        "price_act": price_act,
        "fc_raw": fc,
    }

def date_of(day_index: int) -> _dt.date:
    return DAY0 + _dt.timedelta(days=int(day_index))

_DAY_LP_CACHE: dict = {}

def _day_lp_structure(n: int):
    if n in _DAY_LP_CACHE:
        return _DAY_LP_CACHE[n]
    N = 5 * n
    A_eq = np.zeros((2 * n, N))
    b_eq = np.zeros(2 * n)
    for t in range(n):

        A_eq[t, 2 * n + t] = 1.0
        A_eq[t, 3 * n + t] = 1.0
        A_eq[t, n + t] = 1.0
        A_eq[t, t] = -1.0

        r = n + t
        A_eq[r, 4 * n + t] = 1.0
        if t > 0:
            A_eq[r, 4 * n + t - 1] = -1.0
            A_eq[r, t] = -ETA_C
            A_eq[r, n + t] = 1.0 / ETA_D
    _DAY_LP_CACHE[n] = (A_eq, b_eq)

    return A_eq, b_eq

def solve_day_lp(c, load_kwh, pv_kwh, s_init, s_end=None, lam=None, charge_cap=None, extra_throughput_penalty=EPS_THROUGHPUT):
    c = np.asarray(c, dtype=float)
    n = len(c)
    A_eq, _ = _day_lp_structure(n)
    N = 5 * n

    b_eq = np.empty(2 * n)
    b_eq[:n] = load_kwh
    b_eq[n] = s_init
    b_eq[n + 1:] = 0.0

    obj = np.zeros(N)
    obj[2 * n:3 * n] = c
    obj[:n] += extra_throughput_penalty
    obj[n:2 * n] += extra_throughput_penalty
    if lam is not None:
        obj[4 * n + n - 1] = -lam

    lb = np.zeros(N)
    ub = np.full(N, np.inf)
    ub[:n] = B_MAX
    ub[n:2 * n] = B_MAX
    ub[2 * n:3 * n] = np.inf
    ub[3 * n:4 * n] = pv_kwh
    lb[4 * n:5 * n] = S_MIN
    ub[4 * n:5 * n] = S_MAX
    if charge_cap is not None:
        ub[:n] = np.minimum(ub[:n], charge_cap)
    if s_end is not None:
        lb[4 * n + n - 1] = s_end
        ub[4 * n + n - 1] = s_end

    bounds = list(zip(lb, ub))
    res = linprog(obj, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError(f"日前 LP 求解失败: {res.message}")
    z = res.x
    return {
        "x": z[:n], "y": z[n:2 * n], "g": z[2 * n:3 * n],
        "v": z[3 * n:4 * n], "S": z[4 * n:5 * n],
        "curtail": pv_kwh - z[3 * n:4 * n],
        "objective": float(res.fun), "status": res.status,
    }

def causal_dispatch(p, load_kwh, pv_kwh, s_init):
    n = len(p)
    x = np.zeros(n); y = np.zeros(n); u = np.zeros(n)
    w = np.zeros(n); v = np.zeros(n); S = np.zeros(n)
    s = float(s_init)
    for t in range(n):
        A = p[t] + pv_kwh[t] - load_kwh[t]
        if A >= 0.0:
            xe = min(A, B_MAX, (S_MAX - s) / ETA_C)
            xe = max(xe, 0.0)
            ye = 0.0
            ue = 0.0
            r = A - xe
            we = min(p[t], r)
            ve = pv_kwh[t] - max(0.0, r - we)
        else:
            xe = 0.0
            ye = min(-A, B_MAX, (s - S_MIN) * ETA_D)
            ue = -A - ye
            we = 0.0

```

```

        ve = pv_kwh[t]
        x[t], y[t], u[t], w[t], v[t] = xe, ye, ue, we, ve
        s = s + ETA_C * xe - ye / ETA_D
        S[t] = s
    return {"x": x, "y": y, "u": u, "w": w, "v": v, "S": S}

```

```

_ROLL_LP_CACHE: dict = {}

```

```

def _rolling_lp_structure(n: int):
    cached = _ROLL_LP_CACHE.get(n)
    if cached is not None:
        return cached

    N = 5 * n + (n + 1)
    o = {k: i * n for i, k in enumerate(("x", "y", "v", "u", "w"))}
    s0 = 5 * n
    A_eq = np.zeros((2 * n, N))
    for t in range(n):
        A_eq[t, o["x"] + t] = -1.0
        A_eq[t, o["y"] + t] = 1.0
        A_eq[t, o["v"] + t] = 1.0
        A_eq[t, o["u"] + t] = 1.0
        A_eq[t, o["w"] + t] = -1.0
        A_eq[n + t, o["x"] + t] = -ETA_C
        A_eq[n + t, o["y"] + t] = 1.0 / ETA_D
        A_eq[n + t, s0 + t] = -1.0
        A_eq[n + t, s0 + t + 1] = 1.0
    result = (A_eq, o, s0, N)
    _ROLL_LP_CACHE[n] = result
    return result

```

```

def rolling_lp_action(s_init, g_seg, load_seg, pv_seg, c_seg, lam,
                    time_limit=None):
    n = len(g_seg)
    A_eq, o, s0, N = _rolling_lp_structure(n)
    b_eq = np.concatenate([load_seg - g_seg, np.zeros(n)])

    obj = np.zeros(N)
    obj[o["u"] + 1 : o["u"] + n] = EMERGENCY_FACTOR * c_seg
    obj[o["x"] + 1 : o["x"] + n] += EPS_THROUGHPUT
    obj[o["y"] + 1 : o["y"] + n] += EPS_THROUGHPUT
    obj[s0 + 1 : s0 + n] = -lam

```

```

surplus = np.maximum(g_seg + pv_seg - load_seg, 0.0)
deficit = np.maximum(load_seg - g_seg - pv_seg, 0.0)

```

```

lb = np.zeros(N); ub = np.full(N, np.inf)
ub[o["x"] + 1 : o["x"] + n] = np.minimum(B_MAX, surplus)
ub[o["y"] + 1 : o["y"] + n] = B_MAX
ub[o["v"] + 1 : o["v"] + n] = pv_seg
ub[o["u"] + 1 : o["u"] + n] = deficit
ub[o["w"] + 1 : o["w"] + n] = g_seg
lb[s0] = ub[s0] = s_init
lb[s0 + 1 : s0 + n + 1] = S_MIN
ub[s0 + 1 : s0 + n + 1] = S_MAX

```

```

kwargs = {} if time_limit is None else {"options": {"time_limit":
    float(time_limit)}}
res = linprog(obj, A_eq=A_eq, b_eq=b_eq, bounds=list(zip(lb, ub)),
    method="highs", **kwargs)
if not res.success or res.x is None:
    return None
return float(res.x[o["x"]]), float(res.x[o["y"]])

```

```

def clip_slot(xm, ym, g, load_kwh, pv_kwh):
    xe = min(xm, max(g + pv_kwh - load_kwh, 0.0))
    xe = max(xe, 0.0)

```

```

    ye = min(ym, load_kwh + xe)
    ye = max(ye, 0.0)
    b = load_kwh + xe - ye - g - pv_kwh
    if b >= 0.0:
        u, w, v = b, 0.0, pv_kwh
    else:
        u = 0.0
        w = min(g, -b)
        v = pv_kwh - (-b - w)
        v = max(v, 0.0)
    return xe, ye, u, w, v, b

```

```

def rolling_execute(g_contract, load_kwh, pv_kwh, price, s_init,
                  horizon=36,
                  lam_price=None):
    n = len(g_contract)
    x = np.zeros(n); y = np.zeros(n); u = np.zeros(n)
    w = np.zeros(n); v = np.zeros(n); S = np.zeros(n)
    s = float(s_init)
    for t in range(n):
        K = min(t + horizon - 1, n - 1)
        seg = slice(t, K + 1)
        c_seg = price[seg]
        if lam_price is None:
            lam = ETA_D * float(np.median(c_seg))
        else:
            lam = lam_price[t]
        act = rolling_lp_action(s, g_contract[seg], load_kwh[seg], pv_
            kwh[seg],
                                c_seg, lam)
        if act is None:
            xm = ym = 0.0
        else:
            xm, ym = act
        xe, ye, ue, we, ve, _ = clip_slot(xm, ym, g_contract[t], load_
            kwh[t], pv_kwh[t])
        x[t], y[t], u[t], w[t], v[t] = xe, ye, ue, we, ve
        s = s + ETA_C * xe - ye / ETA_D
        if s < S_MIN - 1e-6 or s > S_MAX + 1e-6:
            raise RuntimeError(f"滚动执行 SOC 越界: t={t}, S={s}")
        s = min(max(s, S_MIN), S_MAX)
        S[t] = s
    return {"x": x, "y": y, "u": u, "w": w, "v": v, "S": S}

```

```

_CONTRACT_UPDATE_CACHE: dict = {}

```

```

def _contract_update_structure(n: int):
    cached = _CONTRACT_UPDATE_CACHE.get(n)
    if cached is not None:
        return cached
    N = 7 * n
    o = {"x": 0, "y": n, "v": 2 * n, "d": 3 * n, "e": 4 * n,
        "u": 5 * n, "S": 6 * n}
    A_eq = np.zeros((2 * n, N))
    for t in range(n):
        A_eq[t, o["x"] + t] = -1.0
        A_eq[t, o["y"] + t] = 1.0
        A_eq[t, o["v"] + t] = 1.0
        A_eq[t, o["d"] + t] = 1.0
        A_eq[t, o["e"] + t] = -1.0
        A_eq[t, o["u"] + t] = 1.0
        r = n + t
        A_eq[r, o["S"] + t] = 1.0
        if t > 0:
            A_eq[r, o["S"] + t - 1] = -1.0
        A_eq[r, o["x"] + t] = -ETA_C
        A_eq[r, o["y"] + t] = 1.0 / ETA_D
    result = (A_eq, o, N)

```

```

_CONTRACT_UPDATE_CACHE[n] = result
return result

def contract_update_lp(p_rem, c_rem, load_kwh, pv_kwh, s_init, s_end=
    None, lam=None):
    n = len(p_rem)
    A_eq, o, N = _contract_update_structure(n)
    bx, by, bv, bd, be, bu, bS = (o[k] for k in ("x", "y", "v", "d", "
        e", "u", "S"))
    b_eq = np.zeros(2 * n)
    for t in range(n):
        b_eq[t] = load_kwh[t] - p_rem[t]
    b_eq[n] = s_init

    obj = np.zeros(N)
    obj[bd:bd + n] = OVER_FACTOR * c_rem
    obj[be:be + n] = -UNDER_FACTOR * c_rem
    obj[bu:bu + n] = EMERGENCY_FACTOR * c_rem
    obj[bx:bx + n] += EPS_THROUGHPUT
    obj[by:by + n] += EPS_THROUGHPUT
    if lam is not None:
        obj[bS + n - 1] = -lam

    lb = np.zeros(N); ub = np.full(N, np.inf)

    ub[bx:bx + n] = np.minimum(B_MAX, np.maximum(p_rem + pv_kwh - load
        _kwh, 0.0))
    ub[by:by + n] = B_MAX
    ub[bv:bv + n] = pv_kwh
    ub[bu:bu + n] = np.maximum(load_kwh - pv_kwh, 0.0)
    ub[be:be + n] = p_rem
    lb[bS:bS + n] = S_MIN
    ub[bS:bS + n] = S_MAX
    if s_end is not None:
        lb[bS + n - 1] = ub[bS + n - 1] = s_end

    res = linprog(obj, A_eq=A_eq, b_eq=b_eq, bounds=list(zip(lb, ub)),
        method="highs")
    if not res.success:
        return None
    z = res.x
    q = p_rem + z[bd:bd + n] - z[be:be + n]
    return {
        "q": q, "x": z[bx:bx + n], "y": z[by:by + n],
        "v": z[bv:bv + n], "u": z[bu:bu + n],
        "S": z[bS:bS + n], "objective": float(res.fun),
    }

def settle_q12(p, u, price):
    return float(np.sum(price * p) + EMERGENCY_FACTOR * np.sum(price *
        u))

def settle_q3(p, q, u, price):
    over = np.maximum(q - p, 0.0)
    under = np.maximum(p - q, 0.0)
    return float(np.sum(price * np.minimum(p, q) + OVER_FACTOR * price
        * over
        + UNDER_FACTOR * price * under
        + EMERGENCY_FACTOR * price * u))

def forecast_f1(day_index, load_act, pv_act, load1, pv1):
    if day_index >= 7:
        lhat = load_act[day_index - 7].copy()
        vhat = pv_act[day_index - 7:day_index].mean(axis=0)

    elif day_index >= 1:
        lhat = load_act[:day_index].mean(axis=0)
        vhat = pv_act[:day_index].mean(axis=0)
    else:
        lhat = load1.copy()
        vhat = pv1.copy()
    return lhat, vhat

def empirical_quantile(samples, alpha):
    s = np.sort(np.asarray(samples, dtype=float))
    if s.size == 0:
        return 0.0
    k = int(np.ceil(alpha * s.size))
    k = min(max(k, 1), s.size)
    return float(s[k - 1])

def residual_quantile_safe(day_index, nhath_kw, vhat_kw, resid_hist,
    alpha, W):
    lo = max(1, day_index - W)
    n = len(nhath_kw)
    q = np.zeros(n)
    for t in range(n):
        samples = [resid_hist[(j, t)] for j in range(lo, day_index)
            if (j, t) in resid_hist]
        q[t] = empirical_quantile(samples, alpha) if samples else 0.0
    ntilde = nhath_kw + q
    vtilde = np.maximum(np.maximum(vhat_kw, -ntilde), 0.0)
    ltilde = vtilde + ntilde
    return ltilde, vtilde, ntilde

def bias_corrected_official_pv(day_index, issue_slot, pv_act, fc_raw,
    err_hist, W,
    anchor_prev):
    raw = fc_raw[day_index, issue_slot, :].copy()
    corr = np.zeros(24)
    for h in range(1, 25):
        key_h = h - 1
        samples = [err_hist[(j, issue_slot, key_h)] for j in range(
            max(0, day_index - W), day_index) if (j, issue_slot, key_h)
            in err_hist]
        corr[key_h] = float(np.mean(samples)) if samples else 0.0
    return np.maximum(raw + corr, 0.0)

def interp_official_profile(corrected_hourly, issue_hour, anchor_kw):
    pos0 = issue_hour * 6
    knots_x = [pos0]
    knots_y = [anchor_kw]
    for h in range(1, 25):
        pos = pos0 + 6 * h
        if pos > 144:
            break
        knots_x.append(pos)
        knots_y.append(corrected_hourly[h - 1])
    grid = np.arange(knots_x[0], knots_x[-1] + 1)
    prof = np.interp(grid, knots_x, knots_y)
    out = np.zeros(T + 1)
    out[knots_x[0]:knots_x[-1] + 1] = prof
    if knots_x[0] > 0:
        out[:knots_x[0]] = knots_y[0]
    return out

def price_forecast_0h(day_index, price_act, price1, base="median30"):
    if day_index <= 0:
        return price1.copy()
    lo = max(0, day_index - 30)
    hist = price_act[lo:day_index]
    if hist.shape[0] == 0:
        return price1.copy()
    if base == "median30":

```

```

        return np.median(hist, axis=0)
    if base == "mean30":
        return hist.mean(axis=0)
    if base == "mean7":
        return price_act[max(0, day_index - 7):day_index].mean(axis=0)
    if base == "prev_day":
        return price_act[day_index - 1].copy()
    raise ValueError(base)

def price_intraday_correct(base_pred, price_act_day, t, n_recent=18,
    decay=3.0, floor=1e-4):
    lo = max(0, t - n_recent)
    if t <= 0 or lo >= t:
        m = 0.0
    else:
        m = float(np.mean(price_act_day[lo:t] - base_pred[lo:t]))
    lead = (np.arange(len(base_pred)) - t) * DT
    corr = np.where(lead > 0, m * np.exp(-lead / decay), 0.0)
    return np.maximum(base_pred + corr, floor)

def four_hour_blocks(arr):
    a = np.asarray(arr, dtype=float).reshape(6, 24)
    return a.sum(axis=1)

def merge_intervals(mask, threshold=1e-8):
    spans = []
    t = 0
    n = len(mask)
    while t < n:
        if mask[t]:
            start = t
            while t + 1 < n and mask[t + 1]:
                t += 1
            spans.append((start, t))
            t += 1
    return spans

def slot_label_start(i):
    m = i * 10
    return f"{m // 60}:{m % 60:02d}"

def slot_label_end(i):
    m = (i + 1) * 10
    if m == 1440:
        return "0:00+1"
    return f"{m // 60}:{m % 60:02d}"

def interval_label(i0, i1):
    return f"{slot_label_start(i0)}-{slot_label_end(i1)}"

```

Listing 3: solution_lib.py

```

from __future__ import annotations

import time

import numpy as np
import openpyxl
import pandas as pd

try:
    import microgrid_core as mc
except ModuleNotFoundError:
    from scripts import microgrid_core as mc

```

```

ROOT = mc.ROOT
TPL_DIR = ROOT / "data" / "附件" / "附件5"
RES = ROOT / "results"
CSV = RES / "csv"
FIG = ROOT / "figures"

REP_DATES = [_dt.date(2025, 3, 20), _dt.date(2025, 6, 21),
    _dt.date(2025, 9, 23), _dt.date(2025, 12, 21)]
REP_SLOTS = [60, 72, 84, 96, 108, 120]
DAY_INDEX = {d: (d - mc.DAY0).days for d in REP_DATES}
BLOCK_LABELS = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
    "12:00-16:00", "16:00-20:00", "20:00-24:00"]

def predict_center(kind: str, d: int, data: dict):
    load_act, pv_act = data["load_act"], data["pv_act"]
    if kind == "F1":
        return mc.forecast_f1(d, load_act, pv_act, data["load1"], data[
            "pv1"])
    if kind == "F2":
        cand = [load_act[d - k] for k in (7, 14, 21) if d - k >= 0]
        if not cand:
            lhat = load_act[:d].mean(axis=0) if d >= 1 else data["load1"]
        elif len(cand) == 1:
            lhat = cand[0].copy()
        else:
            lhat = np.median(np.vstack(cand), axis=0)
        if d >= 1:
            vhat = pv_act[max(0, d - 7):d].mean(axis=0)
        else:
            vhat = data["pv1"].copy()
        return lhat, vhat
    raise ValueError(kind)

def net_residual_matrix(kind: str, data: dict) -> np.ndarray:
    load_act, pv_act = data["load_act"], data["pv_act"]
    out = np.zeros((365, mc.T))
    for d in range(365):
        lhat, vhat = predict_center(kind, d, data)
        out[d] = (load_act[d] - pv_act[d]) - (lhat - vhat)
    return out

def safe_trajectory(d: int, lhat_kw, vhat_kw, resid: np.ndarray,
    alpha: float, W: int):
    lo = max(1, d - W)
    if d <= 0 or lo >= d:
        q = np.zeros(mc.T)
    else:
        srt = np.sort(resid[lo:d], axis=0)
        k = int(np.ceil(alpha * srt.shape[0]))
        k = min(max(k, 1), srt.shape[0])
        q = srt[k - 1]
    ntilde = lhat_kw - vhat_kw + q
    vtilde = np.maximum(np.maximum(vhat_kw, -ntilde), 0.0)
    ltilde = vtilde + ntilde
    return ltilde, vtilde, ntilde

def point_error_metrics(kind: str, data: dict) -> dict:
    eL, eV, eN = [], [], []
    for d in range(31, 365):
        lhat, vhat = predict_center(kind, d, data)
        eL.append(data["load_act"][d] - lhat)
        eV.append(data["pv_act"][d] - vhat)
        eN.append((data["load_act"][d] - data["pv_act"][d]) - (lhat -
            vhat))
    eL, eV, eN = np.asarray(eL), np.asarray(eV), np.asarray(eN)

```

```

return {
    "load_MAE": float(np.abs(eL).mean()),
    "load_Bias": float(eL.mean()),
    "pv_MAE": float(np.abs(eV).mean()),
    "pv_Bias": float(eV.mean()),
    "net_MAE": float(np.abs(eN).mean()),
    "net_RMSE": float(np.sqrt((eN ** 2).mean())),
    "net_Bias": float(eN.mean()),
}

def run_year_q2(data: dict, kind: str = "F1", alpha: float = 0.805, W
: int = 35,
    price_mode: str = "fixed", resid: np.ndarray | None =
        None,
    no_battery: bool = False, horizon: int = 36,
    n_days: int = 365, executor: str = "mpc",
    terminal_value_factor: float = 1.0) -> dict:
    if not no_battery:
        if price_mode == "fixed":
            mpc_price_mode, settle_price = "fixed", "fixed"
        elif price_mode == "actual":
            mpc_price_mode, settle_price = "oracle", "actual"
        else:
            raise ValueError(f"未知价格模式: {price_mode}")
    run = run_year_q34(
        data, prof=None, kind=kind, alpha=alpha, W=W, mode="uniform
        ",
        H=horizon, allowed=(0,), updates=(), pv_source="center",
        price_mode=mpc_price_mode, settle_price=settle_price,
        n_days=n_days, resid_override=resid, executor=executor,
        terminal_value_factor=terminal_value_factor,
    )
    return run

load_act, pv_act = data["load_act"], data["pv_act"]
s = float(mc.S0)
n = n_days
P = np.zeros((n, mc.T))
X = np.zeros((n, mc.T))
Y = np.zeros((n, mc.T))
U = np.zeros((n, mc.T))
Wd = np.zeros((n, mc.T))
Vv = np.zeros((n, mc.T))
S = np.zeros((n, mc.T))
cash = np.zeros(n)
pv_used_ratio = np.zeros(n)
load_kwh = load_act * mc.DT
pv_kwh = pv_act * mc.DT
for d in range(n):
    c = data["price1"] if price_mode == "fixed" else data["price_
act"][d]
    L = load_kwh[d]
    V = pv_kwh[d]
    p = np.maximum(L - V, 0.0)
    S[d] = s
    Vv[d] = np.minimum(V, L)
    U[d] = np.maximum(L - V - p, 0.0)
    Wd[d] = np.maximum(p - (L - Vv[d]), 0.0)
    cash[d] = float(np.sum(c * p) + mc.EMERGENCY_FACTOR * np.sum(c
* U[d]))
    P[d] = p
return dict(P=P, X=X, Y=Y, U=U, W=Wd, V=Vv, S=S, cash=cash,
    kind=kind, alpha=alpha, resid_W=W, price_mode=price_mode
    ,
    S_init=mc.S0, S_end=float(s), pv_curtail_ratio=pv_used_
ratio,
    resid=resid, load_kwh=load_kwh, pv_kwh=pv_kwh,
    load_kw=load_act, pv_kw=pv_act)

def perfect_foresight_q2(data: dict, horizon: int = 36) -> dict:
    load_act, pv_act = data["load_act"], data["pv_act"]
    s = float(mc.S0)
    n = 365
    out = {k: np.zeros((n, mc.T)) for k in ("P", "X", "Y", "U", "W", "
V", "S")}
    cash = np.zeros(n)
    for d in range(n):
        c = data["price1"]
        L = load_act[d] * mc.DT
        V = pv_act[d] * mc.DT
        lp = mc.solve_day_lp(c, L, V, s, lam=mc.ETA_D * float(np.median
(c)))
        p = np.maximum(lp["g"], 0.0)
        pv_versions = np.tile(V, (N_ISSUE, 1))
        price_seg_fn = lambda t, K, c=c: c[t:K + 1]
        ex = rolling_execute_q34(
            p, L, pv_versions, L, V, price_seg_fn, s,
            horizon=horizon, allowed=(0,), time_limit=5.0,
        )
        out["P"][d] = p
        out["X"][d], out["Y"][d] = ex["x"], ex["y"]
        out["U"][d], out["W"][d], out["V"][d], out["S"][d] = ex["u"],
            ex["w"], ex["v"], ex["s"]
        cash[d] = mc.settle_q12(out["P"][d], out["U"][d], c)
        s = float(min(max(ex["S"][-1], mc.S_MIN), mc.S_MAX))
    out["cash"] = cash
    out["S_end"] = s
    return out

def official_slice(arr, offset: int = 31):
    return arr[offset:365]

def year_metrics(run: dict, offset: int = 31) -> dict:
    cash = official_slice(run["cash"], offset)
    U = official_slice(run["U"], offset)
    P = official_slice(run["P"], offset)
    Y = official_slice(run["Y"], offset)
    X = official_slice(run["X"], offset)
    Wd = official_slice(run["W"], offset)
    S = official_slice(run["S"], offset)
    price = None
    return {
        "daily_cash": cash,
        "cash_total": float(cash.sum()),
        "contract_kwh": float(P.sum()),
        "charge_kwh": float(X.sum()),
        "discharge_kwh": float(Y.sum()),
        "unused_contract_kwh": float(Wd.sum()),
        "emergency_kwh": float(U.sum()),
        "emergency_slots": int((U > 1e-8).sum()),
        "S_min": float(S.min()), "S_max": float(S.max()),
        "S_end": float(run["S_end"]),
    }

def energy_block_table(run: dict, day_indices, offset: int = 31) ->
    pd.DataFrame:
    rows = []
    for d in day_indices:
        ch = mc.four_hour_blocks(run["X"][d])
        dis = mc.four_hour_blocks(run["Y"][d])
        s0 = run["S"][d - 1][-1] if d >= 1 else mc.S0
        for k in range(6):
            rows.append({"日期": str(mc.date_of(d)), "时间段": BLOCK_
LABELS[k],
                "充电量_kWh": ch[k], "放电量_kWh": dis[k],
                "日初储能_kWh": s0, "日末储能_kWh": run["S"][
d][-1]})

```

```

return pd.DataFrame(rows)

def slot_table(run: dict, price_day, day_indices, slots=None) -> pd.
    DataFrame:
    slots = REP_SLOTS if slots is None else slots
    rows = []
    for d in day_indices:
        c = price_day if np.ndim(price_day) == 1 else price_day[d]
        for t in slots:
            rows.append({
                "日期": str(mc.date_of(d)),
                "时段": f"{mc.slot_label_start(t)}-{mc.slot_label_end(t)}",
                "电价_元每kWh": c[t],
                "负荷_kW": float(run["load_kw"][d][t]),
                "光伏实际_kW": float(run["pv_kw"][d][t]),
                "合同购电_p_kWh": run["P"][d][t],
                "实际购电_kWh": run["P"][d][t] - run["W"][d][t] + run["U"]
                    [d][t],
                "应急购电_u_kWh": run["U"][d][t],
                "充电量_kWh": run["X"][d][t],
                "放电量_kWh": run["Y"][d][t],
                "时段末储电量_kWh": run["S"][d][t],
            })
    return pd.DataFrame(rows)

def emergency_intervals(U_day: np.ndarray, date, threshold: float = 1
    e-8,
                        max_intervals: int = 3) -> list:
    spans = mc.merge_intervals(np.asarray(U_day) > threshold)
    out = []
    for i0, i1 in spans[:max_intervals]:
        out.append({"日期": str(date), "购电时间段": mc.interval_label(
            i0, i1),
                    "购电量_kWh": float(U_day[i0:i1 + 1].sum())})
    return out

def emergency_table(run: dict, day_indices, threshold: float = 1e-8)
    -> pd.DataFrame:
    rows = []
    for d in day_indices:
        ivs = emergency_intervals(run["U"][d], mc.date_of(d), threshold)
        if not ivs:
            rows.append({"日期": str(mc.date_of(d)), "购电时间段": "无",
                        "购电量_kWh": 0.0})
        rows.extend(ivs)
    return pd.DataFrame(rows)

def check_run(run: dict, offset: int = 31) -> dict:
    P, X, Y, U, Wd, V, S = (np.asarray(run[k])
                            for k in ("P", "X", "Y", "U", "W", "V", "S"))

    n = P.shape[0]
    L = np.asarray(run["load_kw"])[0:n]
    pv = np.asarray(run["pv_kw"])[0:n]
    bal = np.abs((P - Wd) + V + Y + U - X - L)
    soc_chain = 0.0
    for d in range(1, n):
        expected = S[d - 1, -1] + mc.ETA_C * X[d, 0] - Y[d, 0] / mc.ETA
            _D
        soc_chain = max(soc_chain, abs(S[d, 0] - expected))
    price = run.get("price_settle")
    settle = float("nan") if price is None else float(np.abs(
        np.asarray(run["cash"])[0:n]
        - (np.asarray(run["cash_contract"])[0:n]
          + mc.EMERGENCY_FACTOR * (np.asarray(price)[0:n] * U).sum(axis
            =1))
    ).max())
    bounds = {
        "bound_p": float(-min(P.min(), 0.0)),
        "bound_x": float(max(-min(X.min(), 0.0), (X - mc.B_MAX).max())),
        ,
        "bound_y": float(max(-min(Y.min(), 0.0), (Y - mc.B_MAX).max())),
        ,
        "bound_v": float(max(-min(V.min(), 0.0), (V - pv).max())),
        "bound_u": float(-min(U.min(), 0.0)),
        "bound_w": float(max(-min(Wd.min(), 0.0), (Wd - P).max())),
        "bound_S": float(max(mc.S_MIN - S.min(), S.max() - mc.S_MAX)),
    }
    return {
        "balance_max": float(bal.max()),
        "overlap_max": float(np.minimum(X, Y).max()),
        "charge_emergency_max": float(np.minimum(X, U).max()),
        "S_min": float(S.min()), "S_max": float(S.max()),
        "soc_chain_max_kWh": float(soc_chain),
        "soc_end_mismatch_kWh": float(abs(S[-1, -1] - run["S_end"])),
        "settle_flow_max_yuan": settle,
        "official_days": int(max(0, n - offset)),
        **bounds,
    }

def _date_blocks(ws, n_rows: int) -> list:
    blocks = []
    for row in range(2, n_rows + 1):
        value = ws.cell(row=row, column=1).value
        if value is None:
            continue
        text = str(value).strip()
        if text[:4].isdigit() and "-" in text:
            blocks.append((row, text[:10]))
    return blocks

def _day_index_of(date_text: str, offset: int = 31):
    try:
        d = _dt.date.fromisoformat(date_text[:10])
    except ValueError:
        return None
    idx = (d - mc.DAY0).days
    return idx if offset <= idx < 365 else None

def _fill_plan_sheet(ws, run: dict, offset: int = 31, key: str = "P",
    fee_key: str = "cash") -> None:
    arr = run[key]
    fee = run.get(fee_key, run["cash"])
    for r, d in enumerate(range(offset, arr.shape[0])):
        row = 2 + r
        a = arr[d]
        for t in range(mc.T):
            ws.cell(row=row, column=2 + t).value = float(a[t])
            ws.cell(row=row, column=146).value = float(a.sum())
            ws.cell(row=row, column=147).value = float(fee[d])

def _fill_cycle_sheet(ws, run: dict, offset: int = 31) -> None:
    for base, date_text in _date_blocks(ws, ws.max_row):
        d = _day_index_of(date_text, offset)
        if d is None or d >= run["X"].shape[0]:
            continue
        ch = mc.four_hour_blocks(run["X"][d])
        dis = mc.four_hour_blocks(run["Y"][d])
        for k in range(6):
            ws.cell(row=base + k, column=3).value = float(ch[k])
            ws.cell(row=base + k, column=4).value = float(dis[k])
            ws.cell(row=base, column=6).value = float(
                run["S"][d - 1][-1] if d >= 1 else mc.S0)
            ws.cell(row=base + 1, column=6).value = float(run["S"][d][-1])

```

```

def _fill_emergency_sheet(ws, run: dict, offset: int = 31) -> None:
    for base, date_text in _date_blocks(ws, ws.max_row):
        d = _day_index_of(date_text, offset)
        if d is None or d >= run["U"].shape[0]:
            continue
        ivs = emergency_intervals(run["U"][d], mc.date_of(d))
        if not ivs:
            ws.cell(row=base, column=2).value = "无"
            ws.cell(row=base, column=3).value = 0.0
            continue
        for k, iv in enumerate(ivs):
            ws.cell(row=base + k, column=2).value = iv["购电时段"]
            ws.cell(row=base + k, column=3).value = iv["购电量_kWh"]

def write_workbook(template: str, out_name: str, sheets: dict) -> None:
    RES.mkdir(parents=True, exist_ok=True)
    wb = openpyxl.load_workbook(TPL_DIR / template)
    for name, filler in sheets.items():
        filler(wb[name])
    wb.save(RES / out_name)

def monthly_summary(run: dict, offset: int = 31) -> pd.DataFrame:
    rows = []
    for d in range(offset, run["P"].shape[0]):
        dt = mc.date_of(d)
        rows.append({"日期": dt, "月份": dt.strftime("%Y-%m"),
            "现金费_元": run["cash"][d],
            "应急购电量_kWh": run["U"][d].sum(),
            "合同购电量_kWh": run["P"][d].sum(),
            "充电量_kWh": run["X"][d].sum(),
            "放电量_kWh": run["Y"][d].sum(),
            "弃电量_kWh": run["W"][d].sum(),
            "弃光量_kWh": float((run["pv_kwh"][d] - run["V"][d]).sum()),
            "pv_kwh": run["pv_kwh"][d] if "pv_kwh" in run else np.nan,
            "日末储电量_kWh": run["S"][d][-1]})
    return pd.DataFrame(rows)

ISSUE_SLOTS = (0, 36, 72, 108)
ISSUE_HOURS = (0, 6, 12, 18)
N_ISSUE = 4
N_STRATA = 6

def build_pv_archive(data: dict, W: int = 30) -> np.ndarray:
    pv_act = data["pv_act"]
    fc = data["fc_raw"]
    prof = np.zeros((365, N_ISSUE, mc.T + 1))
    for d in range(365):
        lo = max(0, d - W)
        for r in range(N_ISSUE):
            corr = np.zeros(24)
            if d > lo:
                for h in range(1, 25):
                    pos = ISSUE_SLOTS[r] + 6 * h
                    if pos > mc.T:
                        break
                    corr[h - 1] = float(np.mean(
                        [pv_act[j, pos - 1] - fc[j, r, h - 1] for j in
                        range(lo, d)]))
            corrected = np.maximum(fc[d, r, :] + corr, 0.0)
            if r == 0:
                anchor = float(pv_act[d - 1, -1]) if d >= 1 else 0.0
            else:
                anchor = float(pv_act[d, ISSUE_SLOTS[r] - 1])
            prof[d, r] = mc.interp_official_profile(corrected, ISSUE_

HOURS[r], anchor)
    return prof

def issue_index_of_slot(t: int, allowed=(0, 1, 2, 3)) -> int:
    best = 0
    for r in allowed:
        if ISSUE_SLOTS[r] <= t:
            best = r
    return best

def version_of_slot_array(allowed=(0, 1, 2, 3)) -> np.ndarray:
    return np.array([issue_index_of_slot(t, allowed) for t in range(mc
        .T)], dtype=int)

def pv_version_curve(prof_day: np.ndarray, allowed=(0, 1, 2, 3)) ->
    np.ndarray:
    return np.array([prof_day[issue_index_of_slot(t, allowed), t + 1]
        for t in range(mc.T)])

def horizon_stratum(allowed=(0, 1, 2, 3)) -> np.ndarray:
    out = np.empty(mc.T, dtype=int)
    for t in range(mc.T):
        r = issue_index_of_slot(t, allowed)
        out[t] = min((t - ISSUE_SLOTS[r]) // 6, N_STRATA - 1)
    return out

STRATUM = horizon_stratum()

def q3_net_residual(data: dict, prof: np.ndarray, allowed=(0, 1, 2,
    3),
        kind: str = "F1", pv_source: str = "archive"):
    pv_ver = np.zeros((365, mc.T))
    resid = np.zeros((365, mc.T))
    for d in range(365):
        lhat, vhat = predict_center(kind, d, data)
        if pv_source == "archive":
            pv_ver[d] = pv_version_curve(prof[d], allowed)
        else:
            pv_ver[d] = vhat
        resid[d] = (data["load_act"][d] - data["pv_act"][d]) - (lhat -
            pv_ver[d])
    return resid, pv_ver

def q3_reserve(d: int, resid: np.ndarray, alpha: float, W: int,
    mode: str = "stratified", allowed=(0, 1, 2, 3)):
    lo = max(1, d - W)
    if d <= 0 or lo >= d:
        return np.zeros(mc.T), np.ones(N_STRATA)
    hist = resid[lo:d]
    if mode == "uniform":
        return (np.array([mc.empirical_quantile(hist[:, t], alpha)
            for t in range(mc.T)]), np.ones(N_STRATA))
    stratum = horizon_stratum(allowed)
    sig = np.ones(N_STRATA)
    for h in range(N_STRATA):
        cols = np.where(stratum == h)[0]
        if cols.size:
            sig[h] = float(np.sqrt((hist[:, cols] ** 2).mean()))
    sig = np.maximum.accumulate(np.maximum(sig, 1e-9))
    z = hist / sig[stratum][None, :]
    qs = mc.empirical_quantile(z.ravel(), alpha)
    return sig[stratum] * qs, sig

def safe_from_reserve(lhat_kwh, vhat_kwh, reserve_kwh):
    ntilde = lhat_kwh - vhat_kwh + reserve_kwh
    vtilde = np.maximum(np.maximum(vhat_kwh, -ntilde), 0.0)

```

```

return vtilde + ntilde, vtilde, ntilde

def rolling_execute_q34(g_contract, load_pred_kwh, pv_ver_kwh, load_kwh, pv_kwh,
                        price_seg_fn, s_init, horizon: int = 36,
                        allowed=(0, 1, 2, 3), time_limit: float = 5.0,
                        contract_update_fn=None, executor: str = "mpc",
                        terminal_value_factor: float = 1.0):
    n = mc.T
    contract = np.asarray(g_contract, dtype=float).copy()
    x = np.zeros(n); y = np.zeros(n); u = np.zeros(n)
    w = np.zeros(n); v = np.zeros(n); S = np.zeros(n)
    plan_x = np.zeros(n); plan_y = np.zeros(n); plan_s_next = np.zeros(n)
    s = float(s_init)
    stat = {"clip_charge": 0, "clip_discharge": 0, "fallback": 0,
            "updates": 0, "lp_time": []}
    if executor not in ("mpc", "greedy"):
        raise ValueError(f"未知执行器: {executor}")
    prev_net = None
    for t in range(n):
        if contract_update_fn is not None:
            q_rem = contract_update_fn(t, s, contract)
            if q_rem is not None:
                q_rem = np.asarray(q_rem, dtype=float)
                if q_rem.shape != (n - t,):
                    raise ValueError(f"合同更新新长度异常: t={t}, shape={q_rem.shape}")
                contract[t:] = np.maximum(q_rem, 0.0)
                stat["updates"] += 1
        r = issue_index_of_slot(t, allowed)
        load_curve = (load_pred_kwh[r] if np.ndim(load_pred_kwh) == 2
                      else load_pred_kwh)
        if executor == "mpc":
            K = min(t + horizon - 1, n - 1)
            seg = slice(t, K + 1)
            c_seg = price_seg_fn(t, K)
            lam = terminal_value_factor * mc.ETA_D * float(np.median(c_seg))
            tic = time.perf_counter()
            act = mc.rolling_lp_action(s, contract[seg], load_curve[seg],
                                     pv_ver_kwh[r][seg], c_seg, lam,
                                     time_limit=time_limit)
            stat["lp_time"].append(time.perf_counter() - tic)
            if act is None:
                net_est = float(prev_net) if prev_net is not None else float(
                    load_curve[t] - pv_ver_kwh[r][t])
                if net_est < 0.0:
                    xm = min(mc.B_MAX, -net_est,
                             max((mc.S_MAX - s) / mc.ETA_C, 0.0))
                    ym = 0.0
                else:
                    xm = 0.0
                    ym = min(mc.B_MAX, net_est,
                             max((s - mc.S_MIN) * mc.ETA_D, 0.0))
                stat["fallback"] += 1
            else:
                xm, ym = act
        else:
            surplus = max(contract[t] + pv_ver_kwh[r][t] - load_curve[t], 0.0)
            deficit = max(load_curve[t] - contract[t] - pv_ver_kwh[r][t], 0.0)
            xm = min(mc.B_MAX, surplus, max((mc.S_MAX - s) / mc.ETA_C, 0.0))
            ym = min(mc.B_MAX, deficit, max((s - mc.S_MIN) * mc.ETA_D, 0.0))
            xe, ye, ue, we, ve, _ = mc.clip_slot(xm, ym, contract[t], load_kwh[t],
                                                  pv_kwh[t])
            plan_x[t], plan_y[t] = xm, ym

```

```

plan_s_next[t] = min(max(s + mc.ETA_C * xm - ym / mc.ETA_D,
                          mc.S_MIN), mc.S_MAX)
if xm - xe > 1e-9:
    stat["clip_charge"] += 1
if ym - ye > 1e-9:
    stat["clip_discharge"] += 1
x[t], y[t], u[t], w[t], v[t] = xe, ye, ue, we, ve
s = s + mc.ETA_C * xe - ye / mc.ETA_D
s = min(max(s, mc.S_MIN), mc.S_MAX)
S[t] = s
prev_net = float(load_kwh[t] - pv_kwh[t])
return {"x": x, "y": y, "u": u, "w": w, "v": v, "S": S,
        "plan_x": plan_x, "plan_y": plan_y,
        "plan_s_next": plan_s_next,
        "contract": contract, "stats": stat}

def run_year_q34(data: dict, prof: np.ndarray | None = None, kind:
                 str = "F1",
                 alpha: float = 0.7725, W: int = 35, mode: str = "stratified",
                 H: int = 6, allowed=(0, 1, 2, 3), updates=(1, 2, 3),
                 pv_source: str = "archive", price_mode: str = "fixed",
                 price_base: str = "median30", settle_price: str = "fixed",
                 time_limit: float = 5.0, n_days: int = 365,
                 resid_override: np.ndarray | None = None,
                 executor: str = "mpc", terminal_value_factor: float = 1.0) -> dict:
    if prof is None and pv_source == "archive":
        prof = build_pv_archive(data, W)
    resid, pv_ver = q3_net_residual(data, prof, allowed, kind, pv_source)
    if resid_override is not None:
        resid = np.asarray(resid_override, dtype=float)
    load_kw = data["load_act"]
    pv_kw = data["pv_act"]
    load_kwh = load_kw * mc.DT
    pv_kwh = pv_kw * mc.DT
    keys = ("P", "Q", "X", "Y", "U", "W", "V", "S",
            "plan_x", "plan_y", "plan_s_next")
    arr = {k: np.zeros((n_days, mc.T)) for k in keys}
    cash = np.zeros(n_days)
    cash_plan = np.zeros(n_days)
    cash_contract = np.zeros(n_days)
    price_settle = np.zeros((n_days, mc.T))
    stats = {"clip_charge": 0, "clip_discharge": 0, "fallback": 0,
            "updates": 0, "lp_time": []}
    daily_cnt = {k: np.zeros(n_days, dtype=int)
                  for k in ("clip_charge", "clip_discharge", "fallback",
                           "updates")}
    lp_by_day = []
    s = float(mc.S0)
    for d in range(n_days):
        lhat_kw, vhat_kw = predict_center(kind, d, data)
        load_pred = lhat_kw * mc.DT
        if pv_source == "archive":
            pv_curves = np.vstack([prof[d, r, 1:mc.T + 1] for r in
                                   range(N_ISSUE)]) * mc.DT
        else:
            pv_curves = np.tile(vhat_kw * mc.DT, (N_ISSUE, 1))
        if price_mode == "fixed":
            forecast_price = data["price1"]
            price_seg_fn = (lambda t, K, bp=forecast_price: bp[t:K + 1])
        elif price_mode == "oracle":
            forecast_price = data["price_act"][d]
            price_seg_fn = (lambda t, K, bp=forecast_price: bp[t:K + 1])
        else:
            forecast_price = mc.price_forecast_0h(d, data["price_act"],

```

```

        data["price1"],
                                price_base)

    def price_seg_fn(t, K, bp=forecast_price, d=d):
        return mc.price_intraday_correct(bp, data["price_act"][d
            ], t)[t:K + 1]

settle_day_price = data["price1"] if settle_price == "fixed"
    else data["price_act"][d]
price_settle[d] = settle_day_price

reserve_kw, _ = q3_reserve(d, resid, alpha, W, mode, allowed)
reserve = reserve_kw * mc.DT

safe_load_curves = np.zeros_like(pv_curves)
safe_pv_curves = np.zeros_like(pv_curves)
for r in range(N_ISSUE):
    safe_load_curves[r], safe_pv_curves[r], _ = safe_from_
        reserve(
            load_pred, pv_curves[r], reserve
        )

ltil, vtil = safe_load_curves[0], safe_pv_curves[0]
lp = mc.solve_day_lp(forecast_price, ltil, vtil, s,
    lam=terminal_value_factor * mc.ETA_D * float(
        np.median(forecast_price)))
p = np.maximum(lp["g"], 0.0)
q = p.copy()
cash_plan[d] = float(np.sum(forecast_price * p))
update_at = {ISSUE_SLOTS[r]: r for r in updates}

def update_contract(t, current_s, current_q):
    r_up = update_at.get(t)
    if r_up is None:
        return None
    c_up = price_seg_fn(t, mc.T - 1)
    ltil_up = safe_load_curves[r_up][t:]
    vtil_up = safe_pv_curves[r_up][t:]

    up = mc.contract_update_lp(
        p[t:], c_up, ltil_up, vtil_up, current_s,
        lam=terminal_value_factor * mc.ETA_D * float(np.median(c
            _up))),
    )
    return None if up is None else up["q"]

ex = rolling_execute_q34(q, safe_load_curves, safe_pv_curves,
    load_kwh[d], pv_kwh[d],
    price_seg_fn, s, horizon=H, allowed=
        allowed,
    time_limit=time_limit,
    contract_update_fn=update_contract if
        updates else None,
    executor=executor,
    terminal_value_factor=terminal_value_
        factor)

q = ex["contract"]
cash_contract[d] = mc.settle_q3(p, q, np.zeros(mc.T), settle_
    day_price)
for k, key in (("x", "X"), ("y", "Y"), ("u", "U"), ("w", "W"),
    ("v", "V"), ("S", "S"), ("plan_x", "plan_x"),
    ("plan_y", "plan_y"), ("plan_s_next", "plan_s_next
        ")):
    arr[key][d] = ex[k]
arr["P"][d] = p
arr["Q"][d] = q
cash[d] = mc.settle_q3(p, q, ex["u"], settle_day_price)
for k in ("clip_charge", "clip_discharge", "fallback", "updates
    "):
    stats[k] += ex["stats"][k]
stats["lp_time"].extend(ex["stats"]["lp_time"])
for k in ("clip_charge", "clip_discharge", "fallback", "updates
    "):
    daily_cnt[k][d] += ex["stats"][k]

    lp_by_day.append(list(ex["stats"]["lp_time"]))
    s = float(min(max(ex["S"][-1], mc.S_MIN), mc.S_MAX))
stats["daily_cnt"] = daily_cnt
stats["lp_by_day"] = lp_by_day
out = dict(arr)
out.update(cash=cash, cash_plan=cash_plan, cash_contract=cash_
    contract,
    price_settle=price_settle,
    stats=stats, kind=kind, alpha=alpha, resid=W, mode=mode
        , H=H,
    allowed=tuple(allowed), updates=tuple(updates), pv_source
        =pv_source,
    price_mode=price_mode, price_base=price_base,
    settle_price=settle_price, executor=executor,
    terminal_value_factor=terminal_value_factor,
    S_init=mc.S0, S_end=float(s),
    resid=resid, pv_ver=pv_ver, load_kwh=load_kwh, pv_kwh=pv_
        kwh,
    load_kw=load_kw, pv_kw=pv_kw)
return out

def _sliced_count(stats: dict, key: str, offset: int) -> int:
    dc = stats.get("daily_cnt")
    if dc is None:
        return int(stats[key])
    return int(np.asarray(dc[key])[offset:].sum())

def year_metrics_q34(run: dict, offset: int = 31) -> dict:
    P = official_slice(run["P"], offset)
    Q = official_slice(run["Q"], offset)
    U = official_slice(run["U"], offset)
    X = official_slice(run["X"], offset)
    Y = official_slice(run["Y"], offset)
    Wd = official_slice(run["W"], offset)
    S = official_slice(run["S"], offset)
    lp = np.asarray(run["stats"]["lp_time"], dtype=float)
    by_day = run["stats"].get("lp_by_day")
    if by_day is not None:
        lp = np.asarray([v for day in by_day[offset:] for v in day],
            dtype=float)
    return {
        "daily_cash": official_slice(run["cash"], offset),
        "cash_total": float(official_slice(run["cash"], offset).sum()),
        "plan_kwh": float(P.sum()),
        "contract_kwh": float(Q.sum()),
        "adjust_up_kwh": float(np.maximum(Q - P, 0.0).sum()),
        "adjust_down_kwh": float(np.maximum(P - Q, 0.0).sum()),
        "charge_kwh": float(X.sum()),
        "discharge_kwh": float(Y.sum()),
        "unused_contract_kwh": float(Wd.sum()),
        "emergency_kwh": float(U.sum()),
        "emergency_slots": int((U > 1e-8).sum()),
        "S_min": float(S.min()), "S_max": float(S.max()), "S_end":
            float(run["S_end"]),
        "updates": _sliced_count(run["stats"], "updates", offset),
        "clip_charge": _sliced_count(run["stats"], "clip_charge",
            offset),
        "clip_discharge": _sliced_count(run["stats"], "clip_discharge",
            offset),
        "fallback": _sliced_count(run["stats"], "fallback", offset),
        "lp_ms_median": float(np.median(lp) * 1e3) if lp.size else
            float("nan"),
        "lp_ms_p95": float(np.percentile(lp, 95) * 1e3) if lp.size else
            float("nan"),
        "lp_ms_max": float(lp.max() * 1e3) if lp.size else float("nan")
    }

def check_run_q34(run: dict, offset: int = 31) -> dict:
    P, Q, X, Y, U, Wd, V, S = (np.asarray(run[k])

```

```

        for k in ("P", "Q", "X", "Y", "U", "W", "V",
                  "S"))

n = P.shape[0]

L, pv = np.asarray(run["load_kwh"][:n], np.asarray(run["pv_kwh"][:n])

bal = np.abs((Q - Wd) + V + Y + U - X - L)

b_p = float(-min(P.min(), 0.0))
b_q = float(-min(Q.min(), 0.0))
b_x = float(max(-min(X.min(), 0.0), (X - mc.B_MAX).max()))
b_y = float(max(-min(Y.min(), 0.0), (Y - mc.B_MAX).max()))
b_v = float(max(-min(V.min(), 0.0), (V - pv).max()))
b_u = float(-min(U.min(), 0.0))
b_w = float(max(-min(Wd.min(), 0.0), (Wd - Q).max()))
b_s = float(max(mc.S_MIN - S.min(), S.max() - mc.S_MAX))
chain = 0.0
for d in range(1, n):
    chain = max(chain, abs((S[d][0] - S[d - 1][-1])
                           - (mc.ETA_C * X[d][0] - Y[d][0] / mc.ETA_D))
                )

price = run.get("price_settle")
if price is None:
    flow = float("nan")
else:
    flow = float(np.abs(run["cash"] - (run["cash_contract"]
                                       + mc.EMERGENCY_FACTOR * (price * U
                                                                  ).sum(axis=1))))

return {
    "balance_max_kWh": float(bal.max()),
    "overlap_max_kWh": float(np.minimum(X, Y).max()),
    "charge_emergency_max_kWh": float(np.minimum(X, U).max()),
    "bound_p": b_p, "bound_q": b_q, "bound_x": b_x, "bound_y": b_y,
    "bound_v": b_v, "bound_u": b_u, "bound_w": b_w, "bound_S": b_s,
    "soc_chain_max_kWh": float(chain),
    "soc_end_mismatch_kWh": float(abs(S[-1][-1] - run["S_end"])),
    "settle_flow_max_yuan": flow,
    "official_days": int(uniform_slice_count(n, offset)),
}

def uniform_slice_count(n: int, offset: int) -> int:
    return max(0, n - offset)

```

Listing 4: 问题 1_ 求解.py

```

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))

import matplotlib.pyplot as plt
import numpy as np
import openpyxl
import pandas as pd

import microgrid_core as mc
import utils.plot_style as ps

ROOT = mc.ROOT
TPL_DIR = ROOT / "data" / "附件" / "附件5"
RES = ROOT / "results"
CSV = RES / "csv"
FIG = ROOT / "figures" / "问题1"

def solve() -> dict:
    data = mc.load_inputs()

```

```

c = data["price1"]
load_kwh = data["load1"] * mc.DT
pv_kwh = data["pv1"] * mc.DT
res = mc.solve_day_lp(c, load_kwh, pv_kwh, mc.S0, s_end=mc.S0)

S = np.concatenate([[mc.S0], res["S"]])
g = np.maximum(res["g"], 0.0)
x = np.maximum(res["x"], 0.0)
y = np.maximum(res["y"], 0.0)
v = np.maximum(res["v"], 0.0)
out = dict(
    c=c, load_kwh=load_kwh, pv_kwh=pv_kwh,
    g=g, x=x, y=y, v=v, S=S,
    curtail=np.maximum(res["curtail"], 0.0),
    cash_cost=float(np.sum(c * g)),
    solver_objective=res["objective"],
)
return out

def verify(sol: dict) -> dict:
    g, x, y, v, S = sol["g"], sol["x"], sol["y"], sol["v"], sol["S"]
    L, V = sol["load_kwh"], sol["pv_kwh"]
    balance_res = float(np.abs(g + v + y - x - L).max())
    soc_res = float(np.abs(np.diff(S) - (mc.ETA_C * x - y / mc.ETA_D))
                    .max())
    bound_res = float(max(
        max(np.maximum(S - mc.S_MAX, 0).max(), np.maximum(mc.S_MIN - S,
                                                             0).max()),
        max(np.maximum(x - mc.B_MAX, 0).max(), np.maximum(y - mc.B_MAX,
                                                             0).max()),
        max(np.maximum(v - V, 0).max(), -min(v.min(), 0.0)),
        -min(g.min(), 0.0),
    ))
    overlap = float(np.minimum(x, y).max())
    X, Y, G = float(x.sum()), float(y.sum()), float(g.sum())
    ident = {
        "Y_minus_0.81X": Y - mc.ETA_C * mc.ETA_D * X,
        "G_identity": G - (float(L.sum()) - float(v.sum()) + (1 - mc.
                                                                ETA_C * mc.ETA_D) * X),
        "no_battery_purchase": float((L - V).clip(min=0).sum()),
    }
    return dict(
        balance_max=balance_res, soc_res_max=soc_res, bound_max=bound_res,
        overlap_max=overlap, X=X, Y=Y, G=G,
        S_min=float(S.min()), S_max=float(S.max()),
        curtail_total=float(sol["curtail"].sum()),
        cash_cost=float(sol["cash_cost"]),
        solver_objective=float(sol["solver_objective"]),
        **ident,
    )

def write_template(sol: dict) -> None:
    RES.mkdir(parents=True, exist_ok=True)
    wb = openpyxl.load_workbook(TPL_DIR / "result1.xlsx")
    ws = wb["计划购电量"]
    for i in range(mc.T):
        ws.cell(row=2 + i, column=2).value = float(sol["g"][i])
    ws2 = wb["充放电量"]
    ch = mc.four_hour_blocks(sol["x"])
    dis = mc.four_hour_blocks(sol["y"])
    for k in range(6):
        ws2.cell(row=2 + k, column=2).value = float(ch[k])
        ws2.cell(row=2 + k, column=3).value = float(dis[k])
    ws2.cell(row=2, column=5).value = float(sol["S"][0])
    ws2.cell(row=3, column=5).value = float(sol["S"][-1])
    wb.save(RES / "result1.xlsx")

def write_csv(sol: dict, met: dict) -> None:
    CSV.mkdir(parents=True, exist_ok=True)
    idx = np.arange(mc.T)

```

```

df = pd.DataFrame({
    "时段序号": idx + 1,
    "自然区间": [f"{mc.slot_label_start(i)}-{mc.slot_label_end(i)}"
                  for i in idx],
    "电价_元每kWh": sol["c"],
    "负荷_kWh": sol["load_kwh"],
    "光伏可用_kWh": sol["pv_kwh"],
    "光伏利用_kWh": sol["v"],
    "弃光_kWh": sol["curtail"],
    "计划购电量_kWh": sol["g"],
    "充电量_kWh": sol["x"],
    "放电量_kWh": sol["y"],
    "时段末储能_kWh": sol["S"][1:],
})

df.to_csv(CSV / "问题1_逐时段计划.csv", index=False, encoding="utf-8-sig")

blocks = pd.DataFrame({
    "时段": ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00",
            "16:00-20:00", "20:00-24:00"],
    "充电量_kWh": mc.four_hour_blocks(sol["x"]),
    "放电量_kWh": mc.four_hour_blocks(sol["y"]),
})

blocks.loc[len(blocks)] = ["全天", blocks["充电量_kWh"].sum(),
                           blocks["放电量_kWh"].sum()]

blocks.to_csv(CSV / "问题1_四小时聚合.csv", index=False, encoding="utf-8-sig")

summary = pd.DataFrame({{
    "项目": "数值", "全天计划购电量_kWh": met["G"],
    "全天购电费_元": met["cash_cost"],
    "含吞吐正则的求解目标值": met["solver_objective"],
    "全天充电量_kWh": met["X"], "全天放电量_kWh": met["Y"],
    "0时储能_kWh": sol["S"][0], "24时储能_kWh": sol["S"][-1],
    "弃光量_kWh": met["curtail_total"],
    "无储能购电量_kWh": met["no_battery_purchase"],
}})

summary.to_csv(CSV / "问题1_汇总.csv", index=False, encoding="utf-8-sig")

with open(CSV / "问题1_校验.json", "w", encoding="utf-8") as fh:
    json.dump(met, fh, ensure_ascii=False, indent=2)

def figures(sol: dict, met: dict) -> None:
    FIG.mkdir(parents=True, exist_ok=True)
    ps.apply_publication_style("zh", "report")
    t = (np.arange(mc.T) + 0.5) * mc.DT
    L = sol["load_kwh"] / mc.DT
    V = sol["pv_kwh"] / mc.DT
    g = sol["g"] / mc.DT

    fig, axes = ps.publication_subplots(3, 1, aspect=1.55, height_ratios=[1, 1.35, 1.25])
    ax = axes[0]
    ax.plot(t, sol["c"], color=ps.PALETTE["secondary"])
    ax.set_ylabel("电价/(元/kWh)")
    ax.set_title("典型日电价")
    ax.set_xlim(0, 24)

    ax = axes[1]
    ax.plot(t, L, color=ps.PALETTE["dark"], label="负荷")
    ax.plot(t, V, color=ps.PALETTE["positive"], label="可用光伏")
    ax.plot(t, g, color=ps.PALETTE["primary"], label="计划购电")
    ax.plot(t, sol["v"] / mc.DT, color=ps.PALETTE["sky"], lw=0.9,
            label="光伏利用")
    ax.set_ylabel("功率/kW")
    ax.set_title("负荷、光伏与计划购电")
    ax.legend(ncols=2)
    ax.set_xlim(0, 24)

    ax = axes[2]
    ax.plot(t, sol["S"][1:], color=ps.PALETTE["contrast"], label="储能")
    ax.axhline(mc.S_MIN, color=ps.PALETTE["neutral"], lw=0.7, ls="--",
               label="SOC 上下限")
    ax.axhline(mc.S_MAX, color=ps.PALETTE["neutral"], lw=0.7, ls="--")
    ax.set_ylabel("储能/kWh")
    ax.set_title("储能状态轨迹")
    ax.set_xlabel("时刻/h")
    ax.legend()
    ax.set_xlim(0, 24)
    ps.add_panel_labels(axes)
    ps.export_figure(fig, FIG / "问题1_典型日最优计划")
    plt.close(fig)

fig, ax = ps.publication_subplots(1, 1, aspect=0.55)
labels = ["0-4", "4-8", "8-12", "12-16", "16-20", "20-24"]
pos = np.arange(6)
ch = mc.four_hour_blocks(sol["x"])
dis = mc.four_hour_blocks(sol["y"])
ax.bar(pos - 0.18, ch, 0.36, color=ps.PALETTE["primary"], label="充电量")
ax.bar(pos + 0.18, dis, 0.36, color=ps.PALETTE["contrast"], label="放电量")
ax.set_xticks(pos, labels)
ax.set_xlabel("时段/h")
ax.set_ylabel("电量/kWh")
ax.set_title("四小时聚合充放电量")
ax.legend()
ps.export_figure(fig, FIG / "问题1_四小时聚合充放电")
plt.close(fig)

fig, axes = ps.publication_subplots(1, 2, aspect=0.72)
g0 = (sol["load_kwh"] - sol["pv_kwh"]).clip(min=0)
labels = ["有储能", "无储能"]
axes[0].bar(labels, [met["G"], met["no_battery_purchase"]],
            color=[ps.PALETTE["primary"], ps.PALETTE["neutral"]],
            width=0.5)
axes[0].set_ylabel("全天购电量/kWh")
axes[0].set_title("全天购电量")
axes[1].bar(labels, [met["objective"], float((sol["c"] * g0).sum())],
            color=[ps.PALETTE["primary"], ps.PALETTE["neutral"]],
            width=0.5)
axes[1].set_ylabel("全天购电费/元")
axes[1].set_title("全天购电费")
ps.export_figure(fig, FIG / "问题1_储能效益对比")
plt.close(fig)

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--no-figures", action="store_true",
                    help="只生成数值结果和模板, 不生成或修改图片")
    args = ap.parse_args()
    sol = solve()
    met = verify(sol)
    write_template(sol)
    write_csv(sol, met)
    if not args.no_figures:
        figures(sol, met)
    print(json.dumps(met, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

Listing 5: 问题 2_ 求解.py

```

import os
import sys
import time
from multiprocessing import Pool
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl
import utils.plot_style as ps

FIG = sl.FIG / "问题2"

_SENS_G = {}

def _sens_init(data: dict) -> None:
    _SENS_G["data"] = data

def _sens_worker(cfg: tuple) -> dict:
    kind, W, alpha, tag, horizon = cfg
    data = _SENS_G["data"]
    resid = sl.net_residual_matrix(kind, data)
    run = sl.run_year_q2(data, kind, alpha, W, resid=resid, horizon=
        horizon)
    out = {k: v for k, v in sl.year_metrics(run).items() if k != "
        daily_cash"}
    out.update({"方案": tag, "预测器": kind, "W": W, "alpha": alpha, "H":
        horizon})
    if tag.startswith("主配置"):
        out["_run"] = run
    return out

def sensitivity(data: dict, resid_cache: dict, workers: int = 1) ->
    pd.DataFrame:
    configs = [
        ("F1", 35, 0.805, "主配置 F1-W35-a0.805-H49", 49),
        ("F1", 35, 0.800, "分位 a=0.800", 49),
        ("F1", 35, 0.810, "分位 a=0.810", 49),
        ("F1", 35, 0.805, "时域 H=48", 48),
        ("F1", 35, 0.805, "时域 H=50", 50),
        ("F1", 30, 0.805, "窗口 W=30", 49),
        ("F1", 45, 0.805, "窗口 W=45", 49),
        ("F2", 35, 0.805, "预测器 F2", 49),
    ]
    if workers > 1:
        with Pool(workers, initializer=_sens_init, initargs=(data,)) as
            pool:
                packed = pool.map(_sens_worker, configs)
        base = next(row.pop("_run") for row in packed if "_run" in row)
        return pd.DataFrame(packed, base)
    rows = []
    base = None
    for kind, W, alpha, tag, horizon in configs:
        if kind not in resid_cache:
            resid_cache[kind] = sl.net_residual_matrix(kind, data)
        run = sl.run_year_q2(data, kind, alpha, W, resid=resid_cache[
            kind], horizon=horizon)
        m = {k: v for k, v in sl.year_metrics(run).items() if k != "
            daily_cash"}
        m.update({"方案": tag, "预测器": kind, "W": W, "alpha": alpha,
            "H": horizon})
        rows.append(m)
        if tag.startswith("主配置"):
            base = run
    return pd.DataFrame(rows, base)

def figures(data: dict, run: dict, base: dict, sens: pd.DataFrame,
    day: int = 265) -> None:
    FIG.mkdir(parents=True, exist_ok=True)
    ps.apply_publication_style("zh", "report")
    t = (np.arange(mc.T) + 0.5) * mc.DT
    ticks = list(range(0, 25, 4))

    lhat, vhat = sl.predict_center("F1", day, data)
    fig, axes = ps.publication_subplots(1, 3, aspect=0.95)
    axes[0].plot(t, data["load_act"][day], color=ps.PALETTE["dark"],
        label="实测")
    axes[0].plot(t, lhat, color=ps.PALETTE["primary"], label="F1 预测")
    axes[0].set_title("负荷")
    axes[0].set_ylabel("功率/kW")
    axes[1].plot(t, data["pv_act"][day], color=ps.PALETTE["dark"],
        label="实测")
    axes[1].plot(t, vhat, color=ps.PALETTE["positive"], label="F1 预测")
    axes[1].set_title("光伏")
    axes[2].plot(t, data["load_act"][day] - data["pv_act"][day],
        color=ps.PALETTE["dark"], label="实测")
    axes[2].plot(t, lhat - vhat, color=ps.PALETTE["contrast"], label="
        F1 预测")
    axes[2].set_title("净负荷")
    for ax in axes:
        ax.set_xlabel("时刻/h")
        ax.set_xticks(ticks)
        ax.legend()
    ps.add_panel_labels(axes)
    fig.suptitle(f"F1 中心预测与实测 ({mc.date_of(day)}) ", fontsize
        =8.5)
    ps.export_figure(fig, FIG / "问题2_中心预测对比")
    plt.close(fig)

    ltilde, vtilde, ntilde = sl.safe_trajectory(day, lhat, vhat, run["
        resid"], 0.9, 30)
    nbar = lhat - vhat
    nact = data["load_act"][day] - data["pv_act"][day]
    ncontract = run["P"][day] / mc.DT
    fig, ax = ps.publication_subplots(1, 1, aspect=0.6)
    ax.fill_between(t, nbar, ntilde, color=ps.PALETTE["sky"], alpha
        =0.35,
        label="分位风险余量")
    ax.plot(t, nbar, color=ps.PALETTE["primary"], label="中心净负荷")
    ax.plot(t, nact, color=ps.PALETTE["dark"], label="实测净负荷")
    ax.plot(t, ncontract, color=ps.PALETTE["contrast"], lw=0.9, label="
        合同购电")
    ax.set_xlabel("时刻/h")
    ax.set_ylabel("功率/kW")
    ax.set_xticks(ticks)
    ax.legend(ncols=2)
    ax.set_title("安全轨迹、合同与实测净负荷")
    fig.suptitle(f"{mc.date_of(day)}", fontsize=8.5)
    ps.export_figure(fig, FIG / "问题2_安全轨迹与合同")
    plt.close(fig)

    days = np.arange(31, 365)
    dates = [mc.date_of(int(d)) for d in days]
    fig, axes = ps.publication_subplots(2, 1, aspect=1.15, height_
        ratios=[1.3, 1])
    axes[0].plot(dates, run["cash"][31:365], color=ps.PALETTE["primary"]
        , lw=0.8)
    axes[0].set_ylabel("日现金流/元")
    axes[0].set_title("逐日购电现金流")
    axes[1].bar(dates, run["U"][31:365].sum(axis=1), color=ps.PALETTE[
        "contrast"],
        width=0.9)
    axes[1].set_ylabel("应急购电量/kWh")
    axes[1].set_title("逐日应急购电量")

```

```

ps.add_panel_labels(axes)
ps.export_figure(fig, FIG / "问题2_年度费用与应急")
plt.close(fig)

fig, axes = ps.publication_subplots(2, 1, aspect=1.3, height_
    ratios=[1, 1])
pos = (np.arange(mc.T) + 0.5) * mc.DT
axes[0].bar(pos, run["X"][day], 0.15, color=ps.PALETTE["primary"],
    label="充电量")
axes[0].bar(pos, -run["Y"][day], 0.15, color=ps.PALETTE["contrast"]
    ], label="放电量")
axes[0].axhline(0, color=ps.PALETTE["neutral"], lw=0.6)
axes[0].set_ylabel("电量/kWh")
axes[0].set_title("实际充放电")
axes[0].legend()
axes[1].plot(pos, run["S"][day], color=ps.PALETTE["secondary"],
    label="实际储电量")
axes[1].axhline(mc.S_MIN, color=ps.PALETTE["neutral"], ls="--", lw
    =0.7, label="SOC 上下限")
axes[1].axhline(mc.S_MAX, color=ps.PALETTE["neutral"], ls="--", lw
    =0.7)
axes[1].set_ylabel("储电量/kWh")
axes[1].set_xlabel("时刻/h")
axes[1].set_title("储能状态")
axes[1].legend()
for ax in axes:
    ax.set_xlim(0, 24)
    ax.set_xticks(ticks)
ps.add_panel_labels(axes)
fig.suptitle(f"{mc.date_of(day)}", fontsize=8.5)
ps.export_figure(fig, FIG / "问题2_重点日充放电与储能")
plt.close(fig)

fig, axes = ps.publication_subplots(1, 3, aspect=0.8)
cfg = sens.set_index("方案")
MAIN = "主配置 F1-W30-a0.9"
groups = [
    ("分位 alpha", ["分位 a=0.8", MAIN, "分位 a=0.95"],
    ["0.8", "0.9(主)", "0.95"]),
    ("残差窗口 W", ["窗口 W=21", MAIN, "窗口 W=45"],
    ["21", "30(主)", "45"]),
]
for ax, (name, keys, labs) in zip(axes[:2], groups):
    ax.bar(labs, [float(cfg.loc[k, "cash_total"])/1e4 for k in
        keys],
        color=ps.PALETTE["primary"], width=0.55)
    ax.set_title(name)
    ax.set_ylabel("年度现金费/万元")
lab = ["F1", "F2"]

vals = [float(sens.loc[sens["预测器"] == k, "cash_total"].iloc[0])
    / 1e4 for k in lab]
axes[2].bar(lab, vals, color=ps.PALETTE["primary"], width=0.55)
axes[2].set_title("预测器")
axes[2].set_ylabel("年度现金费/万元")
ps.add_panel_labels(axes)
ps.export_figure(fig, FIG / "问题2_配置敏感性")
plt.close(fig)

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--main-only", action="store_true",
        help="只运行 F1-W30-alpha0.9 主配置, 不跑敏感性和绘图"
    )
    ap.add_argument("--executor", choices=["mpc", "greedy"], default="
        mpc",
        help="执行器; greedy 为不读取当前时段实测的预测驱动贪心"
    )
    ap.add_argument("--days", type=int, default=365,
        help="运行前 N 天; 小于 365 时只作 Lite 烟雾验证")
    ap.add_argument("--no-figures", action="store_true",
        help="完成全部计算与表格输出, 但不生成图件")

    ap.add_argument("--workers", type=int, default=0,
        help="完整敏感性计算的并行进程数")

    args = ap.parse_args()
    t0 = time.perf_counter()
    data = mc.load_inputs()
    sl.CSV.mkdir(parents=True, exist_ok=True)

    if args.main_only:
        resid = sl.net_residual_matrix("F1", data)
        run = sl.run_year_q2(data, "F1", 0.805, 35, resid=resid,
            horizon=49, n_days=args.days, executor=args.
            executor)

        gates = sl.check_run(run)
        with open(sl.CSV / "问题2_门禁.json", "w", encoding="utf-8") as
            fh:
                json.dump(gates, fh, ensure_ascii=False, indent=2)
        daily = sl.monthly_summary(run)
        daily.to_csv(sl.CSV / "问题2_逐日汇总.csv", index=False,
            encoding="utf-8-sig")

        if args.days == 365:
            sl.write_workbook("result2.xlsx", "result2.xlsx", {
                "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run),
                "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run),
                "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws,
                    run),
            })
            np.savez_compressed(sl.CSV / "q2_bundle.npz",
                **{k: run[k] for k in
                    ("P", "X", "Y", "U", "W", "V", "S",
                    "cash", "plan_x", "plan_y", "plan_s_
                    next")})

        metrics = {k: v for k, v in sl.year_metrics(run).items()
            if k != "daily_cash"}
        print(json.dumps(dict(gates=gates, **metrics), ensure_ascii=
            False,
            indent=2))
        print("elapsed_s", round(time.perf_counter() - t0, 1))
        return

    err = pd.DataFrame([dict(预测器=k, **sl.point_error_metrics(k,
        data))
        for k in ("F1", "F2")])
    err.to_csv(sl.CSV / "问题2_预测误差对比.csv", index=False, encoding=
        "utf-8-sig")

    resid_cache = {k: sl.net_residual_matrix(k, data) for k in ("F1",
        "F2")}
    workers = args.workers or min(4, max(1, (os.cpu_count() or 4) - 2)
    )
    sens, run = sensitivity(data, resid_cache, workers=workers)
    sens.to_csv(sl.CSV / "问题2_配置敏感性.csv", index=False, encoding=
        "utf-8-sig")

    pf = sl.perfect_foresight_q2(data)
    nb = sl.run_year_q2(data, no_battery=True)
    ref = pd.DataFrame([
        dict(方案="主配置", **{k: v for k, v in sl.year_metrics(run).
            items()
            if k != "daily_cash"}),
        dict(方案="理想信息参照", **{k: v for k, v in sl.year_metrics(pf
            ).items()
            if k != "daily_cash"}),
        dict(方案="无储能", **{k: v for k, v in sl.year_metrics(nb).
            items()
            if k != "daily_cash"}),
    ])
    ref.to_csv(sl.CSV / "问题2_参照方案.csv", index=False, encoding="
        utf-8-sig")

    gates = sl.check_run(run)
    with open(sl.CSV / "问题2_门禁.json", "w", encoding="utf-8") as fh

```

```

:
json.dump(gates, fh, ensure_ascii=False, indent=2)

day_idx = [sl.DAY_INDEX[d] for d in sl.REP_DATES]
t1 = sl.slot_table(run, data["price1"], day_idx)
t1.to_csv(sl.CSV / "问题2_表1_指定时段.csv", index=False, encoding="utf-8-sig")
t2 = sl.energy_block_table(run, day_idx)
t2.to_csv(sl.CSV / "问题2_表2_四小时聚合.csv", index=False, encoding="utf-8-sig")
t3 = sl.emergency_table(run, day_idx)
t3.to_csv(sl.CSV / "问题2_表3_紧急购电.csv", index=False, encoding="utf-8-sig")

daily = sl.monthly_summary(run)
daily.to_csv(sl.CSV / "问题2_逐日汇总.csv", index=False, encoding="utf-8-sig")

sl.write_workbook("result2.xlsx", "result2.xlsx", {
    "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run),
    "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run),
    "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run),
})

if not args.no_figures:
    figures(data, run, pf, sens)
print(json.dumps(dict(gates=gates, **{k: v for k, v in sl.year_metrics(run).items()
                        if k != "daily_cash"}), ensure_ascii=False, indent=2))
print("elapsed_s", round(time.perf_counter() - t0, 1))

if __name__ == "__main__":
    main()

```

Listing 6: 问题 3_ 求解.py

```

from __future__ import annotations

import argparse
import json
import os
import time
from multiprocessing import Pool

import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

KEYS = ("P", "Q", "X", "Y", "U", "W", "V", "S", "cash", "cash_plan",
        "cash_contract", "plan_x", "plan_y", "plan_s_next")
FULL_SET = (0, 1, 2, 3)

def _cfg(tag, mode, H, allowed, updates, note, keep=False):
    return {"tag": tag, "mode": mode, "H": H, "allowed": tuple(allowed),
            "updates": tuple(updates), "note": note, "keep": keep}

def build_configs() -> list:
    cfs = [
        _cfg("main", "stratified", 6, FULL_SET, (1, 2, 3),
            "主配置: 分层储备 + H=6 + 三次调整", keep=True),
        _cfg("uniform", "uniform", 6, FULL_SET, (1, 2, 3),
            "对照: 统一逐时段分位储备 + H=6", keep=True),
        _cfg("h72", "stratified", 72, FULL_SET, (1, 2, 3),
            "对照: 分层储备 + H=72", keep=True),
    ]

```

```

]
for allowed, updates, note in [
    ((0,), ()), "消融 空集: 仅 0:00 预报, 仍共享每 10 min 重算机会",
    ((0, 1), (1,)), "消融 {6}",
    ((0, 2), (2,)), "消融 {12}",
    ((0, 3), (3,)), "消融 {18}",
    ((0, 1, 2), (1, 2)), "消融 {6,12}",
    ((0, 1, 3), (1, 3)), "消融 {6,18}",
    ((0, 2, 3), (2, 3)), "消融 {12,18}",
]:
    tag = "abl_" + (".".join(str(h) for h in updates) or "none")
    cfs.append(_cfg(tag, "stratified", 6, allowed, updates, note,
                    keep=True))

return cfs

```

```
_G = {}
```

```

def _init(data, prof, n_days):
    _G["data"] = data
    _G["prof"] = prof
    _G["n_days"] = n_days

```

```

def _coverage(data, prof, allowed, mode, alpha, W, d0=31, d1=365):
    resid, pv_ver = sl.q3_net_residual(data, prof, allowed, "F1", "archive")
    L, V = data["load_act"], data["pv_act"]
    R = np.empty((d1 - d0, mc.T))
    for i, d in enumerate(range(d0, d1)):
        lhat, _ = sl.predict_center("F1", d, data)
        res_kw, _ = sl.q3_reserve(d, resid, alpha, W, mode, allowed)
        R[i] = (L[d] - V[d]) - (lhat - pv_ver[d] + res_kw)
    resid_f1 = resid[d0:d1]
    return {
        "Ncov": float((R <= 0.0).mean()),
        "pinball": float((R * (alpha - (R < 0.0))).mean()),
        "resid_MAE_kW": float(np.abs(resid_f1).mean()),
        "resid_RMSE_kW": float(np.sqrt((resid_f1 ** 2).mean()),
    }

```

```

def _worker(cfg):
    data, prof, n_days = _G["data"], _G["prof"], _G["n_days"]
    run = sl.run_year_q34(data, prof, "F1", 0.7725, 35, cfg["mode"],
                          cfg["H"],
                          cfg["allowed"], cfg["updates"], "archive", "fixed",
                          "median30", "fixed", 5.0, n_days)
    out = dict(cfg)
    if n_days > 31:
        out["metrics"] = sl.year_metrics_q34(run, 31)
        out["coverage"] = _coverage(data, prof, cfg["allowed"], cfg["mode"],
                                     0.7725, 35)
        out["gates"] = sl.check_run_q34(run, 31)
    if cfg["keep"]:
        out["arrays"] = {k: run[k] for k in KEYS}
        out["stats"] = run["stats"]
        out["S_end"] = run["S_end"]
        out["resid"] = run["resid"]
        out["pv_ver"] = run["pv_ver"]
    return out

```

```

def slot_table_q3(run, price_day, day_indices):
    rows = []
    for d in day_indices:

```

```

c = price_day if np.ndim(price_day) == 1 else price_day[d]
for t in sl.REP_SLOTS:
    rows.append({
        "日期": str(mc.date_of(d)),
        "时段": "%s-%s" % (mc.slot_label_start(t), mc.slot_label_end(t)),
        "电价_元每kWh": c[t],
        "负荷_kW": float(run["load_kw"][d][t]),
        "光伏实际_kW": float(run["pv_kw"][d][t]),
        "原计划合同_p_kWh": run["P"][d][t],
        "最终合同_q_kWh": run["Q"][d][t],
        "实际购电_kWh": run["Q"][d][t] - run["W"][d][t] + run["U"][d][t],
        "应急购电_u_kWh": run["U"][d][t],
        "充电量_kWh": run["X"][d][t],
        "放电量_kWh": run["Y"][d][t],
        "时段末储电量_kWh": run["S"][d][t],
        "合同调整量_kWh": run["Q"][d][t] - run["P"][d][t],
    })
return pd.DataFrame(rows)

def ablation_table(records):
    by_tag = {r["tag"]: r for r in records if "metrics" in r}
    base = by_tag["abl_none"]["metrics"]
    rows = []
    for rec in records:
        if "metrics" not in rec or not (rec["tag"].startswith("abl_") or rec["tag"] == "main"):
            continue
        m = rec["metrics"]
        dn = int(m["updates"] - base["updates"])
        dc = float(base["cash_total"] - m["cash_total"])
        rows.append({
            "获准集合S": ("{6,12,18}" if rec["tag"] == "main" else ("{" if rec["tag"] == "abl_none" else rec["note"].replace("消融", ""))),
            "tag": rec["tag"],
            "允许发布时刻": " ".join(str(sl.ISSUE_HOURS[r]) for r in rec["allowed"]),
            "现金总费_元": m["cash_total"],
            "计划购电量_kWh": m["plan_kwh"],
            "最终合同量_kWh": m["contract_kwh"],
            "调整事件数": m["updates"],
            "dn_相对空集": dn,
            "应急购电量_kWh": m["emergency_kwh"],
            "应急时段数": m["emergency_slots"],
            "弃购量_kWh": m["unused_contract_kwh"],
            "期末储电量_kWh": m["S_end"],
            "相对空集节约_元": dc,
            "kappa_star_元每次": (dc / dn) if dn > 0 else float("nan"),
        })
    df = pd.DataFrame(rows)
    cash = {r["tag"]: (r["metrics"]["cash_total"], r["allowed"]) for r in records if "metrics" in r and (r["tag"].startswith("abl_") or r["tag"] == "main")}
    for r in (1, 2, 3):
        diffs = []
        for tag, (c_s, allowed) in cash.items():
            if r in allowed:
                continue
            enlarged = tuple(sorted((set(allowed) | {r}) - {0}))
            tag2 = "main" if enlarged == (1, 2, 3) else ("abl_" + ("_" if len(enlarged) == 2 else ".") + str(h) for h in enlarged) or "none")
            if tag2 in cash:
                diffs.append(c_s - cash[tag2][0])
        df["版本%d时段_平均边际收益_元" % sl.ISSUE_HOURS[r]] = (float(np.mean(diffs)) if diffs else float("nan"))
    return df

def lead_scale_table(data, prof, offset=31):
    rows = []
    for r in range(sl.N_ISSUE):
        for h in range(1, 25):
            pos = sl.ISSUE_SLOTS[r] + 6 * h
            if pos > mc.T:
                continue
            err = np.array([data["pv_act"][d, pos - 1] - prof[d, r, pos] for d in range(offset, 365)])
            rows.append({
                "发布时刻_h": sl.ISSUE_HOURS[r], "提前期_h": h,
                "MAE_kW": float(np.abs(err).mean()),
                "RMSE_kW": float(np.sqrt((err ** 2).mean())),
                "Bias_kW": float(err.mean()),
                "q90_kW": mc.empirical_quantile(err, 0.9),
                "尺度_s_kW": float(np.sqrt((err ** 2).mean())),
            })
    df = pd.DataFrame(rows)
    agg = df.groupby("提前期_h").agg(
        MAE_kW=("MAE_kW", "mean"), RMSE_kW=("RMSE_kW", "mean"),
        Bias_kW=("Bias_kW", "mean"), q90_kW=("q90_kW", "mean")).reset_index()
    return df, agg

def monthly_table(run, offset=31):
    rows = []
    for d in range(offset, run["P"].shape[0]):
        dt = mc.date_of(d)
        rows.append({
            "日期": dt, "月份": dt.strftime("%Y-%m"),
            "现金费_元": run["cash"][d],
            "计划购电量_kWh": run["P"][d].sum(),
            "最终合同量_kWh": run["Q"][d].sum(),
            "调整量_kWh": (run["Q"][d] - run["P"][d]).sum(),
            "应急购电量_kWh": run["U"][d].sum(),
            "充电量_kWh": run["X"][d].sum(),
            "放电量_kWh": run["Y"][d].sum(),
            "弃购量_kWh": run["W"][d].sum(),
            "弃光量_kWh": float((run["pv_kwh"][d] - run["V"][d]).sum()),
            "日末储电量_kWh": run["S"][d][-1],
        })
    return pd.DataFrame(rows)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--days", type=int, default=365)
    ap.add_argument("--workers", type=int, default=0)
    ap.add_argument("--main-only", action="store_true")
    args = ap.parse_args()
    t0 = time.perf_counter()
    mc.RESULT_DIR.mkdir(parents=True, exist_ok=True)
    sl.CSV.mkdir(parents=True, exist_ok=True)

    data = mc.load_inputs()
    prof = sl.build_pv_archive(data, 35)
    cfgs = build_configs()
    if args.main_only:
        cfgs = cfgs[:1]
    n_cpu = os.cpu_count() or 4
    workers = args.workers or min(len(cfgs), max(1, n_cpu - 4))
    print("[Q3] configs=%d workers=%d days=%d" % (len(cfgs), workers, args.days), flush=True)

    if workers <= 1:
        _init(data, prof, args.days)
        records = [_worker(c) for c in cfgs]
    else:
        with Pool(workers, initializer=_init, initargs=(data, prof, args.days)) as pool:

```

```

        records = pool.map(_worker, cfigs)
    print("[Q3] runs done in %.0fs" % (time.perf_counter() - t0),
        flush=True)

specs = []
for rec in records:
    if "metrics" not in rec:
        continue
    row = {"方案": rec["note"], "tag": rec["tag"], "mode": rec["mode"],
        "H": rec["H"], "updates": rec["updates"]}
    row.update({k: v for k, v in rec["metrics"].items() if k != "daily_cash"})
    row.update({"cov_" + k: v for k, v in rec["coverage"].items()})
    specs.append(row)
pd.DataFrame(specs).to_csv(sl.CSV / "问题3_方案对比.csv", index=False,
    encoding="utf-8-sig")

if any(r["tag"].startswith("abl_") for r in records):
    ablation_table(records).to_csv(sl.CSV / "问题3_信息消融.csv",
        index=False,
        encoding="utf-8-sig")

lead_df, lead_agg = lead_scale_table(data, prof)
lead_df.to_csv(sl.CSV / "问题3_预见期误差_分来源.csv", index=False,
    encoding="utf-8-sig")
lead_agg.to_csv(sl.CSV / "问题3_预见期尺度.csv", index=False,
    encoding="utf-8-sig")

main_rec = next(r for r in records if r["tag"] == "main")
run = dict(main_rec["arrays"])
run["stats"] = main_rec["stats"]
run["S_end"] = main_rec["S_end"]
run["resid"] = main_rec["resid"]
run["pv_ver"] = main_rec["pv_ver"]
run["load_kwh"] = data["load_act"] * mc.DT
run["pv_kwh"] = data["pv_act"] * mc.DT
run["load_kw"] = data["load_act"]
run["pv_kw"] = data["pv_act"]
run["price_settle"] = np.tile(data["price1"], (run["P"].shape[0],
    1))

n_run = run["P"].shape[0]
day_idx = [sl.DAY_INDEX[d] for d in sl.REP_DATES if sl.DAY_INDEX[d]
    ] < n_run
if day_idx:
    slot_table_q3(run, data["price1"], day_idx).to_csv(
        sl.CSV / "问题3_表1_指定时段.csv", index=False, encoding="utf-8-sig")
    sl.energy_block_table(run, day_idx).to_csv(
        sl.CSV / "问题3_表2_四小时聚合.csv", index=False, encoding="utf-8-sig")
    sl.emergency_table(run, day_idx).to_csv(
        sl.CSV / "问题3_表3_紧急购电.csv", index=False, encoding="utf-8-sig")
monthly_table(run).to_csv(sl.CSV / "问题3_逐日汇总.csv", index=False,
    encoding="utf-8-sig")

gates = dict(main_rec["gates"])
gates["coverage"] = main_rec["coverage"]
gates["all_numeric_ok"] = bool(
    gates["balance_max_kWh"] <= 1e-6 and gates["overlap_max_kWh"]
        <= 1e-5
    and gates["charge_emergency_max_kWh"] <= 1e-5
    and max(gates[k] for k in ("bound_p", "bound_q", "bound_x", "bound_y",
        "bound_v", "bound_u", "bound_w", "bound_S")) <= 1e-7
    and gates["settle_flow_max_yuan"] <= 1e-5)
with open(sl.CSV / "问题3_门禁.json", "w", encoding="utf-8") as fh:

```

```

    json.dump(gates, fh, ensure_ascii=False, indent=2, default=float)

sl.write_workbook("result3.xlsx", "result3.xlsx", {
    "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run, key="P",
        fee_key="cash_plan"),
    "调整购电量": lambda ws: sl._fill_plan_sheet(ws, run, key="Q",
        fee_key="cash_contract"),
    "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run),
    "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run),
})

np.savez_compressed(sl.CSV / "q3_bundle.npz",
    **{"%s_%s" % (r["tag"], k): r["arrays"][k]
        for r in records if "arrays" in r for k in KEYS})

def _jsonable(rec):
    out = {}
    for k, v in rec.items():
        if k in ("arrays", "resid", "pv_ver", "stats"):
            continue
        if k == "metrics":
            v = {kk: vv for kk, vv in v.items() if kk != "daily_cash"}
        out[k] = v
    return out

meta = {"days": args.days, "configs": [_jsonable(r) for r in records]}
with open(sl.CSV / "q3_metrics.json", "w", encoding="utf-8") as fh:
    json.dump(meta, fh, ensure_ascii=False, indent=2, default=float)

summary = {k: v for k, v in main_rec["metrics"].items() if k != "daily_cash"}
print(json.dumps({"summary": summary, "gates": gates}, ensure_ascii=False,
    indent=2, default=float))
print("elapsed_s", round(time.perf_counter() - t0, 1))

if __name__ == "__main__":
    main()

```

Listing 7: 问题 4_ 求解.py

```

from __future__ import annotations

import argparse
import json
import os
import time
from multiprocessing import Pool

import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

KEYS = ("P", "Q", "X", "Y", "U", "W", "V", "S", "cash", "cash_plan",
    "cash_contract", "plan_x", "plan_y", "plan_s_next")
BASES = ("prev_day", "mean7", "mean30", "median30")
BASE_LABEL = {"prev_day": "昨日同时段", "mean7": "近7日均值",
    "mean30": "近30日均值", "median30": "近30日中位数",
    "oracle": "0时已知真价 (理想信息)"}
DEFAULT_BASE = "median30"
FULL_SET = (0, 1, 2, 3)
UPDATES2 = (1, 2, 3)
SPIKE_Q = 0.75
OFFSET = 31
NTOT = 365

```

```

def build_configs():
    cfigs = []
    for base in BASES:
        cfigs.append({"tag": "Q4-2_" + base, "mode_note": "模式一(固定合同)",
            "base": base, "updates": (), "pv": "archive",
            "price_mode": "forecast", "keep": base == DEFAULT_BASE})
    for base in BASES:
        cfigs.append({"tag": "Q4-3_" + base, "mode_note": "模式二(可调合同)",
            "base": base, "updates": UPDATES2, "pv": "archive",
            "price_mode": "forecast", "keep": base == DEFAULT_BASE})
    cfigs.append({"tag": "Q4-2_oracle", "mode_note": "模式一(固定合同)",
        "base": "oracle", "updates": (), "pv": "archive",
        "price_mode": "oracle", "keep": True})
    cfigs.append({"tag": "Q4-3_oracle", "mode_note": "模式二(可调合同)",
        "base": "oracle", "updates": UPDATES2, "pv": "archive",
        "price_mode": "oracle", "keep": True})
    cfigs.append({"tag": "Q4-2_center_" + DEFAULT_BASE,
        "mode_note": "模式一(固定合同)", "base": DEFAULT_BASE,
        "updates": (), "pv": "center", "price_mode": "forecast",
        "keep": True})
    return cfigs

```

```
_G = {}
```

```
def _init(data, prof, n_days=NTOT):
```

```

    _G["data"] = data
    _G["prof"] = prof
    _G["n_days"] = n_days

```

```
def _worker(cfg):
```

```

    data, prof = _G["data"], _G["prof"]
    n_days = _G["n_days"]
    run = sl.run_year_q34(data, prof, "F1", 0.7725, 35, "stratified",
        6, FULL_SET,
        cfg["updates"], cfg["pv"], cfg["price_mode"],
        cfg["base"], "actual", 5.0, n_days)

    out = dict(cfg)
    out["metrics"] = sl.year_metrics_q34(run, OFFSET)
    out["gates"] = sl.check_run_q34(run, OFFSET)
    if cfg["keep"]:
        out["arrays"] = {k: run[k] for k in KEYS}
        out["stats"] = run["stats"]
        out["S_end"] = run["S_end"]
    return out

```

```
def price_error_table(data, offset=OFFSET, end=NTOT):
```

```

    A = data["price_act"]
    days = list(range(offset, end))
    spike = np.vstack([A[d] >= np.quantile(A[d], SPIKE_Q) for d in
        days])
    rows = []
    for base in BASES:
        e0, e12 = [], []
        for d in days:
            bp = mc.price_forecast_0h(d, A, data["price1"], base)
            e0.append(bp - A[d])
            e12.append(mc.price_intraday_correct(bp, A[d], 72) - A[d])

```

```

E0 = np.vstack(e0)
E12 = np.vstack(e12)
rows.append({
    "价格基线": base, "基线说明": BASE_LABEL[base],
    "MAE0_元每kWh": float(np.abs(E0).mean()),
    "RMSE0_元每kWh": float(np.sqrt((E0 ** 2).mean())),
    "Bias0_元每kWh": float(E0.mean()),
    "尖峰MAE0_元每kWh": float(np.abs(E0[spike]).mean()),
    "MAE12_元每kWh": float(np.abs(E12).mean()),
    "尖峰MAE12_元每kWh": float(np.abs(E12[spike]).mean()),
    "最大绝对误差0": float(np.abs(E0).max()),
})
return pd.DataFrame(rows)

```

```
def coverage_of(data, prof, alpha=0.7725, W=35, mode="stratified", d0=OFFSET,
```

```

    d1=NTOT):
    resid, pv_ver = sl.q3_net_residual(data, prof, FULL_SET, "F1", "archive")
    L, V = data["load_act"], data["pv_act"]
    R = np.empty((d1 - d0, mc.T))
    for i, d in enumerate(range(d0, d1)):
        lhat, _ = sl.predict_center("F1", d, data)
        res_kw, _ = sl.q3_reserve(d, resid, alpha, W, mode, FULL_SET)
        R[i] = (L[d] - V[d]) - (lhat - pv_ver[d] + res_kw)
    return {"Ncov": float((R <= 0.0).mean()),
        "pinball": float((R * (alpha - (R < 0.0))).mean()),
        "resid_MAE_kw": float(np.abs(resid[d0:d1]).mean())}

```

```
def slot_table(run, price_act, day_indices):
```

```

    rows = []
    for d in day_indices:
        c = price_act[d]
        for t in sl.REP_SLOTS:
            rows.append({
                "日期": str(mc.date_of(d)),
                "时段": "%s-%s" % (mc.slot_label_start(t), mc.slot_label_end(t)),
                "实际结算电价_元每kWh": float(c[t]),
                "负荷_kW": float(run["load_kw"][d][t]),
                "光伏实际_kW": float(run["pv_kw"][d][t]),
                "原计划合同_p_kWh": float(run["P"][d][t]),
                "最终合同_q_kWh": float(run["Q"][d][t]),
                "实际购电_kWh": float(run["Q"][d][t] - run["W"][d][t] + run["U"][d][t]),
                "应急购电_u_kWh": float(run["U"][d][t]),
                "充电量_kWh": float(run["X"][d][t]),
                "放电量_kWh": float(run["Y"][d][t]),
                "时段末储电量_kWh": float(run["S"][d][t]),
                "合同调整量_kWh": float(run["Q"][d][t] - run["P"][d][t])
            })
    return pd.DataFrame(rows)

```

```
def daily_table(run, offset=OFFSET):
```

```

    rows = []
    for d in range(offset, run["P"].shape[0]):
        dt = mc.date_of(d)
        rows.append({
            "日期": dt, "月份": dt.strftime("%Y-%m"),
            "现金流_元": run["cash"][d],
            "计划购电量_kWh": run["P"][d].sum(),
            "最终合同量_kWh": run["Q"][d].sum(),
            "调整量_kWh": (run["Q"][d] - run["P"][d]).sum(),
            "应急购电量_kWh": run["U"][d].sum(),
            "充电量_kWh": run["X"][d].sum(),
            "放电量_kWh": run["Y"][d].sum(),
            "弃购量_kWh": run["W"][d].sum(),

```

```

        "弃光量_kWh": (run["pv_kwh"][d] - run["V"][d]).sum(),
        "日末储电量_kWh": run["S"][d][-1])
    return pd.DataFrame(rows)

def _jsonable(rec):
    out = {}
    for k, v in rec.items():
        if k in ("arrays", "stats"):
            continue
        if k == "metrics":
            v = {kk: vv for kk, vv in v.items() if kk != "daily_cash"}
            out[k] = v
    return out

def _numeric_ok(g: dict) -> bool:
    return bool(
        g["balance_max_kWh"] <= 1e-6 and g["overlap_max_kWh"] <= 1e-5
        and g["charge_emergency_max_kWh"] <= 1e-5
        and max(g[k] for k in ("bound_p", "bound_q", "bound_x", "bound_y",
                                "bound_v", "bound_u", "bound_w", "bound_S")
                ) <= 1e-7
        and g["soc_chain_max_kWh"] <= 1e-6
        and g["soc_end_mismatch_kWh"] <= 1e-6
        and g["settle_flow_max_yuan"] <= 1e-5)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--workers", type=int, default=0)
    ap.add_argument("--days", type=int, default=NTOT,
                    help="运行前 N 天; 小于 365 时只作 Lite 烟雾验证")
    ap.add_argument("--main-only", action="store_true",
                    help="只运行 Q4-2/Q4-3 的 median30 主配置")
    args = ap.parse_args()
    if not OFFSET < args.days <= NTOT:
        ap.error("--days 必须在 32..365 之间")
    t0 = time.perf_counter()
    sl.CSV.mkdir(parents=True, exist_ok=True)

    data = mc.load_inputs()
    prof = sl.build_pv_archive(data, 35)
    cfigs = build_configs()
    if args.main_only:
        keep_tags = {"Q4-2_" + DEFAULT_BASE, "Q4-3_" + DEFAULT_BASE}
        cfigs = [c for c in cfigs if c["tag"] in keep_tags]
    workers = args.workers or min(len(cfigs), max(1, (os.cpu_count() or 4) - 4))
    print("[Q4] configs=%d workers=%d" % (len(cfigs), workers), flush=True)

    if workers <= 1:
        _init(data, prof, args.days)
        records = [_worker(c) for c in cfigs]
    else:
        with Pool(workers, initializer=_init, initargs=(data, prof, args.days)) as pool:
            records = pool.map(_worker, cfigs)
    print("[Q4] runs done in %.0fs" % (time.perf_counter() - t0), flush=True)

    err = price_error_table(data, end=args.days)
    err.to_csv(sl.CSV / "问题4_价格基线误差.csv", index=False,
              encoding="utf-8-sig")
    err_map = dict(zip(err["价格基线"], err["MAEO_元每kWh"]))
    cov = coverage_of(data, prof, d1=args.days)

    rows = []
    for rec in records:
        m = rec["metrics"]
        row = {"方案": rec["mode_note"], "tag": rec["tag"],
              "价格基线": BASE_LABEL.get(rec["base"], rec["base"]),
              "光伏预报源": rec["pv"], "定价信息": rec["price_mode"]}
        row.update({k: v for k, v in m.items() if k != "daily_cash"})
        row["价格MAEO_元每kWh"] = float(
            err_map.get(rec["base"], 0.0 if rec["base"] == "oracle"
                        else np.nan))
        row["安全轨迹覆盖率"] = cov["Ncov"]
        rows.append(row)
    cmp_df = pd.DataFrame(rows)
    cmp_df.to_csv(sl.CSV / "问题4_方案对比.csv", index=False,
                  encoding="utf-8-sig")

    gate_report = {r["tag"]: dict(r["gates"], coverage_Ncov=cov["Ncov"]
                                   ),
                   all_numeric_ok=_numeric_ok(r["gates"]))
    for r in records:
        gate_report["all_numeric_ok"] = bool(
            all(v["all_numeric_ok"] for k, v in gate_report.items()
                if not k.startswith("_")))
    with open(sl.CSV / "问题4_门禁.json", "w", encoding="utf-8") as fh:
        json.dump(gate_report, fh, ensure_ascii=False, indent=2,
                  default=float)

    by_tag = {r["tag"]: r for r in records}
    prim2 = by_tag["Q4-2_" + DEFAULT_BASE]
    prim3 = by_tag["Q4-3_" + DEFAULT_BASE]
    common = {"load_kwh": data["load_act"] * mc.DT,
              "pv_kwh": data["pv_act"] * mc.DT,
              "load_kw": data["load_act"], "pv_kw": data["pv_act"],
              "cash_plan": None, "cash_contract": None}
    day_idx = [sl.DAY_INDEX[d] for d in sl.REP_DATES
               if sl.DAY_INDEX[d] < args.days]
    price_act = data["price_act"]
    for prim, key in ((prim2, "问题4-2"), (prim3, "问题4-3")):
        run = dict(prim["arrays"])
        for k, v in common.items():
            if v is not None:
                run[k] = v
        slot_table(run, price_act, day_idx).to_csv(
            sl.CSV / (key + "_表1_指定时段.csv"), index=False,
            encoding="utf-8-sig")
        sl.energy_block_table(run, day_idx).to_csv(
            sl.CSV / (key + "_表2_四小时聚合.csv"), index=False,
            encoding="utf-8-sig")
        sl.emergency_table(run, day_idx).to_csv(
            sl.CSV / (key + "_表3_紧急购电.csv"), index=False,
            encoding="utf-8-sig")
        daily_table(run).to_csv(sl.CSV / (key + "_逐日汇总.csv"), index=False,
                                encoding="utf-8-sig")

    run2 = dict(prim2["arrays"])
    run2.update({k: v for k, v in common.items() if v is not None})
    run3 = dict(prim3["arrays"])
    run3.update({k: v for k, v in common.items() if v is not None})
    if args.days == NTOT:
        sl.write_workbook("result4-2.xlsx", "result4-2.xlsx", {
            "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run2, key="P",
                                                           fee_key="cash_plan"),
            "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run2),
            "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run2),
        })
        sl.write_workbook("result4-3.xlsx", "result4-3.xlsx", {
            "计划购电量": lambda ws: sl._fill_plan_sheet(ws, run3, key="P",
                                                           fee_key="cash_plan"),
            "调整购电量": lambda ws: sl._fill_plan_sheet(ws, run3, key="Q",
                                                           fee_key="cash_contract")
        })

```

```

        "充放电量": lambda ws: sl._fill_cycle_sheet(ws, run3),
        "紧急购电量": lambda ws: sl._fill_emergency_sheet(ws, run3),
    })

    np.savez_compressed(sl.CSV / "q4_bundle.npz",
        **{("%s_%s" % (r["tag"], k)): r["arrays"][k]
           for r in records if "arrays" in r for k in
           KEYS})
    with open(sl.CSV / "q4_metrics.json", "w", encoding="utf-8") as fh:
        :
        json.dump({"configs": [_jsonable(r) for r in records],
                    "price_error": err.to_dict("records"), "coverage":
                    cov},
                  fh, ensure_ascii=False, indent=2, default=float)

    print(cmp_df[["tag", "cash_total", "emergency_kwh", "updates"
                  ]].to_string())
    summary = {"main_base": DEFAULT_BASE,
               "Q4-2_cash": float(prim2["metrics"]["cash_total"]),
               "Q4-3_cash": float(prim3["metrics"]["cash_total"]),
               "Q4-2_emergency_kWh": float(prim2["metrics"]["emergency_
               kWh"]),
               "Q4-3_emergency_kWh": float(prim3["metrics"]["emergency_
               kWh"]),
               "all_numeric_ok": gate_report["all_numeric_ok"]}
    print(json.dumps(summary, ensure_ascii=False, indent=2, default=
    float))
    print("elapsed_s", round(time.perf_counter() - t0, 1))

if __name__ == "__main__":
    main()

```

Listing 8: 问题 3_ 其他时刻增量.py

```

from __future__ import annotations

import argparse
import json
import os
from pathlib import Path
from multiprocessing import Pool

os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "results" / "csv"
KEYS = ("P", "Q", "X", "Y", "U", "W", "V", "S", "cash", "cash_plan",
        "cash_contract", "plan_x", "plan_y", "plan_s_next")

def make_proxy_profiles(data, W=35):
    old_slots, old_hours, old_n = sl.ISSUE_SLOTS, sl.ISSUE_HOURS, sl.N_
    _ISSUE
    base = sl.build_pv_archive(data, W)
    candidate_hours = (3, 9, 15, 21)
    candidate_slots = tuple(6 * h for h in candidate_hours)

    source = (0, 1, 2, 3)
    proxy = np.stack([base[:, r, :] for r in source], axis=1)
    prof = np.concatenate([base, proxy], axis=1)
    sl.ISSUE_SLOTS = old_slots + candidate_slots
    sl.ISSUE_HOURS = old_hours + candidate_hours
    sl.N_ISSUE = old_n + 4

```

```

    return prof, old_slots, old_hours, old_n

def restore(state):
    sl.ISSUE_SLOTS, sl.ISSUE_HOURS, sl.N_ISSUE = state

def run_case(data, prof, allowed, updates, n_days):
    run = sl.run_year_q34(data, prof, "F1", 0.7725, 35, "stratified",
        6,
        allowed, updates, "archive", "fixed", "median30",
        "fixed", 5.0, n_days)
    return run, sl.year_metrics_q34(run, 31)

def load_existing_baseline(n_days):
    path = OUT / "q3_bundle.npz"
    if not path.exists():
        raise FileNotFoundError("缺少已有问题三结果: %s" % path)
    with np.load(path) as z:
        cash = np.asarray(z["main_cash"][:n_days], dtype=float)
        emergency = np.asarray(z["main_U"][:n_days], dtype=float)
    if n_days <= 31:
        raise ValueError("--days 必须大于 31")
    return {
        "daily_cash": cash[31:],
        "cash_total": float(cash[31:].sum()),
        "emergency_kwh": float(emergency[31:].sum()),
    }

def candidate_worker(job):
    hour, r, n_days = job
    data = mc.load_inputs()
    prof, *state = make_proxy_profiles(data, 35)
    try:
        _, metrics = run_case(
            data, prof, (0, 1, 2, 3, r), (1, 2, 3, r), n_days
        )
        return hour, metrics
    finally:
        restore(state)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--days", type=int, default=365)
    ap.add_argument("--workers", type=int, default=4)
    args = ap.parse_args()
    OUT.mkdir(parents=True, exist_ok=True)
    base_m = load_existing_baseline(args.days)
    jobs = [(hour, r, args.days)
             for hour, r in zip((3, 9, 15, 21), (4, 5, 6, 7))]
    workers = max(1, min(args.workers, len(jobs)))
    print("[Q3-extra] C++ HiGHS scenarios=%d workers=%d days=%d"
          % (len(jobs), workers, args.days), flush=True)
    if workers == 1:
        results = [candidate_worker(job) for job in jobs]
    else:
        with Pool(workers) as pool:
            results = pool.map(candidate_worker, jobs)

    rows = []
    for hour, m in results:
        daily_base = np.asarray(base_m["daily_cash"])
        daily_new = np.asarray(m["daily_cash"])
        saving = daily_base - daily_new
        rows.append({
            "候选时刻_h": hour,
            "对照方案": "{0,6,12,18}",
            "增强方案": "{0,6,12,18,%d}" % hour,
            "代理来源": "此前最近官方版本",
            "delta_n": 1,

```

```

"现金总费_对照_元": base_m["cash_total"],
"现金总费_增强_元": m["cash_total"],
"全年节省_元": float(base_m["cash_total"] - m["cash_
    total"]),

"kappa_star_元每次": float(saving.mean()),
"日均节省_元": float(saving.mean()),
"节省为正天数": int((saving > 1e-8).sum()),
"验证天数": int(saving.size),
"多数验证日节省为正": bool((saving > 1e-8).sum() > saving
    .size / 2),
"应急购电_对照_kWh": base_m["emergency_kwh"],
"应急购电_增强_kWh": m["emergency_kwh"],
"调整事件数_增强": m["updates"],
"结论": (
    "数值正收益, 但夜间无光伏, 不建议实际引入" if hour ==
        21 else
        "代理情景下可谨慎考虑" if (
            base_m["cash_total"] > m["cash_total"] and
            (saving > 1e-8).sum() > saving.size / 2
        ) else "代理情景下不建议引入"
    ),
))
df = pd.DataFrame(rows).sort_values("候选时刻_h")
df.to_csv(OUT / "问题3_其他时刻单次边际测试.csv", index=False,
    encoding="utf-8-sig")
meta = {
    "status": "proxy_scenario_only",
    "engine": "SciPy linprog + compiled C++ HiGHS, scenarios
        parallelized",
    "workers": workers,
    "candidate_hours": [3, 9, 15, 21],
    "proxy_rule": "each candidate uses the corrected profile of the
        previous official issue time",
    "base_metrics": {k: v for k, v in base_m.items() if k != "daily
        _cash"},
    "results": rows,
}
with open(OUT / "问题3_其他时刻单次边际测试.json", "w", encoding="
    utf-8") as f:
    json.dump(meta, f, ensure_ascii=False, indent=2, default=float)
print(df.to_string(index=False))
print("output:", OUT / "问题3_其他时刻单次边际测试.csv")

if __name__ == "__main__":
    main()

```

Listing 9: 补充敏感性计算.py

```

from __future__ import annotations

import argparse
import json
import os
import time
from multiprocessing import Pool

import pandas as pd

import microgrid_core as mc
import solution_lib as sl

BASE = {
    "ETA_C": mc.ETA_C, "ETA_D": mc.ETA_D, "E_MAX": mc.E_MAX,
    "S_MIN": mc.S_MIN, "S_MAX": mc.S_MAX, "P_MAX": mc.P_MAX,
    "B_MAX": mc.B_MAX, "SO": mc.SO,
}

def _set_parameters(cfg: dict) -> None:
    for key, value in BASE.items():
        setattr(mc, key, value)

if "eta" in cfg:
    mc.ETA_C = mc.ETA_D = float(cfg["eta"])
if "capacity_factor" in cfg:
    factor = float(cfg["capacity_factor"])
    mc.E_MAX = BASE["E_MAX"] * factor
    mc.S_MIN = 0.1 * mc.E_MAX
    mc.S_MAX = 0.9 * mc.E_MAX
if "power_factor" in cfg:
    factor = float(cfg["power_factor"])
    mc.P_MAX = BASE["P_MAX"] * factor
    mc.B_MAX = mc.P_MAX * mc.DT
if "soc_ratio" in cfg:
    mc.SO = float(cfg["soc_ratio"]) * mc.E_MAX

def _worker(cfg: dict) -> dict:
    _set_parameters(cfg)
    data = mc.load_inputs()
    resid = sl.net_residual_matrix("F1", data)
    gamma = float(cfg.get("gamma", 1.0))
    run = sl.run_year_q2(data, "F1", 0.805, 35, resid=resid * gamma,
        horizon=49, n_days=365, executor="mpc")
    metrics = sl.year_metrics(run)
    gates = sl.check_run(run)
    return {
        **cfg,
        **{k: v for k, v in metrics.items() if k != "daily_cash"},
        "eta_c": mc.ETA_C, "eta_d": mc.ETA_D,
        "capacity_kWh": mc.E_MAX, "power_kW": mc.P_MAX,
        "S_min_kWh": mc.S_MIN, "S_max_kWh": mc.S_MAX,
        "SO_kWh": mc.SO,
        "numeric_ok": bool(
            gates["balance_max"] <= 1e-6
            and gates["overlap_max"] <= 1e-5
            and gates["bound_S"] <= 1e-7
        ),
    }

def configs() -> list[dict]:
    return [
        {"group": "基准", "scenario": "基准"},
        {"group": "初始SOC", "scenario": "初始SOC=20%", "soc_ratio":
            0.2},
        {"group": "初始SOC", "scenario": "初始SOC=80%", "soc_ratio":
            0.8},
        {"group": "容量", "scenario": "容量-20%", "capacity_factor":
            0.8},
        {"group": "容量", "scenario": "容量+20%", "capacity_factor":
            1.2},
        {"group": "功率", "scenario": "功率-20%", "power_factor": 0.8},
        {"group": "功率", "scenario": "功率+20%", "power_factor": 1.2},
        {"group": "误差倍率", "scenario": "gamma=0.5", "gamma": 0.5},
        {"group": "误差倍率", "scenario": "gamma=1.5", "gamma": 1.5},
        {"group": "误差倍率", "scenario": "gamma=2.0", "gamma": 2.0},
    ]

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--workers", type=int, default=0)
    args = ap.parse_args()
    cfgs = configs()
    workers = args.workers or min(4, max(1, (os.cpu_count() or 4) - 2))
    t0 = time.perf_counter()
    print(f"[SENS] scenarios={len(cfgs)} workers={workers}", flush=
        True)
    if workers <= 1:
        rows = [_worker(c) for c in cfgs]
    else:
        with Pool(workers) as pool:
            rows = pool.map(_worker, cfgs)

```

```

base = next(r for r in rows if r["scenario"] == "基准")
for group, scenario, extra in [
    ("初始SOC", "初始SOC=50%", {"soc_ratio": 0.5}),
    ("误差倍率", "gamma=1.0", {"gamma": 1.0}),
]:
    alias = dict(base)
    alias.update(group=group, scenario=scenario, **extra)
    rows.append(alias)
df = pd.DataFrame(rows)
base_cost = float(df.loc[df["scenario"] == "基准", "cash_total"].iloc[0])
df["相对基准费用变化_元"] = df["cash_total"] - base_cost
df["相对基准费用变化_pct"] = (df["cash_total"] / base_cost - 1.0) * 100.0
sl.CSV.mkdir(parents=True, exist_ok=True)
common = ["group", "scenario", "eta_c", "eta_d", "capacity_kWh",
    "power_kW", "SO_kWh", "cash_total", "相对基准费用变化_元",
    "相对基准费用变化_pct", "contract_kwh", "emergency_kwh",
    "emergency_slots", "charge_kwh", "discharge_kwh",
    "unused_contract_kwh", "S_min", "S_max", "S_end", "numeric_ok"]
structural = df[df["group"] != "误差倍率"].copy()
pressure = df[df["group"] == "误差倍率"][["gamma"] + common].copy()
structural.to_csv(sl.CSV / "模型评价_结构敏感性.csv", index=False,
    encoding="utf-8-sig")
pressure.to_csv(sl.CSV / "模型评价_误差倍率压力测试.csv", index=False,
    encoding="utf-8-sig")
with pd.ExcelWriter(mc.RESULT_DIR / "补充分析结果.xlsx", engine="openpyxl") as writer:
    structural.to_excel(writer, sheet_name="结构敏感性", index=False)
    pressure.to_excel(writer, sheet_name="误差倍率压力测试", index=False)
summary = {
    "scenario_count": len(df),
    "all_numeric_ok": bool(df["numeric_ok"].all()),
    "elapsed_s": round(time.perf_counter() - t0, 1),
}
with open(sl.CSV / "模型评价_补充计算门禁.json", "w", encoding="utf-8") as fh:
    json.dump(summary, fh, ensure_ascii=False, indent=2)
print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)

if __name__ == "__main__":
    main()

```

Listing 10: 终端价值敏感性.py

```

from __future__ import annotations

import json
import time

import pandas as pd

import microgrid_core as mc
import solution_lib as sl

def main() -> None:
    t0 = time.perf_counter()
    data = mc.load_inputs()
    resid = sl.net_residual_matrix("F1", data)
    rows = []
    for factor in (0.0, 0.5, 1.0, 1.5):
        run = sl.run_year_q2(data, "F1", 0.805, 35, resid=resid,
            horizon=49, n_days=365, executor="mpc",
            terminal_value_factor=factor)
        m = sl.year_metrics(run)

```

```

g = sl.check_run(run)
rows.append({
    "终端价值倍率": factor,
    "现金费用_元": m["cash_total"],
    "合同购电量_kWh": m["contract_kwh"],
    "应急购电量_kWh": m["emergency_kwh"],
    "充电量_kWh": m["charge_kwh"],
    "放电量_kWh": m["discharge_kwh"],
    "弃购量_kWh": m["unused_contract_kwh"],
    "期末SOC_kWh": m["S_end"],
    "SOC下限触及比例": float((run["S"] <= mc.S_MIN + 1e-7).mean()),
    "SOC上限触及比例": float((run["S"] >= mc.S_MAX - 1e-7).mean()),
    "数值门禁": bool(g["balance_max"] <= 1e-6 and
        g["overlap_max"] <= 1e-5 and
        g["bound_S"] <= 1e-7 and
        g["soc_chain_max_kWh"] <= 1e-6),
})
df = pd.DataFrame(rows)
base = float(df.loc[df["终端价值倍率"] == 1.0, "现金费用_元"].iloc[0])
df["相对倍率1费用变化_元"] = df["现金费用_元"] - base
df["相对倍率1费用变化_pct"] = (df["现金费用_元"] / base - 1.0) * 100.0
sl.CSV.mkdir(parents=True, exist_ok=True)
df.to_csv(sl.CSV / "终端价值敏感性.csv", index=False, encoding="utf-8-sig")
report = {"scenario_count": len(df), "all_numeric_ok": bool(df["数值门禁"].all()),
    "elapsed_s": round(time.perf_counter() - t0, 1)}
with open(sl.CSV / "终端价值敏感性_门禁.json", "w", encoding="utf-8") as fh:
    json.dump(report, fh, ensure_ascii=False, indent=2)
print(json.dumps(report, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

Listing 11: 收口计算.py

```

from __future__ import annotations

import json
import math
import time
from pathlib import Path

import numpy as np
import pandas as pd

import microgrid_core as mc
import solution_lib as sl

ROOT = Path(__file__).resolve().parent
CSV = ROOT / "results" / "csv"
OFF = 31
END = 365

def load_npz(name):
    return {k: v for k, v in np.load(CSV / name).items()}

def fixed_price(data):
    return np.tile(data["price1"], (END, 1))

def actual_price(data):
    return data["price_act"]

```

```

def fee_parts(P, Q, U, price, q3):
    P, Q, U, price = map(lambda a: np.asarray(a, dtype=float), (P, Q,
        U, price))
    if price.ndim == 1:
        price = np.tile(price, (P.shape[0], 1))
    over = np.maximum(Q - P, 0.0)
    under = np.maximum(P - Q, 0.0)
    plan_a = (price * np.minimum(P, Q)).sum(axis=1)
    adj = (1.5 * price * over + 0.5 * price * under).sum(axis=1) if q3
        else np.zeros(P.shape[0])
    plan_b = (price * P).sum(axis=1)
    emg = (5.0 * price * U).sum(axis=1)
    return {"plan_A": plan_a, "plan_B": plan_b, "adjust": adj, "
        emergency": emg,
        "total_A": plan_a + adj + emg, "total_B": plan_b + adj +
            emg}

def diag_row(label, arrays, price, q3, data):
    parts = fee_parts(arrays["P"], arrays["Q"] if q3 else arrays["P"],
        arrays["U"], price, q3)
    X, Y, S = arrays["X"], arrays["Y"], arrays["S"]
    plan_x = arrays.get("plan_x", np.full_like(X, np.nan))
    plan_y = arrays.get("plan_y", np.full_like(Y, np.nan))
    plan_s = arrays.get("plan_s_next", np.full_like(S, np.nan))
    slc = slice(OFF, END)
    soc_e = S[slc]
    soc_p = plan_s[slc]
    finite = np.isfinite(soc_p)
    e = soc_e[finite] - soc_p[finite]
    cash = parts["total_A"][slc]
    row = {
        "方案": label, "口径A现金总费用_元": float(parts["total_A"][slc].
            sum()),
        "计划费用A_元": float(parts["plan_A"][slc].sum()),
        "调整费用_元": float(parts["adjust"][slc].sum()),
        "紧急费用_元": float(parts["emergency"][slc].sum()),
        "口径B现金总费用_元": float(parts["total_B"][slc].sum()),
        "口径B相对A_元": float((parts["total_B"] - parts["total_A"])[
            slc].sum()),
        "计划购电量_kWh": float(arrays["P"][slc].sum()),
        "最终合同量_kWh": float((arrays["Q"] if q3 else arrays["P"])[
            slc].sum()),
        "调整量_kWh": float(((arrays["Q"] - arrays["P"]) if q3 else np.
            zeros_like(arrays["P"]))[slc].sum()),
        "应急购电量_kWh": float(arrays["U"][slc].sum()),
        "弃购量_kWh": float(arrays["W"][slc].sum()),
        "弃光量_kWh": float((data["pv_act"] * mc.DT - arrays["V"])[slc
            ].sum()),
        "充电量_kWh": float(X[slc].sum()), "放电量_kWh": float(Y[slc].
            sum()),
        "期末SOC_kWh": float(S[-1, -1]),
        "SOC一步MAE_kWh": float(np.abs(e).mean()) if e.size else np.nan
            ,
        "SOC一步RMSE_kWh": float(np.sqrt(np.mean(e * e))) if e.size
            else np.nan,
        "SOC一步最大绝对误差_kWh": float(np.abs(e).max()) if e.size else
            np.nan,
        "充电一步MAE_kWh": float(np.abs(X[slc][finite] - plan_x[slc][
            finite]).mean()) if e.size else np.nan,
        "放电一步MAE_kWh": float(np.abs(Y[slc][finite] - plan_y[slc][
            finite]).mean()) if e.size else np.nan,
        "费用加总误差_元": float(np.max(np.abs(parts["total_A"] - (
            parts["plan_A"] + parts["adjust"] + parts["emergency"])))
            [slc])),
    }
    return row

def settlement_sensitivity(data, q3, q4):
    rows = []
    q3_main = {k[len("main-")]: v for k, v in q3.items()
        if k.startswith("main-")}
    q4_main = {k[len("Q4-3_median30-")]: v for k, v in q4.items()
        if k.startswith("Q4-3_median30-")}
    specs = [
        ("Q3主方案", q3_main, fixed_price(data), True),
        ("Q4-3_median30", q4_main, actual_price(data), True),
    ]
    for tag in ("main", "abl_none", "abl_1", "abl_2", "abl_3", "abl_1_
        2", "abl_1_3", "abl_2_3"):
        a = {k[len(tag)+1:]: v for k, v in q3.items() if k.startswith(
            tag + "-")}
        p = fee_parts(a["P"], a["Q"], a["U"], fixed_price(data), True)
        rows.append({"问题": "Q3", "方案": tag, "口径A总费用_元": p["total_A"
            ] [OFF:END].sum(),
            "口径B总费用_元": p["total_B"] [OFF:END].sum(),
            "口径B-A_元": (p["total_B"] - p["total_A"]) [OFF:END].
                sum()})
        for label, a, price, q3flag in specs:
            p = fee_parts(a["P"], a["Q"] if q3flag else a["P"], a["U"],
                price, q3flag)
            rows.append({"问题": label.split("-")[0], "方案": label,
                "口径A总费用_元": p["total_A"] [OFF:END].sum(),
                "口径B总费用_元": p["total_B"] [OFF:END].sum(),
                "口径B-A_元": (p["total_B"] - p["total_A"]) [OFF:END].
                    sum()})
        df = pd.DataFrame(rows)
        df["口径A排序"] = df.groupby("问题")["口径A总费用_元"].rank(method="
            min")
        df["口径B排序"] = df.groupby("问题")["口径B总费用_元"].rank(method="
            min")
        return df

def tail_risk(label, daily, dates):
    x = np.asarray(daily, dtype=float) [OFF:END]
    q = float(np.quantile(x, 0.95, method="linear"))
    tail = x[x >= q]
    i = int(np.argmax(x))
    d = pd.Timestamp(str(dates[OFF:END][i])).date()
    return {"方案": label, "日均费用_元": float(x.mean()), "VaR95_元":
        q,
        "CVaR95_元": float(tail.mean()), "最坏日费用_元": float(x.
            max()),
        "最坏日": str(d), "最坏5日均费用_元": float(np.sort(x)[-5:].
            mean())}

def monthly_pair(label_a, a, label_b, b, dates):
    da = pd.DataFrame({"日期": pd.to_datetime(dates[OFF:END]), "a": a[
        OFF:END], "b": b[OFF:END]})
    da["月份"] = da["日期"].dt.strftime("%Y-%m")
    out = da.groupby("月份").agg(方案A日均费用_元=("a", "mean"), 方案B
        日均费用_元=("b", "mean"),
        天数=("a", "size")).reset_index()
    out["A相对B日均差_元"] = out["方案A日均费用_元"] - out["方案B日均费
        用_元"]
    out.insert(0, "方案A", label_a); out.insert(1, "方案B", label_b)
    return out

def time_audit(data, q2, q3, q4):
    cols = mc._TIME_COLS
    checks = []
    for i, c in enumerate(cols):
        checks.append({"时段序号": i + 1, "原始列标签": c, "自然区间": f
            "{mc.slot_label_start(i)}-{mc.slot_label_end(i)}",
            "是否首时段": i == 0, "是否末时段": i == mc.T - 1})
    df = pd.DataFrame(checks)
    boundary = pd.DataFrame([
        {"审计项": "附件2首列", "结果": cols[0], "判定": "第1时段按0
            :00-0:10解释"},
        {"审计项": "附件2末列", "结果": cols[-1], "判定": "第144时段按23
            :50-24:00解释"},
        {"审计项": "官方发布时间段", "结果": "0,36,72,108", "判定": "对应0
            /6/12/18时刻"},
    ])

```

```

        {"审计项": "重点时段", "结果": "60,72,84,96,108,120", "判定": "对应10:00至20:00起始时段"},
        {"审计项": "跨午夜应急合并", "结果": "不跨午夜", "判定": "连续区间按自然日截断"},
    ])

rows = []
dates = np.array([str(mc.date_of(d)) for d in range(END)])
for label, a, price, q3flag in [(("Q2", q2, np.tile(data["price1"], (END,1)), False),
                                ("Q3", q3, np.tile(data["price1"], (END,1)), True))]:
    for shift in (-1, 0, 1):
        pp = np.roll(price, shift, axis=1)
        p = fee_parts(a["P"], a["Q"] if q3flag else a["P"], a["U"], pp, q3flag)
        rows.append({"方案": label, "价格整体平移时段数": shift, "重放费用_元": p["total_A"] [OFF:END].sum()})
audit = pd.DataFrame(rows)
baseline = audit.loc[audit["价格整体平移时段数"] == 0].set_index("方案")["重放费用_元"]
audit["相对原映射_元"] = audit["重放费用_元"] - audit["方案"].map(baseline)
return pd.concat([df, boundary, audit], ignore_index=True)

def main():
    t0 = time.perf_counter()
    data = mc.load_inputs()
    q2 = load_npz("q2_bundle.npz")
    q3 = load_npz("q3_bundle.npz")
    q4 = load_npz("q4_bundle.npz")
    dates = np.array([str(mc.date_of(d)) for d in range(END)])
    q2a = {k: v for k, v in q2.items()}
    q3a = {k[len("main_")]: v for k, v in q3.items() if k.startswith("main_")}
    q4a = {k[len("Q4-3_median30_")]: v for k, v in q4.items() if k.startswith("Q4-3_median30_")}
    q4b = {k[len("Q4-2_median30_")]: v for k, v in q4.items() if k.startswith("Q4-2_median30_")}

    main_rows = [diag_row("Q2-F1-W35-alpha0.805-H49", q2a, np.tile(data["price1"], (END,1)), False, data),
                  diag_row("Q3-S={6,12,18}", q3a, np.tile(data["price1"], (END,1)), True, data),
                  diag_row("Q4-2-median30", q4b, data["price_act"], False, data),
                  diag_row("Q4-3-median30", q4a, data["price_act"], True, data)]
    pd.DataFrame(main_rows).to_csv(CSV / "总体主表_费用与执行诊断.csv", index=False, encoding="utf-8-sig")

    settle = settlement_sensitivity(data, q3, q4)
    settle.to_csv(CSV / "结算歧义敏感性.csv", index=False, encoding="utf-8-sig")
    time_audit(data, q2a, q3a, q4a).to_csv(CSV / "时间标签索引审计.csv", index=False, encoding="utf-8-sig")

    rows = []
    for label, a, price, q3flag in [(("Q2", q2a, np.tile(data["price1"], (END,1)), False),
                                    ("Q3", q3a, np.tile(data["price1"], (END,1)), True),
                                    ("Q4-2", q4b, data["price_act"], False),
                                    ("Q4-3", q4a, data["price_act"], True))]:
        for date in sl.REP_DATES:
            d = (date - mc.DAY0).days
            if d >= END: continue
            c = price[d] if price.ndim == 2 else price
            Q = a["Q"] if q3flag else a["P"]
            rows.append({"方案": label, "日期": str(date), "全天购电费用_

```

```

元": float(fee_parts(a["P"][d:d+1], Q[d:d+1], a["U"][d:d+1], c, q3flag) ["total_A"] [0]),
        "计划购电量_kWh": a["P"][d].sum(), "最终合同量_kWh": Q[d].sum(), "充电量_kWh": a["X"][d].sum(),
        "放电量_kWh": a["Y"][d].sum(), "应急购电量_kWh": a["U"][d].sum(), "弃购量_kWh": a["W"][d].sum(),
        "弃光量_kWh": (data["pv_act"][d]*mc.DT-a["V"][d]).sum(), "0时SOC_kWh": (a["S"][d-1,-1] if d < 24 else mc.S0),
        "24时SOC_kWh": a["S"][d,-1]})
pd.DataFrame(rows).to_csv(CSV / "重点日期_统一诊断表.csv", index=False, encoding="utf-8-sig")

risk = pd.DataFrame([
    tail_risk("Q2-F1-W35-alpha0.805-H49", q2["cash"], dates),
    tail_risk("Q3-S={6,12,18}", q3["main_cash"], dates),
    tail_risk("Q4-2-median30", q4["Q4-2_median30_cash"], dates),
    tail_risk("Q4-3-median30", q4["Q4-3_median30_cash"], dates),
])
risk.to_csv(CSV / "尾部风险_日费用VaR_CVaR.csv", index=False, encoding="utf-8-sig")
monthly_pair("Q3-S={6,12,18}", q3["main_cash"], "Q3-空集", q3["abl_none_cash"], dates).to_csv(CSV / "尾部风险_月度配对差.csv", index=False, encoding="utf-8-sig")

checks = {"finite": True, "curtail_nonnegative_max_violation_kWh": 0.0,
          "fee_decomposition_max_error_yuan": 0.0, "required_files": [], "elapsed_s": 0.0}
for a in (q2a, q3a, q4a, q4b):
    for k, v in a.items():
        if isinstance(v, np.ndarray) and not np.isfinite(v).all():
            checks["finite"] = False
        checks["curtail_nonnegative_max_violation_kWh"] = max(checks["curtail_nonnegative_max_violation_kWh"], float(np.maximum(a["V"] - data["pv_act"] * mc.DT, 0).max()))
    checks["required_files"] = [p.name for p in (CSV / "总体主表_费用与执行诊断.csv", CSV / "结算歧义敏感性.csv", CSV / "时间标签索引审计.csv", CSV / "重点日期_统一诊断表.csv", CSV / "尾部风险_日费用VaR_CVaR.csv", CSV / "尾部风险_月度配对差.csv") if p.exists()]
    checks["elapsed_s"] = round(time.perf_counter() - t0, 1)
    with open(CSV / "结果文件校验.json", "w", encoding="utf-8") as f:
        json.dump(checks, f, ensure_ascii=False, indent=2)
    print(json.dumps(checks, ensure_ascii=False, indent=2))

if __name__ == "__main__": main()

```

Listing 12: 汇总结果.py

```

from __future__ import annotations

import json
import math
from pathlib import Path

import numpy as np
import openpyxl
import pandas as pd

ROOT = Path(__file__).resolve().parent
CSV = ROOT / "results" / "csv"
RES = ROOT / "results"

def read(name: str) -> pd.DataFrame:
    return pd.read_csv(CSV / name)

```

```

def clean_tables() -> dict[str, pd.DataFrame]:
    q2s = read("问题2_配置敏感性.csv").drop(columns=["daily_cash"],
        errors="ignore")
    q2r = read("问题2_参照方案.csv").drop(columns=["daily_cash"],
        errors="ignore")
    q3a = read("问题3_信息消融.csv")
    q3a.loc[q3a["tag"] == "abl_none", "获准集合S"] = "{}"
    q3c = read("问题3_方案对比.csv")
    q4e = read("问题4_价格基线误差.csv")
    q4c = read("问题4_方案对比.csv")
    st = read("模型评价_结构敏感性.csv")
    ps = read("模型评价_误差倍率压力测试.csv")
    tv = read("终端价值敏感性.csv")
    diag = read("总体主表_费用与执行诊断.csv")
    settle = read("结算歧义敏感性.csv")
    risk = read("尾部风险_日费用VaR_CVaR.csv")
    month = read("尾部风险_月度配对差.csv")
    common = ["group", "scenario", "eta_c", "eta_d", "capacity_kWh",
        "power_kW", "S0_kWh", "cash_total", "相对基准费用变化_元",
        "相对基准费用变化_pct", "contract_kwh", "emergency_kwh",
        "emergency_slots", "charge_kwh", "discharge_kwh",
        "unused_contract_kwh", "S_min", "S_max", "S_end", "numeric_
            _ok"]
    st = st[common]
    ps = ps[["gamma"] + common]
    return {"Q2参数敏感性": q2s, "Q2参照方案": q2r,
        "Q3八组信息消融": q3a, "Q3方案对比": q3c,
        "Q4价格预测误差": q4e, "Q4方案与理想基准": q4c,
        "结构敏感性": st, "误差倍率压力测试": ps,
        "终端价值敏感性": tv, "费用执行诊断": diag,
        "结算歧义敏感性": settle, "尾部风险": risk,
        "月度配对差": month}

def key_summary(tables: dict[str, pd.DataFrame]) -> pd.DataFrame:
    q1 = read("问题1_汇总.csv").iloc[0]
    q2 = tables["Q2参数敏感性"].query("方案.str.startswith('主配置')",
        engine="python").iloc[0]
    q3 = tables["Q3八组信息消融"].query("tag == 'main'").iloc[0]
    q4 = tables["Q4方案与理想基准"].set_index("tag")
    return pd.DataFrame([
        {"问题": "Q1", "方案": "确定性LP", "现金费用_元": q1["全天购电费_
            元"],
        "合同购电量_kWh": q1["全天计划购电量_kWh"], "应急购电量_kWh":
            0.0},
        {"问题": "Q2", "方案": "F1-W35-alpha0.805-H49", "现金费用_元":
            q2["cash_total"],
        "合同购电量_kWh": q2["contract_kwh"], "应急购电量_kWh": q2["
            emergency_kwh"]},
        {"问题": "Q3", "方案": "S={6,12,18}", "现金费用_元": q3["现金总
            费_元"],
        "合同购电量_kWh": q3["最终合同量_kWh"], "应急购电量_kWh": q3["应
            急购电量_kWh"]},
        {"问题": "Q4-2", "方案": "median30固定合同", "现金费用_元": q4.
            loc["Q4-2_median30", "cash_total"],
        "合同购电量_kWh": q4.loc["Q4-2_median30", "contract_kwh"],
        "应急购电量_kWh": q4.loc["Q4-2_median30", "emergency_kwh"]},
        {"问题": "Q4-3", "方案": "median30可调合同", "现金费用_元": q4.
            loc["Q4-3_median30", "cash_total"],
        "合同购电量_kWh": q4.loc["Q4-3_median30", "contract_kwh"],
        "应急购电量_kWh": q4.loc["Q4-3_median30", "emergency_kwh"]},
    ])

def autosize(path: Path) -> None:
    wb = openpyxl.load_workbook(path)
    for ws in wb.worksheets:
        ws.freeze_panes = "A2"
        ws.auto_filter.ref = ws.dimensions
        for col in ws.columns:
            letter = col[0].column_letter
            width = min(28, max(10, max(len(str(c.value or "")) for c
                in col) + 2))
            ws.column_dimensions[letter].width = width

wb.save(path)

def validate_answer_workbooks() -> dict:
    names = ["result1.xlsx", "result2.xlsx", "result3.xlsx",
        "result4-2.xlsx", "result4-3.xlsx"]
    details = []
    all_ok = True
    for name in names:
        template = openpyxl.load_workbook(ROOT / "data" / "附件" / "附
            件5" / name,
            data_only=False)
        result = openpyxl.load_workbook(RES / name, data_only=False)
        same_sheets = template.sheetnames == result.sheetnames
        same_dimensions = all(template[s].calculate_dimension() ==
            result[s].calculate_dimension()
            for s in template.sheetnames)
        errors, nonfinite, annual_complete = [], [], True
        for ws in result.worksheets:
            for row in ws.iter_rows():
                for cell in row:
                    value = cell.value
                    if isinstance(value, str) and value.startswith("#"):
                        errors.append(f"{ws.title}!{cell.coordinate}:{
                            value}")
                    if isinstance(value, (int, float)) and not math.
                        isfinite(float(value)):
                        nonfinite.append(f"{ws.title}!{cell.coordinate}")
                    if ws.max_row == 335 and ws.max_column == 147:
                        annual_complete = annual_complete and all(
                            ws.cell(r, c).value is not None
                            for r in range(2, 336) for c in range(1, 148))
        ok = same_sheets and same_dimensions and not errors and not
            nonfinite and annual_complete
        all_ok = all_ok and ok
        details.append({"file": name, "same_sheets": same_sheets,
            "same_dimensions": same_dimensions,
            "annual_cells_complete": annual_complete,
            "excel_errors": errors, "nonfinite_cells":
                nonfinite,
            "ok": ok})
    return {"all_ok": all_ok, "details": details}

def main() -> None:
    tables = clean_tables()
    summary = key_summary(tables)
    summary.to_csv(CSV / "全题关键指标.csv", index=False, encoding="utf
        -8-sig")
    tables["Q2参数敏感性"].to_csv(CSV / "问题2_配置敏感性.csv", index=
        False, encoding="utf-8-sig")
    tables["Q2参照方案"].to_csv(CSV / "问题2_参照方案.csv", index=False
        , encoding="utf-8-sig")
    tables["Q3八组信息消融"].to_csv(CSV / "问题3_信息消融.csv", index=
        False, encoding="utf-8-sig")
    tables["结构敏感性"].to_csv(CSV / "模型评价_结构敏感性.csv", index=
        False, encoding="utf-8-sig")
    tables["误差倍率压力测试"].to_csv(CSV / "模型评价_误差倍率压力测试.
        csv", index=False, encoding="utf-8-sig")
    out = RES / "补充分析结果.xlsx"
    with pd.ExcelWriter(out, engine="openpyxl") as writer:
        summary.to_excel(writer, sheet_name="全题关键指标", index=False
            )
        for name, df in tables.items():
            df.to_excel(writer, sheet_name=name, index=False)
    autosize(out)
    answer_check = validate_answer_workbooks()
    report = {"workbook": str(out), "sheets": 1 + len(tables),
        "rows": {"全题关键指标": len(summary),
            **{k: len(v) for k, v in tables.items()}},
        "finite_numeric_nonmissing": bool(all(
            np.isfinite(a[-np.isnan(a)]).all()
            for df in [summary, *tables.values()]
            for a in [df.select_dtypes(include=[np.number]).to_

```

```

        numpy()))),
    "official_answer_files": ["result1.xlsx", "result2.xlsx",
                              "result3.xlsx",
                              "result4-2.xlsx", "result4-3.xlsx"
                              ],
    "official_answer_check": answer_check}
with open(CSV / "结果文件校验.json", "w", encoding="utf-8") as fh:
    json.dump(report, fh, ensure_ascii=False, indent=2)
    print(json.dumps(report, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```